HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN

BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN NĂM 2026

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MEET-ASSISTANT: TỰ ĐỘNG HÓA TAKE-NOTE VÀ ĐỒNG BỘ HÓA QUẢN LÝ CÔNG VIỆC

18-SV-2026-RIPT

Thuộc nhóm ngành khoa học: Công nghệ thông tin

Hà Nội – 2026

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN

BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN

NĂM 2026

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MEET-ASSISTANT: TỰ ĐỘNG HÓA TAKE-NOTE VÀ ĐỒNG BỘ HÓA QUẢN LÝ CÔNG VIỆC

18-SV-2026-RIPT

Thuộc nhóm ngành khoa học: Công nghệ thông tin

Sinh viên thực hiện	Giới tính	Dân tộc	Mã SV	Lớp	Khoa	Năm thứ	Số năm đào tạo	Ngành học
Trần Đức Định	…	…	B23DCCC038	D23CCPM01	…	4	…	…
Hoàng Nguyễn Trà My	…	…	B23DCCC116	D23CCPM02	…	4	…	…

Người hướng dẫn: TS. Lê Văn Hoàng

Hà Nội – 2026

MỤC LỤC

MỞ ĐẦU **1**
1. Tổng quan tình hình nghiên cứu 1
2. Lý do chọn đề tài 2
3. Mục tiêu đề tài 2
4. Phương pháp nghiên cứu 3
5. Đối tượng và phạm vi nghiên cứu 4
6. Bố cục báo cáo 4
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ **6**
1.1. Bài toán trợ lý cuộc họp 6
1.1.1. Phát biểu bài toán *6*
1.1.2. Đặc thù của hội thoại cuộc họp *6*
1.1.3. Ngữ liệu và hướng nghiên cứu liên quan *6*
1.1.4. Các giải pháp trợ lý cuộc họp hiện có *7*
1.2. Nhận dạng tiếng nói tự động 7
1.2.1. Các hướng tiếp cận *7*
1.2.2. Mô hình Whisper *8*
1.2.3. faster-whisper, CTranslate2 và lượng tử hoá int8 ..... *9*
1.2.4. Nhận diện ngôn ngữ và hiện tượng chuyển mã Việt–Anh *10*
1.2.5. Hiện tượng ảo giác của Whisper và cách hạn chế *10*
1.2.6. Độ đo đánh giá nhận dạng tiếng nói *10*
1.3. Tách người nói 11
1.3.1. Định nghĩa bài toán và độ đo DER *11*
1.3.2. Vector đặc trưng giọng nói *11*
1.3.3. Gom cụm phân cấp theo độ tương đồng cosine *12*
1.3.4. Vấn đề nói chồng *13*
1.4. Mô hình ngôn ngữ lớn trong trích xuất thông tin 13
1.4.1. Kiến trúc Transformer và mô hình ngôn ngữ lớn *13*
1.4.2. Mô hình suy luận *13*
1.4.3. Thiết kế prompt và đầu ra JSON có cấu trúc *14*
1.4.4. Ảo giác của LLM, đối chiếu bằng chứng và con người trong vòng lặp *14*
1.4.5. Dịch vụ suy luận và chủ quyền dữ liệu *15*
1.5. Kiến trúc microservices và hướng sự kiện 15
1.5.1. Microservices và cổng API *15*
1.5.2. Giao tiếp bất đồng bộ qua RabbitMQ *16*
1.5.3. Ngữ nghĩa giao nhận và tính lũy đẳng *16*
1.5.4. Mẫu transactional outbox *16*
1.6. Thu nhận nội dung họp trực tuyến 17
1.6.1. WebRTC và xử lý âm thanh trên trình duyệt *17*
1.6.2. Tự động hoá trình duyệt với Playwright và Xvfb *17*

1.6.3. Tiện ích mở rộng Chrome Manifest V3 ..... *17*
1.7. Tích hợp nền tảng quản lý công việc 18
1.7.1. OAuth 2.0 và Webhook *18*
1.7.2. Jira, Trello và Telegram *18*
1.8. Kiểm soát truy cập 19
1.8.1. Mô hình RBAC *19*
1.8.2. Mô hình ABAC *19*
1.8.3. Phân quyền theo vai trò kết hợp phạm vi tài nguyên *19*
1.9. Kết luận chương 20
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 21
2.1. Khảo sát và xác định yêu cầu 21
2.1.1. Đối tượng người dùng *21*
2.1.2. Quy trình họp mẫu *21*
2.2. Yêu cầu chức năng 22
2.3. Yêu cầu phi chức năng 23
2.4. Mô hình ca sử dụng 24
2.5. Kiến trúc tổng thể 25
2.5.1. Các thành phần *25*
2.5.2. Các nguyên tắc kiến trúc *27*
2.6. Luồng xử lý từ âm thanh đến công việc 28
2.6.1. Luồng tổng quát *28*
2.6.2. Hợp đồng sự kiện *29*
2.6.3. Trạng thái xử lý *30*
2.7. Thiết kế cơ sở dữ liệu 30
2.7.1. Tổ chức schema và tài khoản *30*
2.7.2. Các bảng chính *31*
2.7.3. Lưu phiên bản transcript và biên bản *32*
2.7.4. Nhãn người nói và bí danh *32*
2.7.5. Trích dẫn nguồn của công việc *32*
2.8. Thiết kế phân quyền 33
2.8.1. Vai trò theo phạm vi *33*
2.8.2. Trình tự kiểm tra quyền *34*
2.9. Thiết kế bộ phân tích AI 34
2.9.1. Quy tắc chống bịa trong prompt *34*
2.9.2. Cấu trúc đầu ra *35*
2.9.3. Con người trong vòng lặp *36*
2.10. Thiết kế giao diện 36
2.11. Thiết kế module đồng bộ công cụ ngoài và cảnh báo 37
2.11.1. Hiện trạng và dữ liệu *37*
2.11.2. Nguyên tắc chống tạo trùng và cảnh báo *37*
2.12. Kết luận chương 38

CHƯƠNG 3. XÂY DỰNG HỆ THỐNG.....39
3.1. Môi trường và công cụ phát triển.....39
3.2. Xây dựng dịch vụ lõi core-service.....40
3.2.1. Tổ chức mã nguồn và phân quyền.....40
3.2.2. Outbox, publisher và chống xử lý lặp.....41
3.2.3. Điều phối luồng xử lý cuộc họp.....42
3.3. Ba đường thu nhận nội dung cuộc họp.....42
3.3.1. Tải tệp ghi âm lên MinIO.....43
3.3.2. Bot tham gia và ghi âm Google Meet.....43
3.3.3. Tiện ích mở rộng đọc phụ đề.....45
3.4. Mô-đun nhận dạng tiếng nói.....46
3.5. Mô-đun tách người nói.....48
3.6. Mô-đun phân tích biên bản bằng LLM.....49
3.7. Luồng duyệt và quản lý công việc.....50
3.8. Mô-đun đồng bộ Trello và cảnh báo Telegram.....51
3.9. Giao diện web.....54
3.10. Triển khai và vận hành.....58
3.11. Kiểm thử phần mềm.....59
3.11.1. Kiểm thử đơn vị.....59
3.11.2. Kiểm thử tích hợp đầu-cuối.....60
3.11.3. Lỗi phát hiện và cách khắc phục.....61
3.12. Kết luận chương.....62
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ.....63
4.1. Mục tiêu và câu hỏi thực nghiệm.....63
4.2. Môi trường và bộ dữ liệu thực nghiệm.....63
4.2.1. Môi trường.....63
4.2.2. Kịch bản cuộc họp có đáp án.....64
4.2.3. Dựng âm thanh bằng giọng tổng hợp.....64
4.3. Độ đo đánh giá.....65
4.4. Kết quả nhận dạng tiếng nói.....66
4.5. Kết quả tách người nói và phát hiện nói chồng.....68
4.6. Lựa chọn mô hình ngôn ngữ lớn.....69
4.7. Đánh giá đầu-cuối từ âm thanh đến công việc.....70
4.8. Thời gian xử lý, chi phí và hiệu quả tiết kiệm thời gian.....72
4.8.1. Thời gian xử lý.....72
4.8.2. Chi phí.....73
4.8.3. Ước tính hiệu quả tiết kiệm thời gian.....73
4.9. Kiểm thử chức năng hệ thống.....73
4.10. Hạn chế của thực nghiệm.....75
4.11. Kết luận chương.....75
KẾT LUẬN VÀ KIẾN NGHỊ.....77

1. Kết luận..... 77
2. Hạn chế..... 78
3. Hướng phát triển..... 79
4. Kiến nghị..... 79
TÀI LIỆU THAM KHẢO..... 81

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh các giải pháp trợ lý cuộc họp tiêu biểu.....7
Bảng 1.2. So sánh các hướng tiếp cận nhận dạng tiếng nói.....8
Bảng 1.3. Đặc điểm các nền tảng tích hợp được khảo sát.....19
Bảng 1.4. So sánh các mô hình kiểm soát truy cập.....20
Bảng 2.1. Các tác nhân của hệ thống.....21
Bảng 2.2. Yêu cầu chức năng và trạng thái hiện thực.....22
Bảng 2.3. Yêu cầu phi chức năng và giải pháp thiết kế.....23
Bảng 2.4. Đặc tả ca sử dụng "Mời bot vào cuộc họp".....24
Bảng 2.5. Đặc tả ca sử dụng "Duyệt đề xuất công việc".....25
Bảng 2.6. Đặc tả ca sử dụng "Gán tên người nói".....25
Bảng 2.7. Các thành phần của hệ thống.....26
Bảng 2.8. Hợp đồng sự kiện giữa các dịch vụ.....29
Bảng 2.9. Trạng thái của job, cuộc họp và phiên bot.....30
Bảng 2.10. Các bảng chính của schema core.....31
Bảng 2.11. Một số quyền tiêu biểu và vai trò được phép.....33
Bảng 2.12. Cấu trúc JSON đầu ra của bộ phân tích.....35
Bảng 2.13. Các màn hình chính của giao diện web.....36
Bảng 2.14. Các bảng của schema integration.....37
Bảng 3.1. Ngôn ngữ, framework và thư viện chính dùng để xây dựng hệ thống.....39
Bảng 3.2. Hành vi của mô-đun đồng bộ theo từng nhà cung cấp.....52
Bảng 3.3. Tổng hợp kiểm thử đơn vị.....60
Bảng 3.4. Các lỗi bảo mật và tính đúng đắn phát hiện trong quá trình kiểm thử.....61
Bảng 4.1. Môi trường thực nghiệm.....63
Bảng 4.2. Các điều kiện âm thanh thử nghiệm.....65
Bảng 4.3. Kết quả nhận dạng tiếng nói theo điều kiện âm thanh.....66
Bảng 4.4. Kết quả tách người nói (Whisper small).....68
Bảng 4.5. So sánh các LLM trên bài trích xuất biên bản (transcript chuẩn, tối đa 15 điểm) 69
Bảng 4.6. Điểm trích xuất khi đầu vào là transcript do máy nhận dạng (tối đa 15).....70
Bảng 4.7. Thời gian xử lý bản ghi 151 giây ở điều kiện sạch.....72
Bảng 4.8. Ước tính thời gian công sức con người cho một cuộc họp 60 phút.....73
Bảng 4.9. Kết quả các kịch bản kiểm thử chức năng.....73

DANH MỤC HÌNH VẼ

Hình 1.1. Kiến trúc mô hình Whisper với chuỗi token đặc biệt điều khiển ngôn ngữ, nhiệm vụ và mốc thời gian.....9
Hình 1.2. Quy trình tách người nói dựa trên vector đặc trưng giọng và gom cụm phân cấp 12
Hình 1.3. Trích xuất thông tin có cấu trúc bằng LLM, đối chiếu bằng chứng và duyệt bởi con người.....15
Hình 2.1. Sơ đồ ca sử dụng tổng quát của hệ thống MeetingMind.....24
Hình 2.2. Kiến trúc tổng thể hệ thống MeetingMind.....26
Hình 2.3. Luồng xử lý từ âm thanh đến công việc.....28
Hình 2.4. Sơ đồ tuần tự sự kiện giữa core-service và ai-worker.....29
Hình 2.5. Sơ đồ quan hệ các bảng chính của schema core.....31
Hình 2.6. Phân quyền theo vai trò kết hợp phạm vi tài nguyên.....33
Hình 3.1. Ba đường thu nhận nội dung cuộc họp hội tụ tại transcript_versions.....43
Hình 3.2. Trạng thái một phiên bot và các lý do kết thúc.....44
Hình 3.3. Phiên âm hai lượt theo ngôn ngữ của từng đoạn.....47
Hình 3.4. Ngăn chi tiết công việc hiển thị kết quả đồng bộ Trello và Telegram.....52
Hình 3.5. Màn hình tổng quan của người quản lý dự án.....55
Hình 3.6. Danh sách cuộc họp với bộ lọc và giai đoạn xử lý.....55
Hình 3.7. Màn hình transcript với nhãn ngôn ngữ, cảnh báo nói chồng và gán người nói.56
Hình 3.8. Màn hình biên bản và vòng duyệt theo phiên bản.....56
Hình 3.9. Hàng đợi duyệt đề xuất công việc do AI trích xuất.....57
Hình 3.10. Bảng công việc theo trạng thái.....57
Hình 3.11. Hộp thoại mời bot ghi âm vào cuộc họp Google Meet.....58
Hình 3.12. Triển khai hệ thống bằng Docker Compose: các container và volume.....59
Hình 4.1. Tỷ lệ lỗi từ (đã chuẩn hoá số) theo điều kiện âm thanh và cấu hình.....67
Hình 4.2. Độ đúng trích xuất và độ trễ của các LLM trên FPT AI Factory.....69
Hình 4.3. Thời gian xử lý từng bước cho bản ghi 151 giây (CPU, không GPU).....72

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
ABAC	Attribute-Based Access Control	Kiểm soát truy cập dựa trên thuộc tính
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
ASR	Automatic Speech Recognition	Nhận dạng tiếng nói tự động
CER	Character Error Rate	Tỷ lệ lỗi ký tự
CSDL	Database	Cơ sở dữ liệu
CTC	Connectionist Temporal Classification	Phân loại thời gian kết nối
DER	Diarization Error Rate	Tỷ lệ lỗi tách người nói
DOM	Document Object Model	Mô hình đối tượng tài liệu
E2E	End-to-End	Đầu-cuối
GVHD	Advisor	Giảng viên hướng dẫn
JSON	JavaScript Object Notation	Định dạng dữ liệu JSON
JWT	JSON Web Token	Mã thông báo JSON
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MAD	Median Absolute Deviation	Độ lệch tuyệt đối trung vị
MV3	Manifest Version 3	Phiên bản manifest 3 của Chrome Extension
MVP	Minimum Viable Product	Sản phẩm khả dụng tối thiểu
NCKH	Scientific Research	Nghiên cứu khoa học
OAuth	Open Authorization	Giao thức ủy quyền mở
PM	Project Manager	Quản lý dự án
RAG	Retrieval-Augmented Generation	Sinh văn bản có tăng cường truy xuất
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer	Kiểu kiến trúc REST cho dịch vụ web
RTF	Real-Time Factor	Hệ số thời gian thực
SNR	Signal-to-Noise Ratio	Tỷ số tín hiệu trên nhiễu
SRS	Software Requirements Specification	Đặc tả yêu cầu phần mềm
STT	Speech-to-Text	Chuyển giọng nói thành văn bản
TTS	Text-to-Speech	Chuyển văn bản thành giọng nói
UI	User Interface	Giao diện người dùng

VAD	Voice Activity Detection	Phát hiện hoạt động giọng nói
WER	Word Error Rate	Tỷ lệ lỗi từ
WebRTC	Web Real-Time Communication	Giao tiếp thời gian thực trên web

MỞ ĐẦU

1. Tổng quan tình hình nghiên cứu

Cuộc họp là nơi một nhóm làm việc chia sẻ thông tin, ra quyết định và phân công công việc. Họp trực tuyến qua Google Meet, Microsoft Teams hay Zoom nay đã thành thói quen, nhưng khâu biến nội dung họp thành biên bản và thành công việc cụ thể vẫn chủ yếu làm bằng tay. Bài toán trợ lý cuộc họp (meeting assistant) hướng tới tự động hoá khâu này bằng cách kết hợp nhận dạng tiếng nói, tách người nói, xử lý ngôn ngữ tự nhiên và tích hợp hệ thống.

Ngoài nước

Các sản phẩm thương mại đã phát triển mạnh. Otter.ai [22] cung cấp phiên âm, tóm tắt và trợ lý tự tham gia cuộc họp; Fireflies.ai [9] đưa bot vào phòng để ghi âm, sinh bản ghi lời nói, tóm tắt và đẩy kết quả sang công cụ khác; Tactiq [30] dùng tiện ích trình duyệt đọc phụ đề của nền tảng họp nên không cần bot; Microsoft Teams Premium tích hợp tính năng tổng hợp cuộc họp bằng trí tuệ nhân tạo ngay trong Teams. Năng lực tách người nói và tóm tắt của các sản phẩm này đã dùng được trong thực tế, trước hết với tiếng Anh.

Về học thuật, ngữ liệu AMI [6] với khoảng một trăm giờ họp có chú giải là nền chung cho nhiều bài toán về cuộc họp; QMSum [36] cho thấy thông tin cần tóm tắt thường nằm rải rác trong bản ghi dài, phải định vị trước khi tóm tắt. Trong nhận dạng tiếng nói, wav2vec 2.0 [3] khai thác học tự giám sát trên âm thanh không nhãn, còn Whisper [25] được huấn luyện trên khoảng 680 nghìn giờ dữ liệu đa ngôn ngữ và bền vững với nhiều miền dữ liệu. Tách người nói đã chuyển sang vector giọng học sâu như x-vector [28] và các bộ công cụ nơ-ron như pyannote.audio [4], dù nói chồng và thu âm xa micro vẫn là thách thức [23]. Mô hình ngôn ngữ lớn (Large Language Model, LLM) học được từ vài ví dụ trong ngữ cảnh [5], cho phép trích xuất thông tin mà không cần tập huấn luyện riêng, nhưng có nguy cơ "ảo giác" (hallucination) — sinh nội dung không có trong nguồn [14].

Với nhóm nhỏ tại Việt Nam, các giải pháp nước ngoài có ba hạn chế: chi phí bản quyền theo người dùng hằng tháng cao; tiếng Việt, nhất là tiếng Việt xen thuật ngữ tiếng Anh, chưa phải trọng tâm; dữ liệu hội thoại được xử lý ở nước ngoài và luồng dữ liệu khó tuỳ biến theo hệ thống quản lý riêng của từng đơn vị.

Trong nước

FPT.AI và Viettel AI đã cung cấp dịch vụ chuyển giọng nói thành văn bản (Speech-to-Text, STT) cho tiếng Việt với chất lượng tốt. Tuy nhiên, các giải pháp này chủ yếu dừng ở bản gỡ băng (transcript) thô hoặc tóm tắt đơn giản; việc nối nội dung

họp với hệ thống quản lý công việc còn rời rạc, nhiều đơn vị vẫn tạo đầu việc trên Jira hay Trello bằng tay sau khi họp. Về học thuật, nhận dạng tiếng Việt đã có các công trình nền như bộ ngữ liệu VIVOS và công thức Kaldi [18], nhưng ngữ liệu cuộc họp tiếng Việt công khai — nhiều người nói, có nhiễu, có nói chồng, xen tiếng Anh — gần như chưa có. Gần đây, FPT AI Factory [10] cung cấp dịch vụ suy luận LLM phi máy chủ (serverless) với giao diện tương thích OpenAI, cho phép dùng LLM mạnh trong khi dữ liệu vẫn xử lý trên hạ tầng trong nước.

Như vậy, các thành phần công nghệ đã tương đối sẵn sàng, nhưng còn thiếu một hệ thống liền mạch, chi phí hợp lý, phù hợp tiếng Việt, đi từ âm thanh cuộc họp đến công việc có nguồn kiểm chứng và được con người xác nhận. Đây là khoảng trống đề tài hướng tới.

2. Lý do chọn đề tài

Đề tài xuất phát từ những khó khăn cụ thể trong làm việc nhóm. Người tham gia vừa thảo luận vừa ghi chép nên dễ bỏ sót; sau họp, người quản lý dự án (Project Manager, PM) mất thời gian chuyển biên bản thành công việc, điền người thực hiện và thời hạn. Một ý kiến mới ở mức đề xuất ("hay là mình dùng X") dễ bị hiểu nhầm thành quyết định; thời hạn bị đính chính giữa chừng có thể bị ghi theo thông tin cũ; và khi cần đối chiếu, rất khó truy lại câu nói gốc đã sinh ra một đầu việc.

Đồng thời, công nghệ đã đủ chín để nhóm nhỏ tự xây dựng giải pháp. Whisper có mã nguồn và trọng số mở; bản hiện thực faster-whisper chạy được trên CPU với trọng số lượng tử hoá. Các LLM có trọng số mở được cung cấp qua dịch vụ suy luận trong nước, tính phí theo lượng dùng. Nhờ đó, hệ thống có thể triển khai với chi phí thấp, chủ động về dữ liệu và tuỳ biến theo quy trình của nhóm.

Đề tài cũng đặt ra các câu hỏi nghiên cứu: nhận dạng tiếng nói đa ngôn ngữ đạt chất lượng ra sao với tiếng Việt xen tiếng Anh, khi có nhiễu và nói chồng; làm thế nào để LLM phân biệt đề xuất với quyết định, lấy thông tin sau cùng khi có đính chính, không bịa người hay hạn, và kết quả đó được kiểm chứng ra sao; kiến trúc nào phù hợp cho các tác vụ trí tuệ nhân tạo chạy lâu, bất đồng bộ mà vẫn bảo đảm nhất quán dữ liệu và kiểm soát truy cập nhiều tầng. Vì những lý do trên, chúng tôi chọn đề tài "Thiết kế và triển khai hệ thống Meet-Assistant: Tự động hóa take-note và đồng bộ hóa quản lý công việc", với sản phẩm là hệ thống MeetingMind.

3. Mục tiêu đề tài

Mục tiêu tổng quát. Xây dựng hệ thống trợ lý cuộc họp ứng dụng trí tuệ nhân tạo (STT và LLM) để tự động hoá ghi chép biên bản, trích xuất chính xác các nhiệm

vụ và kết nối kết quả cuộc họp với quản lý công việc, nhằm nâng cao hiệu suất làm việc nhóm.

Mục tiêu cụ thể:

1. Xây dựng ứng dụng web quản lý workspace, dự án, cuộc họp và công việc, phân quyền theo vai trò kết hợp phạm vi tài nguyên.
2. Thu nhận nội dung họp qua ba đường — tải tệp ghi âm, bot ghi âm Google Meet, tiện ích Chrome đọc phụ đề — hội tụ về một luồng xử lý chung.
3. Nhận dạng tiếng nói tiếng Việt có xử lý câu xen tiếng Anh, kèm tách người nói, chạy được trên máy chủ chỉ có CPU.
4. Dùng LLM sinh biên bản nháp, trích xuất quyết định, vấn đề còn mở và đầu việc kèm bằng chứng; mọi đề xuất của AI phải được người quản lý duyệt trước khi thành công việc chính thức.
5. Xây dựng backend microservices hướng sự kiện; tự động đồng bộ công việc đã duyệt sang Trello và thông báo công việc mới qua Telegram, với kiến trúc sẵn sàng mở rộng sang Jira và cảnh báo rủi ro.
6. Đánh giá định lượng độ chính xác nhận dạng tiếng nói, chất lượng trích xuất của các LLM và thời gian xử lý.

Đồng bộ sang Trello và thông báo qua Telegram được hiện thực ở mức sản phẩm khả dụng tối thiểu (Minimum Viable Product, MVP): mỗi nền tảng một kết nối cấu hình qua biến môi trường, đồng bộ một chiều. Các nội dung còn lại trong đề cương — đồng bộ sang Jira và Confluence, phát hiện rủi ro tiến độ, đối soát cam kết với kết quả thực hiện và mã hoá dữ liệu hội thoại — được trình bày ở mức thiết kế và định hướng phát triển; mức độ hoàn thành được phân tích ở phần Kết luận và kiến nghị.

4. Phương pháp nghiên cứu

Nghiên cứu tài liệu: tổng hợp các công trình về nhận dạng tiếng nói, tách người nói, LLM và tóm tắt cuộc họp; tài liệu kỹ thuật của công cụ sử dụng và của các nền tảng dự kiến tích hợp; khảo sát sản phẩm trợ lý cuộc họp hiện có để xác định hướng tiếp cận.

Phân tích, thiết kế hướng đối tượng và hướng sự kiện: đặc tả yêu cầu bằng tác nhân và ca sử dụng, mô hình hoá dữ liệu bằng sơ đồ thực thể – quan hệ, phân rã hệ thống thành các service theo miền nghiệp vụ. Các luồng xử lý dài được mô tả bằng sơ đồ tuần tự sự kiện, hợp đồng sự kiện đặc tả bằng JSON Schema, tính nhất quán bảo đảm bằng các mẫu transactional outbox và bên nhận lũy đẳng (idempotent consumer).

Thực nghiệm định lượng trên bộ dữ liệu dựng có kiểm soát: do thiếu ngữ liệu họp tiếng Việt có chú giải, nhóm dựng bộ dữ liệu có đáp án từ một kịch bản họp nhóm

phần mềm, tổng hợp giọng nói bằng dịch vụ chuyển văn bản thành giọng nói (Text-to-Speech, TTS) với nhiều giọng, rồi tạo các biến thể có nhiễu trắng ở nhiều mức tỷ số tín hiệu trên nhiễu (Signal-to-Noise Ratio, SNR), nhiễu nói chuyện nền và nói chồng. Cách làm này cho phép thay đổi từng yếu tố độc lập; các đại lượng đo gồm tỷ lệ lỗi từ, tỷ lệ lỗi ký tự, độ đúng gán người nói và thời gian xử lý. Vì giọng tổng hợp sạch hơn giọng thật, kết quả chủ yếu dùng để so sánh tương đối giữa các cấu hình.

So sánh mô hình: chấm nhiều LLM trên FPT AI Factory với cùng một bản ghi họp có đáp án chứa các tình huống khó (đề xuất khác quyết định, hạn bị đính chính, đổi người làm, chuyện ngoài lề, câu tiếng Anh, người nói chỉ có nhãn tự động), so sánh độ đúng, độ ổn định qua nhiều lần chạy và thời gian phản hồi để chọn mô hình chính và dự phòng. Phần mềm được kiểm thử đơn vị và kiểm thử đầu-cuối trên Docker Compose.

5. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu gồm các kỹ thuật nhận dạng tiếng nói, tách người nói và trích xuất thông tin bằng LLM cho hội thoại cuộc họp; kiến trúc microservices hướng sự kiện và kiểm soát truy cập nhiều tầng; quy trình nghiệp vụ của cuộc họp nhóm từ chuẩn bị, ghi nhận, duyệt kết quả đến theo dõi công việc.

Phạm vi nghiên cứu: ngôn ngữ là tiếng Việt và tiếng Việt xen tiếng Anh; nền tảng họp trực tuyến là Google Meet (qua bot ghi âm và tiện ích đọc phụ đề), cuộc họp trực tiếp hoặc đã có bản ghi được đưa vào bằng tải tệp; đối tượng sử dụng là nhóm dự án nhỏ với cuộc họp ít người; nội dung được xử lý sau họp; hệ thống triển khai trên một máy chủ bằng Docker Compose, mô hình nhận dạng tiếng nói chạy trên CPU.

Ngoài phạm vi: Microsoft Teams và Zoom; phiên âm trực tiếp từ âm thanh trong lúc họp; nhận diện danh tính qua mẫu giọng; huấn luyện hay tinh chỉnh mô hình; mã hoá dữ liệu hội thoại khi lưu trữ (at-rest encryption) — hiện dữ liệu được bảo vệ bằng xác thực và phân quyền. Đồng bộ Jira, hỏi đáp bằng truy xuất tăng cường (Retrieval-Augmented Generation, RAG) và phát hiện rủi ro tự động mới ở mức thiết kế, chưa triển khai. Đồng bộ Trello và thông báo Telegram đã triển khai ở mức MVP; màn hình kết nối theo từng workspace và đồng bộ hai chiều trạng thái công việc nằm ngoài phạm vi của đề tài.

6. Bố cục báo cáo

Ngoài Mở đầu và Kết luận và kiến nghị, báo cáo gồm bốn chương. Chương 1 trình bày cơ sở lý thuyết và công nghệ: bài toán trợ lý cuộc họp, nhận dạng tiếng nói, tách người nói, LLM trong trích xuất thông tin, kiến trúc microservices hướng sự kiện, thu nhận nội dung họp trực tuyến, tích hợp nền tảng quản lý công việc và kiểm soát

truy cập. Chương 2 trình bày phân tích và thiết kế hệ thống MeetingMind: ca sử dụng, kiến trúc tổng thể, luồng xử lý và luồng sự kiện, cơ sở dữ liệu và phân quyền. Chương 3 trình bày xây dựng và triển khai: ba đường thu nhận, bot Google Meet, phiên âm hai lượt, tách người nói, phân tích bằng LLM, đồng bộ Trello và cảnh báo Telegram, giao diện và triển khai. Chương 4 trình bày thực nghiệm và đánh giá độ chính xác nhận dạng, tách người nói, chất lượng các LLM và thời gian xử lý. Phần Kết luận và kiến nghị tổng kết kết quả so với đề cương, nêu hạn chế và hướng phát triển.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

Chương này trình bày nền tảng lý thuyết và công nghệ của MeetingMind theo trình tự dòng dữ liệu: bài toán trợ lý cuộc họp, nhận dạng tiếng nói, tách người nói, trích xuất thông tin bằng mô hình ngôn ngữ lớn, kiến trúc phần mềm, thu nhận nội dung họp trực tuyến, tích hợp nền tảng quản lý công việc và kiểm soát truy cập. Với mỗi nội dung, chúng tôi nêu khái niệm và lý do lựa chọn kỹ thuật cho hệ thống.

1.1. Bài toán trợ lý cuộc họp

1.1.1. Phát biểu bài toán

Đầu vào của bài toán là nội dung một cuộc họp — tín hiệu âm thanh hoặc phụ đề kèm tên người nói — cùng ngữ cảnh như dự án, người tham dự và ngày họp. Đầu ra gồm: bản ghi lời nói (transcript) chia theo lượt, có mốc thời gian và nhãn người nói; biên bản gồm tóm tắt, quyết định và vấn đề còn mở; danh sách đầu việc (action item) có mô tả, người thực hiện, thời hạn, mức ưu tiên và bằng chứng trong bản ghi. Sau khi được duyệt, đầu việc trở thành công việc chính thức và có thể đồng bộ sang công cụ quản lý bên ngoài.

Bài toán được phân rã thành chuỗi (pipeline) các khâu: nhận dạng tiếng nói, tách người nói, trích xuất thông tin, duyệt bởi con người và tích hợp. Cách này cho phép chọn công cụ tốt nhất cho từng khâu, nhưng lỗi khâu trước lan sang khâu sau — nhận dạng sai một cái tên làm sai người thực hiện — nên mỗi khâu cần lưu kết quả trung gian theo phiên bản để người dùng sửa và chạy lại.

1.1.2. Đặc thù của hội thoại cuộc họp

Lời nói trong họp là lời tự phát, chứa ngập ngừng, câu bỏ dở và tự sửa ("ngày 15… à không, ngày 18"). Nhiều người nói với lượt ngắn, có lúc nói chồng; ở phòng họp, một micro chung thu tiếng từ nhiều khoảng cách. Một đầu việc thường hình thành qua nhiều lượt và người được giao có thể chỉ được nhắc ở ngôi thứ ba: khi ai đó nói "Lan làm phần này", người nói không phải là Lan. Hệ thống còn phải phân biệt đề xuất, câu hỏi, đồng ý, quyết định, giao việc và bỏ qua chuyện ngoài lề. Với tiếng Việt, họp kỹ thuật thường xen thuật ngữ tiếng Anh và dùng mốc thời gian tương đối ("thứ Sáu", "cuối tuần").

1.1.3. Ngữ liệu và hướng nghiên cứu liên quan

Ngữ liệu AMI [6] gồm khoảng một trăm giờ họp tiếng Anh, ghi đồng thời bằng micro đeo gần miệng và micro đặt xa, có chú giải nhiều tầng (phiên âm, lượt nói, hành vi hội thoại, tóm tắt), là chuẩn so sánh cho nhiều bài toán về cuộc họp. QMSum [36]

đặt bài toán tóm tắt theo truy vấn trên nhiều lĩnh vực với cách làm hai bước "định vị rồi tóm tắt", vì thông tin cần thiết nằm rải rác trong bản ghi dài. Các ngữ liệu này đều bằng tiếng Anh; tiếng Việt chưa có ngữ liệu họp công khai tương đương, nên đề tài tự dựng bộ dữ liệu có kiểm soát để đánh giá.

1.1.4. Các giải pháp trợ lý cuộc họp hiện có

Có ba cách thu nhận nội dung chính: đưa bot vào phòng để ghi âm, đọc phụ đề do nền tảng họp sinh ra, và nhận bản ghi có sẵn qua tải tệp hoặc giao diện lập trình. Bảng 1.1 tóm tắt các giải pháp tiêu biểu.

Bảng 1.1. *So sánh các giải pháp trợ lý cuộc họp tiêu biểu*

Giải pháp	Cách thu nhận	Điểm mạnh	Hạn chế với nhóm nhỏ tại Việt Nam
Otter.ai, Fireflies.ai	Bot vào phòng họp, ghi âm	Tách người nói tốt, tóm tắt, nhiều tích hợp	Chi phí bản quyền; tiếng Việt chưa phải trọng tâm; dữ liệu xử lý ở nước ngoài
Tactiq	Tiện ích trình duyệt đọc phụ đề	Không cần bot; có sẵn tên người nói	Phụ thuộc phụ đề của nền tảng; không có âm thanh để nghe lại
Microsoft Teams Premium	Tích hợp sẵn trong Teams	Liền mạch trong hệ sinh thái Microsoft	Gắn với một nền tảng; khó tuỳ biến luồng dữ liệu
FPT.AI, Viettel AI	Giao diện lập trình chuyển giọng nói thành văn bản	Nhận dạng tiếng Việt tốt	Dừng ở bản ghi thô hoặc tóm tắt đơn giản

Bot cho âm thanh gốc nhưng cần chủ phòng cho vào; phụ đề có sẵn tên người nói nhưng phụ thuộc nền tảng; tải tệp dùng được cho họp trực tiếp nhưng không tự động. Vì vậy MeetingMind hỗ trợ cả ba đường, hội tụ về một luồng xử lý chung.

1.2. Nhận dạng tiếng nói tự động

1.2.1. Các hướng tiếp cận

Nhận dạng tiếng nói tự động (Automatic Speech Recognition, ASR) tìm chuỗi từ có xác suất lớn nhất khi biết chuỗi đặc trưng âm thanh, thường trích từ các khung khoảng 25 ms, bước dịch 10 ms. Hướng truyền thống tách bài toán theo công thức Bayes thành mô hình âm học và mô hình ngôn ngữ. Mô hình âm học dùng mô hình Markov ẩn (Hidden Markov Model, HMM) cho các âm vị phụ thuộc ngữ cảnh, xác suất phát xạ mô hình hoá bằng hỗn hợp Gauss (Gaussian Mixture Model, GMM), về sau bằng mạng nơ-ron sâu; kèm từ điển phát âm và mô hình ngôn ngữ n-gram. Kaldi là

bộ công cụ tiêu biểu; với tiếng Việt, Luong và Vu công bố bộ ngữ liệu VIVOS cùng công thức Kaldi [18], lưu ý đặc điểm đơn âm tiết và có thanh điệu của tiếng Việt.

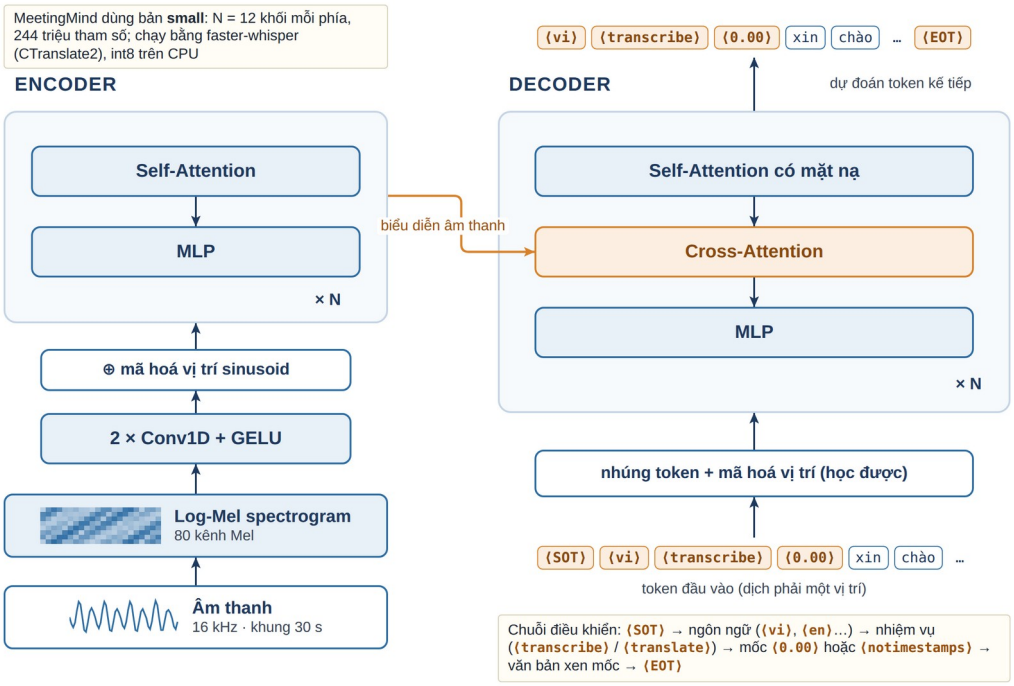
Hướng đầu-cuối (end-to-end) học trực tiếp ánh xạ âm thanh sang ký tự. Hàm mất mát phân loại theo thời gian kết nối (Connectionist Temporal Classification, CTC) thêm một ký hiệu trống, gộp nhãn lặp và cộng xác suất trên mọi cách căn chỉnh, nên không cần căn chỉnh từng khung; đổi lại, CTC giả định đầu ra các khung độc lập có điều kiện. wav2vec 2.0 [3] tiền huấn luyện tự giám sát trên âm thanh không nhãn bằng cách che một phần biểu diễn tiềm ẩn và học phân biệt biểu diễn đúng với mẫu nhiễu, rồi tinh chỉnh bằng CTC với ít dữ liệu có nhãn. Hướng thứ ba là mô hình chuỗi sang chuỗi (sequence-to-sequence) có cơ chế chú ý, ngày nay dựa trên Transformer [33]: bộ giải mã sinh từng token dựa trên các token trước nên tự học mô hình ngôn ngữ và dấu câu, nhưng giải mã tự hồi quy chậm hơn và có thể sinh nội dung không có trong âm thanh. Bảng 1.2 so sánh ba hướng này.

Bảng 1.2. *So sánh các hướng tiếp cận nhận dạng tiếng nói*

Hướng tiếp cận	Thành phần chính	Ưu điểm	Hạn chế
HMM-GMM, lai DNN-HMM (Kaldi)	Mô hình âm học, từ điển phát âm, mô hình ngôn ngữ n-gram	Cần ít dữ liệu; kiểm soát từng thành phần	Nhiều thành phần; cần chuyên môn ngôn ngữ; kém bền với miền mới
CTC, wav2vec 2.0	Bộ mã hoá tự giám sát, tinh chỉnh bằng CTC	Tận dụng âm thanh không nhãn; suy luận nhanh	Giả định độc lập giữa các khung; thường cần thêm mô hình ngôn ngữ
Chuỗi sang chuỗi (Whisper)	Bộ mã hoá và giải mã Transformer	Đa ngôn ngữ, đa nhiệm vụ; có dấu câu; bền vững	Giải mã chậm hơn; có thể ảo giác ở đoạn lặng, ồn

1.2.2. Mô hình Whisper

Whisper [25] là mô hình chuỗi sang chuỗi huấn luyện theo hướng giám sát yếu (weak supervision) trên khoảng 680 nghìn giờ âm thanh kèm văn bản thu thập từ Internet, nhiều ngôn ngữ và nhiều nhiệm vụ. Âm thanh được lấy mẫu lại ở 16 kHz, cắt thành cửa sổ 30 giây và chuyển thành phổ log-Mel. Bộ mã hoá gồm hai lớp tích chập, lớp thứ hai có bước nhảy 2, theo sau là các khối Transformer; bộ giải mã Transformer sinh văn bản tự hồi quy, có chú ý chéo (cross-attention) tới đầu ra bộ mã hoá (Hình 1.1).



Hình 1.1. *Kiến trúc mô hình Whisper với chuỗi token đặc biệt điều khiển ngôn ngữ, nhiệm vụ và mốc thời gian*

Whisper dùng chuỗi token đặc biệt ở đầu bộ giải mã để chỉ định nhiệm vụ: token bắt đầu bản ghi, token ngôn ngữ (ví dụ `<|vi|>`), token nhiệm vụ (phiên âm hay dịch sang tiếng Anh) và token cho biết có sinh mốc thời gian hay không; mốc thời gian là token rời rạc độ phân giải 20 ms. Nhờ đó một mô hình làm được phiên âm, dịch, nhận diện ngôn ngữ và phát hiện đoạn không có lời. Whisper có nhiều kích thước, từ tiny (khoảng 39 triệu tham số), base, small (khoảng 244 triệu), medium đến large (khoảng 1,55 tỷ); mô hình lớn chính xác hơn nhưng chậm hơn nhiều. Tiếng Việt chỉ chiếm phần nhỏ trong dữ liệu huấn luyện nên độ chính xác thấp hơn tiếng Anh.

1.2.3. faster-whisper, CTranslate2 và lượng tử hoá int8

faster-whisper [29] hiện thực lại Whisper trên CTranslate2, bộ máy suy luận Transformer viết bằng C++ với các tối ưu như hợp nhất phép toán, quản lý bộ nhớ và lượng tử hoá trọng số; theo nhóm phát triển, nó nhanh hơn đáng kể và tốn ít bộ nhớ hơn bản gốc với cùng độ chính xác. Thư viện tích hợp bộ phát hiện tiếng nói (Voice Activity Detection, VAD) Silero để bỏ đoạn không có lời, và tính mốc thời gian từng từ dựa trên trọng số chú ý chéo.

Lượng tử hoá int8 biểu diễn trọng số số thực 32 bit bằng số nguyên 8 bit. Với lượng tử hoá đối xứng, mỗi nhóm trọng số w (ví dụ một hàng ma trận) có hệ số tỷ lệ $s = \max|w| / 127$; giá trị lưu là $q = \text{round}(w / s)$ và khi tính toán $w \approx s \times q$. Bộ nhớ trọng số giảm khoảng bốn lần, phép nhân ma trận dùng được lệnh số nguyên của CPU, đổi

lại sai số làm tròn nhỏ. Đây là lý do MeetingMind chọn faster-whisper với mô hình small, chạy CPU ở chế độ int8.

1.2.4. Nhận diện ngôn ngữ và hiện tượng chuyển mã Việt–Anh

Whisper nhận diện ngôn ngữ bằng phân bố xác suất của token ngôn ngữ ở bước giải mã đầu tiên; với bản ghi dài, cách dùng phổ biến chỉ nhận diện trên 30 giây đầu rồi giải mã cả bài bằng ngôn ngữ đó. Giả định này không đúng với hiện tượng chuyển mã (code-switching), khi người nói chuyển qua lại giữa hai ngôn ngữ. Chuyển mã trong câu (xen "deploy", "API" vào câu tiếng Việt) thường được Whisper xử lý chấp nhận được ở chế độ tiếng Việt; chuyển mã giữa các câu thì câu tiếng Anh bị ép giải mã như tiếng Việt, cho ra chuỗi vô nghĩa hoặc bị dịch.

Cách khắc phục là nhận diện lại ngôn ngữ cho từng đoạn. Do đoạn ngắn chứa quá ít âm thanh để nhận diện tin cậy, chỉ nên đổi ngôn ngữ của đoạn khi đoạn đủ dài và xác suất nhận diện vượt ngưỡng đủ cao; còn lại giữ ngôn ngữ chính. Cơ chế phiên âm hai lượt dựa trên ý tưởng này được trình bày ở phần xây dựng hệ thống.

1.2.5. Hiện tượng ảo giác của Whisper và cách hạn chế

Bộ giải mã của Whisper về bản chất là mô hình ngôn ngữ có điều kiện. Khi đoạn âm thanh chỉ có im lặng hoặc nhiễu, bằng chứng âm học yếu, mô hình có xu hướng sinh câu có xác suất tiên nghiệm cao trong dữ liệu huấn luyện; vì dữ liệu lấy từ phụ đề video trên mạng, đó thường là "đăng ký kênh", "cảm ơn các bạn đã theo dõi" hay "thanks for watching". Việc điều kiện hoá trên văn bản đoạn trước còn có thể khuếch đại lỗi thành vòng lặp lặp lại một câu.

Bài báo Whisper [25] đề xuất cho bản ghi dài: tìm kiếm chùm (beam search), giải mã lại với nhiệt độ cao hơn khi xác suất trung bình quá thấp hoặc văn bản bị nén quá mạnh (dấu hiệu lặp), và bỏ đoạn khi xác suất "không có lời" cao. MeetingMind dùng thêm bốn biện pháp: lọc khoảng lặng bằng VAD; tắt điều kiện hoá trên văn bản trước (`condition_on_previous_text=False`); bỏ qua khoảng lặng dài khi có dấu hiệu ảo giác (`hallucination_silence_threshold` của faster-whisper); và lọc hậu kỳ các câu rác điển hình theo danh sách mẫu.

1.2.6. Độ đo đánh giá nhận dạng tiếng nói

Độ đo phổ biến nhất là tỷ lệ lỗi từ (Word Error Rate, WER). Câu nhận dạng được căn chỉnh với câu tham chiếu theo khoảng cách chỉnh sửa nhỏ nhất [17], tính bằng quy hoạch động, rồi đếm các phép sửa:

$$WER = (S + D + I) / N$$

trong đó S là số từ bị thay thế, D là số từ bị xoá (bỏ sót), I là số từ bị chèn thêm, N là số từ của câu tham chiếu. Do có phép chèn, WER có thể vượt 100%. Tỷ lệ lỗi ký tự (Character Error Rate, CER) tính tương tự ở mức ký tự. Với tiếng Việt, khoảng trắng phân tách âm tiết chứ không phải từ, nên WER thực chất là tỷ lệ lỗi âm tiết. Trước khi tính cần chuẩn hoá: chữ thường, bỏ dấu câu, đưa Unicode về dạng dựng sẵn (NFC) và thống nhất cách viết số ("18" hay "mười tám"), vì chuẩn hoá khác nhau có thể làm WER chênh lệch đáng kể. Về tốc độ, hệ số thời gian thực (Real-Time Factor, RTF) là tỷ số giữa thời gian xử lý và thời lượng âm thanh; RTF nhỏ hơn 1 nghĩa là xử lý nhanh hơn thời gian thực.

1.3. Tách người nói

1.3.1. Định nghĩa bài toán và độ đo DER

Tách người nói (speaker diarization) trả lời câu hỏi "ai nói khi nào" [23]: chia bản ghi thành các đoạn và gán mỗi đoạn một nhãn ẩn danh như "Người nói 1", "Người nói 2". Bài toán khác nhận dạng người nói (speaker identification) vốn xác định danh tính qua mẫu giọng đã đăng ký: tách người nói chỉ biết hai đoạn cùng hay khác giọng, không biết người đó là ai, nên việc gắn tên thật do người dùng xác nhận.

Độ đo chuẩn là tỷ lệ lỗi tách người nói (Diarization Error Rate, DER):

DER = (T_FA + T_MISS + T_CONF) / T_REF

trong đó T_FA là thời lượng báo nhầm có lời, T_MISS là thời lượng lời bị bỏ sót, T_CONF là thời lượng lời bị gán nhầm người, T_REF là tổng thời lượng lời trong tham chiếu. Vì nhãn ẩn danh, trước khi tính cần tìm ánh xạ một–một tối ưu giữa nhãn hệ thống và người thật. Khi ranh giới lời nói đã lấy từ khâu nhận dạng tiếng nói, có thể dùng độ đo đơn giản hơn là độ đúng gán người nói theo thời lượng dưới ánh xạ tối ưu, tương ứng phần T_CONF của DER.

1.3.2. Vector đặc trưng giọng nói

Phần lớn hệ thống tách người nói dựa trên vector đặc trưng giọng (speaker embedding): vector có số chiều cố định đại diện cho giọng của một đoạn âm thanh, sao cho hai đoạn cùng người cho vector gần nhau. i-vector là thế hệ thống kê, biểu diễn siêu vector trung bình của mô hình GMM trong một không gian biến thiên tổng số chiều thấp. x-vector [28] chuyển sang học sâu: các lớp mạng nơ-ron trễ thời gian xử lý từng khung, lớp gộp thống kê tính trung bình và độ lệch chuẩn trên cả đoạn, các lớp sau học phân loại người nói, và vector lấy từ lớp ẩn làm đặc trưng.

ERes2Net [7] dựa trên Res2Net — kiến trúc biểu diễn đặc trưng đa tỉ lệ trong mỗi khối dư — và bổ sung cơ chế hợp nhất đặc trưng cục bộ và toàn cục bằng chú ý,

giúp vector phân biệt tốt hơn cả với đoạn âm thanh ngắn. MeetingMind dùng mô hình ERes2Net của bộ 3D-Speaker ở dạng ONNX, chạy CPU qua thư viện sherpa-onnx [15]. Độ giống giữa hai vector u, v đo bằng độ tương đồng cosine:

$$\cos(u, v) = (u \cdot v) / (\|u\| \times \|v\|)$$

giá trị gần 1 khi hai đoạn rất giống nhau; khoảng cách cosine tương ứng là 1 − cos(u, v).

1.3.3. Gom cụm phân cấp theo độ tương đồng cosine

Quy trình tách người nói dạng mô-đun gồm: phát hiện đoạn có lời, cắt thành cửa sổ ngắn, tính vector giọng cho mỗi cửa sổ và gom cụm các vector, mỗi cụm là một người nói (Hình 1.2). pyannote.audio [4] cung cấp các khối mạng nơ-ron cho từng bước, trong đó có mô hình phân đoạn học sâu. Độ dài cửa sổ là một đánh đổi: cửa sổ ngắn bám sát chỗ đổi người nhưng vector nhiễu, cửa sổ dài cho vector ổn định nhưng dễ chứa hai người. Cắt cửa sổ khoảng 1,5 giây theo ranh giới từ, dựa trên mốc thời gian từ của Whisper, giúp không cắt giữa từ.



Hình 1.2. *Quy trình tách người nói dựa trên vector đặc trưng giọng và gom cụm phân cấp*

Gom cụm phân cấp từ dưới lên (Agglomerative Hierarchical Clustering, AHC) coi mỗi cửa sổ là một cụm, lặp lại việc gộp hai cụm giống nhau nhất cho tới khi độ giống lớn nhất còn lại thấp hơn ngưỡng; với liên kết trung bình (average linkage), độ giống hai cụm là trung bình độ giống mọi cặp phần tử giữa chúng. Số người nói nhờ đó được suy ra tự động, nhưng ngưỡng phụ thuộc mô hình vector và điều kiện thu âm; cửa sổ ngắn, ồn còn có thể sinh "người nói ảo", nên thường gộp các cụm có tổng thời lượng quá nhỏ vào cụm gần nhất.

1.3.4. Vấn đề nói chồng

Nói chồng (overlapping speech) rất phổ biến trong họp: chen ngang, đáp lời ngắn, hai người cùng mở lời. Với cách tiếp cận gom cụm, mỗi cửa sổ chỉ nhận một nhãn nên lời người thứ hai bị bỏ sót, góp vào T_MISS; khâu nhận dạng một kênh cũng thường chỉ chép được giọng át. Các hướng xử lý triệt để gồm mô hình phát hiện nói chồng chuyên biệt, mô hình tách người nói đầu-cuối cho phép nhiều nhãn cùng lúc, hoặc tách nguồn âm thanh trước khi nhận dạng [23].

Trong phạm vi đề tài, MeetingMind chỉ đánh dấu cửa sổ có vector giọng giống người nói thứ hai cao bất thường so với chính bản ghi là "có thể nói chồng" để người dùng nghe lại; đây là gợi ý hỗ trợ kiểm tra, không tách được lời người thứ hai.

1.4. Mô hình ngôn ngữ lớn trong trích xuất thông tin

1.4.1. Kiến trúc Transformer và mô hình ngôn ngữ lớn

Transformer [33] thay mạng hồi quy bằng cơ chế tự chú ý (self-attention): mỗi token được chiếu thành vector truy vấn Q, khoá K và giá trị V, đầu ra là tổ hợp có trọng số của các giá trị:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

với d_k là số chiều vector khoá. Nhiều đầu chú ý chạy song song giúp nắm nhiều kiểu quan hệ, và cả chuỗi được xử lý song song khi huấn luyện. Mô hình ngôn ngữ lớn hiện nay chủ yếu chỉ dùng phần giải mã, huấn luyện dự đoán token tiếp theo trên kho văn bản rất lớn. GPT-3 [5] với 175 tỷ tham số cho thấy mô hình đủ lớn có thể làm nhiệm vụ mới chỉ từ mô tả và vài ví dụ trong ngữ cảnh.

DeepSeek-V3 [8] dùng kiến trúc hỗn hợp chuyên gia (Mixture-of-Experts, MoE) với 671 tỷ tham số nhưng mỗi token chỉ kích hoạt khoảng 37 tỷ, giảm mạnh chi phí suy luận. Họ GLM [31] là một dòng mô hình song ngữ trọng số mở khác. MeetingMind sử dụng các thế hệ kế tiếp của hai họ mô hình này.

1.4.2. Mô hình suy luận

Wei và cộng sự [35] chỉ ra rằng yêu cầu mô hình viết các bước lập luận trung gian (chain-of-thought) trước khi trả lời cải thiện rõ khả năng giải bài toán nhiều bước ở mô hình đủ lớn. Các mô hình suy luận (reasoning model) được huấn luyện để tự sinh chuỗi suy nghĩ dài trước câu trả lời, thường được giao diện lập trình trả tách riêng khỏi nội dung chính. Với trích xuất biên bản, suy luận hữu ích khi phải theo dõi cả cuộc hội thoại: thời hạn bị đính chính, đầu việc bị chuyển người, đề xuất cuối cùng không được chấp nhận. Đổi lại, mô hình suy luận tốn nhiều token và có độ trễ cao hơn hẳn; vì phân

tích chạy nền sau phiên âm nên độ trễ này chấp nhận được, nhưng giới hạn token phải đủ lớn cho cả phần suy nghĩ lẫn JSON đầu ra.

1.4.3. Thiết kế prompt và đầu ra JSON có cấu trúc

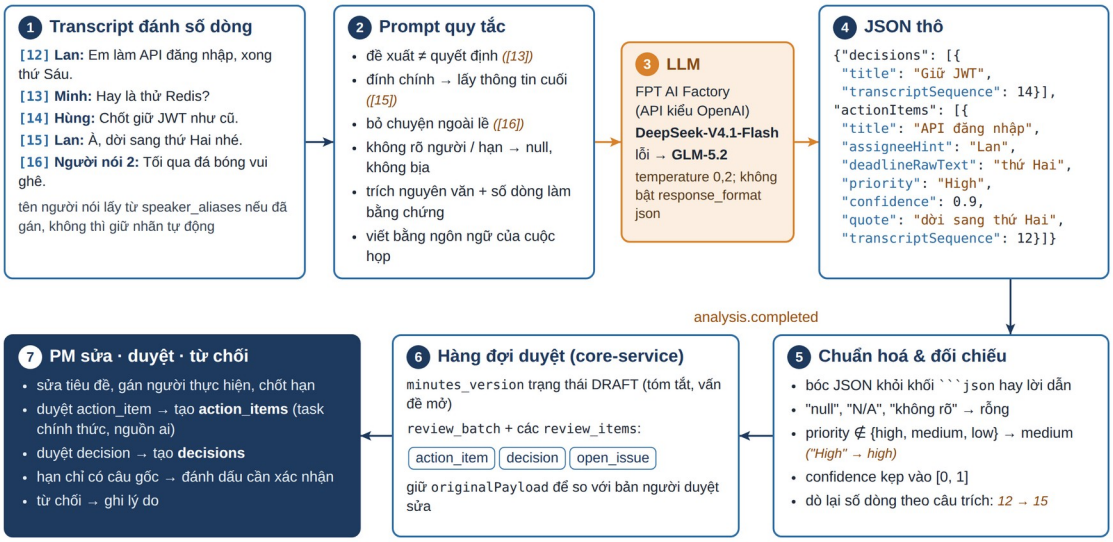
Prompt cho bài toán trích xuất gồm thông điệp hệ thống xác định vai trò (thư ký cuộc họp), các quy tắc nghiệp vụ, lược đồ đầu ra và bản ghi đã đánh số dòng theo dạng "[số thứ tự] Người nói: lời nói". Quy tắc phải nêu rõ những phân biệt con người coi là hiển nhiên: đề xuất chưa ai đồng ý không phải quyết định; có đính chính thì lấy thông tin sau cùng; bỏ chuyện ngoài lề; thiếu người thực hiện hay thời hạn thì để trống. Thời hạn được giữ nguyên văn ("thứ Sáu", "18/11"), việc quy đổi thành ngày cụ thể là bước riêng.

Một số dịch vụ hỗ trợ chế độ ép JSON hoặc giải mã ràng buộc theo lược đồ, nhưng không phải lúc nào cũng tương thích với mô hình suy luận: nhóm ghi nhận DeepSeek-V4.1-Flash trả nội dung rỗng khi bật chế độ JSON. Vì vậy cần lớp hậu xử lý vững: bóc JSON khỏi khối mã Markdown hay lời dẫn thừa, phát hiện câu trả lời bị cắt do hết giới hạn token, ép giá trị về miền hợp lệ (mức ưu tiên thuộc {high, medium, low}, độ tin cậy trong [0, 1]) và quy các chuỗi "null", "N/A" về rỗng. Nhiệt độ lấy mẫu thấp giúp kết quả ổn định hơn giữa các lần chạy.

1.4.4. Ảo giác của LLM, đối chiếu bằng chứng và con người trong vòng lặp

Ji và cộng sự [14] phân biệt ảo giác nội tại (mâu thuẫn với nguồn) và ảo giác ngoại tại (không kiểm chứng được từ nguồn). Trong trích xuất biên bản, ảo giác biểu hiện thành người thực hiện hoặc thời hạn bịa ra, đề xuất bị ghi thành quyết định, thông tin cũ được giữ dù đã đính chính, hay trích dẫn sai dòng — nguy hiểm vì đầu ra trông hợp lý.

Biện pháp thứ nhất là buộc mỗi quyết định, đầu việc kèm câu trích nguyên văn và số dòng nguồn, rồi đối chiếu tự động sau khi chuẩn hoá văn bản; nếu câu trích không nằm trong dòng được chỉ ra thì dò lại dòng chứa nó. Biện pháp thứ hai là đưa con người vào vòng lặp (human-in-the-loop): kết quả của AI chỉ là đề xuất trong hàng đợi duyệt, chỉ đề xuất được người quản lý duyệt mới thành công việc chính thức (Hình 1.3). Độ tin cậy do mô hình tự khai không phải xác suất đã hiệu chỉnh, nên chỉ dùng để gợi ý mức chú ý.



Hình 1.3. *Trích xuất thông tin có cấu trúc bằng LLM, đối chiếu bằng chứng và duyệt bởi con người*

1.4.5. Dịch vụ suy luận và chủ quyền dữ liệu

Có ba cách đưa LLM vào hệ thống: tự vận hành mô hình trọng số mở trên GPU riêng, chủ động nhưng tốn kém; gọi dịch vụ nước ngoài, dễ dùng nhưng dữ liệu hội thoại rời khỏi phạm vi kiểm soát; hoặc dùng dịch vụ suy luận trong nước. FPT AI Factory [10] cung cấp suy luận phi máy chủ với giao diện tương thích OpenAI, tính phí theo token. Cần phân biệt mô hình do FPT tự vận hành, dữ liệu xử lý trên hạ tầng FPT, với mô hình bán lại (resell) từ Azure, Google, Alibaba…, khi đó yêu cầu được chuyển ra ngoài. Vì nội dung họp có thể nhạy cảm, MeetingMind chỉ dùng mô hình do FPT tự vận hành: DeepSeek-V4.1-Flash làm mô hình chính, GLM-5.2 làm dự phòng; đổi mô hình chỉ cần sửa cấu hình.

1.5. Kiến trúc microservices và hướng sự kiện

1.5.1. Microservices và cổng API

Kiến trúc microservices tổ chức hệ thống thành các service nhỏ, triển khai độc lập, mô hình hoá quanh miền nghiệp vụ và sở hữu dữ liệu riêng [21]; mỗi phần dùng được công nghệ phù hợp — dịch vụ AI viết bằng Python, nghiệp vụ viết bằng NestJS [20] — đổi lại là độ phức tạp của hệ phân tán. Nguyên tắc cơ sở dữ liệu riêng cho mỗi service [26] đòi hỏi mỗi bảng chỉ có một service được ghi; service khác lấy dữ liệu qua API hoặc sự kiện. Cổng API (API Gateway) là điểm vào duy nhất, xác thực token, định tuyến và che giấu cấu trúc nội bộ; trong MeetingMind, cổng chỉ kiểm tra token, còn quyết định quyền nghiệp vụ thuộc về service lõi.

1.5.2. Giao tiếp bất đồng bộ qua RabbitMQ

Phiên âm một giờ họp có thể mất nhiều phút, nên không thể giữ yêu cầu HTTP chờ kết quả; với hàng đợi thông điệp, bên gửi đặt việc vào hàng đợi rồi trả lời ngay, bên xử lý nhận việc theo năng lực của mình. RabbitMQ [24] là bộ môi giới thông điệp theo giao thức AMQP 0-9-1: bên gửi đẩy thông điệp vào exchange, exchange định tuyến tới hàng đợi theo khoá định tuyến (routing key) và quy tắc liên kết (binding); bên nhận gửi xác nhận (acknowledgement) sau khi xử lý xong. Hàng đợi và thông điệp lưu bền (durable) giúp không mất việc khi bộ môi giới khởi động lại; tham số prefetch giới hạn số thông điệp một bên nhận giữ cùng lúc. Thông điệp chỉ nên mang tham chiếu (mã cuộc họp, số phiên bản, khoá đối tượng) thay vì cả tệp âm thanh; tệp lớn đặt trong kho lưu trữ đối tượng tương thích S3 như MinIO.

1.5.3. Ngữ nghĩa giao nhận và tính lũy đẳng

Hệ thống thông điệp có ba mức bảo đảm: nhiều nhất một lần (at-most-once) có thể mất thông điệp; ít nhất một lần (at-least-once) không mất nhưng có thể trùng; đúng một lần (exactly-once) rất khó đạt trong hệ phân tán. Với xác nhận thủ công, nếu bên nhận gặp sự cố sau khi xử lý nhưng trước khi xác nhận, RabbitMQ giao lại thông điệp. Kleppmann [16] chỉ ra rằng hiệu quả "đúng một lần" đạt được bằng giao nhận ít nhất một lần kết hợp xử lý lũy đẳng (idempotent): xử lý nhiều lần cho cùng kết quả như một lần.

Mẫu bên nhận lũy đẳng [26] ghi mã định danh mỗi thông điệp đã xử lý vào một bảng (trong MeetingMind là `processed_messages`) trong cùng giao dịch với thay đổi nghiệp vụ; gặp lại mã đã có thì bỏ qua. Ở phía yêu cầu, mỗi công việc xử lý mang một khoá lũy đẳng (idempotency key) duy nhất để yêu cầu gửi lại không tạo việc trùng.

1.5.4. Mẫu transactional outbox

Khi một service vừa cập nhật cơ sở dữ liệu vừa phát sự kiện, hai thao tác không nằm trong cùng giao dịch: một bên thành công còn bên kia thất bại sẽ làm hệ thống mất nhất quán — vấn đề ghi kép (dual write). Mẫu transactional outbox [26] ghi sự kiện vào bảng outbox trong cùng giao dịch cục bộ với thay đổi nghiệp vụ; một tiến trình chuyển phát riêng đọc bảng outbox, đẩy sự kiện lên bộ môi giới và đánh dấu đã gửi. Vì sự kiện có thể bị gửi lại khi tiến trình chuyển phát lỗi giữa chừng, mẫu này đi kèm bên nhận lũy đẳng; kết hợp hai mẫu, mỗi yêu cầu phiên âm hay phân tích có hiệu quả đúng một lần, kể cả khi service khởi động lại.

1.6. Thu nhận nội dung họp trực tuyến

1.6.1. WebRTC và xử lý âm thanh trên trình duyệt

WebRTC [34] là chuẩn truyền âm thanh, hình ảnh thời gian thực giữa các trình duyệt. Đối tượng `RTCPeerConnection` thiết lập kết nối qua cơ chế ICE, truyền dữ liệu đã mã hoá và phát sự kiện `track` mỗi khi nhận một luồng từ phía bên kia. Dịch vụ họp nhiều người thường dùng máy chủ trung gian chuyển tiếp luồng, nhưng ở phía trình duyệt, âm thanh của người khác vẫn đến dưới dạng các luồng từ xa của `RTCPeerConnection`.

Web Audio API với `AudioContext` cho phép trộn nhiều nguồn âm thanh, còn MediaRecorder ghi một luồng thành tệp nén (ví dụ WebM mã hoá Opus) theo từng khúc. Nhờ đó bot có thể ghi âm ngay trong trình duyệt của nó: bọc hàm khởi tạo `RTCPeerConnection` để bắt mọi luồng âm thanh từ xa, trộn lại rồi ghi — tức ghi đúng những gì một người tham dự nghe được.

1.6.2. Tự động hoá trình duyệt với Playwright và Xvfb

Playwright [19] là thư viện điều khiển Chromium, Firefox và WebKit bằng mã nguồn, cho phép chèn mã chạy trước mọi mã của trang (init script), cho mã trên trang gọi ngược về chương trình điều khiển và dùng hồ sơ trình duyệt lưu bền để giữ phiên đăng nhập. Chế độ không giao diện (headless) dễ bị nền tảng nhận ra, nên bot họp thường chạy trình duyệt có giao diện trên màn hình ảo: Xvfb (X virtual framebuffer) là máy chủ đồ hoạ X vẽ vào bộ nhớ, cho phép chạy trình duyệt đầy đủ trong container; PulseAudio cung cấp thiết bị âm thanh ảo.

Các nền tảng họp có cơ chế phát hiện trình duyệt tự động; thực tế Google Meet chặn khách ẩn danh dùng trình duyệt tự động, nên bot cần tài khoản Google đã đăng nhập. Về đạo đức và tuân thủ, bot không vượt cơ chế kiểm soát của nền tảng: hiện đúng tên, phải được chủ phòng cho vào và dừng khi bị từ chối.

1.6.3. Tiện ích mở rộng Chrome Manifest V3

Manifest V3 [11] là thế hệ nền tảng tiện ích hiện hành của Chrome. Trang nền chạy liên tục được thay bằng service worker hướng sự kiện, có thể bị dừng khi rảnh, nên trạng thái phải lưu vào `chrome.storage`; mã thực thi phải đóng gói sẵn, không tải từ xa; quyền truy cập trang được khai báo tường minh. Content script được tiêm vào các trang khớp mẫu địa chỉ và chạy trong "thế giới cô lập" (isolated world): dùng chung DOM với trang nhưng không chung biến JavaScript, trao đổi với service worker bằng truyền thông điệp.

Đọc phụ đề Google Meet là bài toán quan sát DOM. Tên lớp CSS của Meet bị làm rối và đổi định kỳ, nên cần dựa vào cấu trúc DOM và thuộc tính trợ năng ổn định hơn như `role`, `aria-label`; vùng phụ đề còn bị dựng lại sau khoảng im lặng, nên quét định kỳ bền hơn gắn bộ quan sát cố định. Phụ đề là kết quả nhận dạng của chính Meet, không kèm âm thanh: đường thu nhận này nhẹ, có sẵn tên người nói, nhưng chất lượng phụ thuộc nền tảng.

1.7. Tích hợp nền tảng quản lý công việc

Mục này trình bày cơ sở lý thuyết và kết quả khảo sát cho việc đồng bộ ra nền tảng ngoài. Ở thời điểm báo cáo, MeetingMind đã hiện thực đồng bộ một chiều công việc đã duyệt sang Trello và thông báo công việc mới qua Telegram (trình bày ở chương Xây dựng hệ thống); đồng bộ Jira và cơ chế kết nối OAuth theo từng workspace vẫn ở mức thiết kế.

1.7.1. OAuth 2.0 và Webhook

OAuth 2.0 [12] là khung uỷ quyền cho phép ứng dụng truy cập tài nguyên của người dùng trên dịch vụ khác mà không cần mật khẩu, với bốn vai trò: chủ sở hữu tài nguyên, ứng dụng khách, máy chủ uỷ quyền và máy chủ tài nguyên. Trong luồng mã uỷ quyền (authorization code), người dùng đồng ý với một phạm vi quyền (scope) tại máy chủ uỷ quyền; ứng dụng nhận mã rồi đổi lấy access token có thời hạn ngắn và refresh token để gia hạn. Token phải lưu bảo mật phía máy chủ, không trả về giao diện.

Webhook là cơ chế đẩy ngược: thay vì ứng dụng hỏi định kỳ (polling), nền tảng gọi một địa chỉ HTTP đã đăng ký mỗi khi có sự kiện; bên nhận cần xác minh nguồn gửi và xử lý lũy đẳng vì nền tảng có thể gửi lại.

1.7.2. Jira, Trello và Telegram

Jira Cloud cung cấp REST API phiên bản 3 [1], xác thực bằng OAuth 2.0 hoặc API token. Đối tượng trung tâm là issue thuộc một project, có tiêu đề, mô tả theo định dạng tài liệu Atlassian (Atlassian Document Format), người được giao định danh bằng `accountId`, hạn chót và mức ưu tiên; trạng thái đổi qua các bước chuyển (transition). Trello [2] tổ chức dữ liệu theo bảng (board), danh sách (list) và thẻ (card); tạo thẻ cần chỉ định danh sách đích, kèm mô tả, hạn và thành viên; quyền truy cập cấp bằng khoá API kèm token do người dùng uỷ quyền. Telegram Bot API [32] cho phép bot gửi tin qua HTTPS bằng phương thức `sendMessage`, nhận cập nhật bằng `getUpdates` hoặc webhook; bot không thể nhắn trước cho người chưa mở trò chuyện với nó, nên người

dùng phải khởi động bot trước để hệ thống lưu định danh cuộc trò chuyện. Bảng 1.3 tóm tắt các đặc điểm này.

Bảng 1.3. *Đặc điểm các nền tảng tích hợp được khảo sát*

Nền tảng	Cơ chế cấp quyền	Đối tượng chính	Thao tác dự kiến	Nhận thay đổi
Jira Cloud	OAuth 2.0 hoặc API token	Project, issue, transition	Tạo issue từ công việc đã duyệt	Webhook
Trello	Khoá API kèm token người dùng	Board, list, card	Tạo thẻ trong danh sách đích	Webhook
Telegram	Token của bot	Chat, message	Thông báo công việc mới, sắp đến hạn	getUpdates hoặc webhook

1.8. Kiểm soát truy cập

1.8.1. Mô hình RBAC

Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control, RBAC) [27] gán quyền cho vai trò và vai trò cho người dùng. Họ mô hình RBAC96 gồm RBAC0 là lõi (người dùng, vai trò, quyền, phiên làm việc), RBAC1 thêm phân cấp vai trò, RBAC2 thêm ràng buộc như phân tách nhiệm vụ, RBAC3 kết hợp cả hai. RBAC dễ quản trị và kiểm toán, nhưng vai trò toàn cục không diễn đạt được ngữ cảnh: muốn nói "PM của dự án A nhưng chỉ là thành viên dự án B" phải sinh vai trò riêng cho từng dự án, dẫn tới bùng nổ số vai trò.

1.8.2. Mô hình ABAC

Kiểm soát truy cập dựa trên thuộc tính (Attribute-Based Access Control, ABAC), theo NIST SP 800-162 [13], quyết định cho phép hay từ chối dựa trên thuộc tính của chủ thể, đối tượng, môi trường và tập chính sách biểu diễn bằng các thuộc tính đó. ABAC diễn đạt được điều kiện như "người tạo cuộc họp", "người được giao việc", "tài nguyên đang ở trạng thái nháp", nhưng chính sách phức tạp khó kiểm toán và khó trả lời câu hỏi "ai đang có quyền với tài nguyên này".

1.8.3. Phân quyền theo vai trò kết hợp phạm vi tài nguyên

Giải pháp dung hoà là gán vai trò trong phạm vi một tài nguyên, kết hợp thừa kế theo cây tài nguyên và điều kiện thuộc tính bổ sung: người dùng có vai trò ở từng tầng — hệ thống, workspace, dự án, cuộc họp độc lập; cuộc họp thuộc dự án kế thừa quyền của dự án; một số quyền kèm điều kiện như "chỉ với cuộc họp mình tạo". Cách làm này giữ tính dễ hiểu của RBAC, tránh bùng nổ vai trò, chỉ dùng thuộc tính ở nơi cần.

Đi kèm là nguyên tắc đặc quyền tối thiểu (least privilege) — mặc định từ chối — và tách quyền quản trị khỏi quyền đọc nội dung: quản trị viên hệ thống không mặc nhiên đọc được cuộc họp riêng tư. Bảng 1.4 so sánh ba cách tiếp cận.

Bảng 1.4. *So sánh các mô hình kiểm soát truy cập*

Tiêu chí	RBAC	ABAC	Vai trò theo phạm vi tài nguyên
Căn cứ quyết định	Vai trò toàn cục	Thuộc tính và chính sách	Vai trò tại tài nguyên hoặc tài nguyên cha, cộng điều kiện bổ sung
Diễn đạt ngữ cảnh	Kém, dễ bùng nổ vai trò	Rất tốt	Tốt với dữ liệu phân cấp
Độ phức tạp quản trị	Thấp	Cao	Trung bình
Kiểm toán "ai có quyền gì"	Dễ	Khó	Tương đối dễ
Phù hợp	Hệ thống ít ngữ cảnh	Chính sách động, nhiều điều kiện	Ứng dụng nhiều tổ chức, dữ liệu phân cấp

1.9. Kết luận chương

Chương này đã trình bày cơ sở cho từng khâu của MeetingMind. faster-whisper với lượng tử hoá int8 cho phép nhận dạng tiếng nói trên CPU, kèm hai vấn đề cần xử lý riêng là chuyển mã Việt–Anh và ảo giác ở đoạn lặng, ồn. Vector giọng ERes2Net kết hợp gom cụm phân cấp tự suy ra số người nói, còn nói chồng vẫn là giới hạn của cách tiếp cận một kênh. LLM nắm được các tình huống khó của hội thoại nhưng cần quy tắc rõ, đối chiếu bằng chứng và con người duyệt. Microservices hướng sự kiện với outbox và xử lý lũy đẳng phù hợp cho tác vụ AI chạy lâu; phân quyền theo vai trò kết hợp phạm vi tài nguyên dung hoà RBAC và ABAC. Đây là cơ sở cho phân tích và thiết kế hệ thống ở chương tiếp theo.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình phân tích yêu cầu và các quyết định thiết kế của MeetingMind: từ khảo sát người dùng và quy trình họp, tới kiến trúc, luồng sự kiện, cơ sở dữ liệu, phân quyền, bộ phân tích AI, giao diện và module tích hợp. Bộ tài liệu thiết kế gốc được soạn cho toàn bộ phạm vi sản phẩm, rộng hơn khối lượng làm được trong sáu tháng, nên ở các bảng yêu cầu và thành phần, chúng tôi ghi trạng thái đã đối chiếu với mã nguồn theo ba mức: *Đã có* (chạy trong luồng chính), *Một phần* (có phần lõi, chưa trọn chức năng) và *Thiết kế* (mới có đặc tả, bảng dữ liệu hoặc giao diện mẫu).

2.1. Khảo sát và xác định yêu cầu

2.1.1. Đối tượng người dùng

Yêu cầu được thu thập từ hai nguồn. Thứ nhất là khảo sát các sản phẩm dùng bot vào phòng họp như Otter.ai [22], Fireflies.ai [9] và sản phẩm đọc phụ đề như Tactiq [30]; chúng ghi nhận và tóm tắt tốt nhưng dừng ở danh sách việc chưa qua kiểm duyệt, ít chú trọng việc ai được xem gì và chưa tối ưu cho tiếng Việt. Thứ hai là phân tích quy trình họp của các nhóm phát triển phần mềm nhỏ, được hệ thống hoá thành tài liệu tổng quan dự án gồm mười lăm mục. Kết quả là bốn nhóm người dùng và hai tác nhân hệ thống ở Bảng 2.1.

Bảng 2.1. *Các tác nhân của hệ thống*

Tác nhân	Mục tiêu chính
Owner/Admin workspace	Quản lý thành viên, dự án, cấu hình, kết nối tích hợp
PM/Trưởng nhóm	Tổ chức họp, gán tên người nói, duyệt biên bản và đề xuất, phân công
Thành viên	Xem cuộc họp được chia sẻ, ghi chú, cập nhật việc được giao
Khách mời	Chỉ xem tài liệu được chia sẻ riêng; có thể không có tài khoản
Bot ghi âm	Vào phòng Google Meet, ghi âm, tải bản ghi lên theo lệnh của Core
Dịch vụ AI	Phiên âm, tách người nói, đề xuất biên bản và công việc

Khảo sát cũng cho thấy cần tách *quyền trên hệ thống*, *vai trò chuyên môn* (Backend, QA…) và *vai trò trong cuộc họp hoặc đầu việc* (chủ trì, người thực hiện). Một người có thể là thành viên thường của workspace nhưng là PM của dự án A; AI được gợi ý người thực hiện nhưng không bao giờ được thay đổi quyền.

2.1.2. Quy trình họp mẫu

Trước họp, người tổ chức tạo cuộc họp trong một dự án hoặc độc lập, nhập agenda, người tham dự và chọn cách thu nhận: mời bot vào Google Meet, bật tiện ích đọc phụ đề, ghi âm tại phòng họp, hoặc tải bản ghi có sẵn. *Trong họp*, hệ thống thu

nhận nội dung; người dự ghi chú cá nhân hoặc chia sẻ. *Sau họp*, hệ thống phiên âm, tách lượt nói, sinh biên bản nháp và đề xuất công việc; người chủ trì gắn tên cho các nhãn "Người nói 1, 2…", sửa và duyệt; chỉ đề xuất đã duyệt mới thành công việc chính thức.

Tình huống chuẩn cho AI: Minh nói *"Nam phụ trách API đăng nhập, thứ Sáu gửi bản đầu tiên để Lan kiểm thử."* Kết quả mong đợi là công việc giao cho Nam, giữ nguyên "thứ Sáu" để người duyệt chuẩn hoá, kèm câu nguồn; hạn kiểm thử của Lan không được nêu nên để trống. Từ đó rút ra các quy tắc nghiệp vụ: phân biệt người nói với người được nhắc tới, đề xuất với cam kết; có đính chính thì lấy thông tin sau cùng; thiếu thông tin thì để trống; mọi công việc có nguồn; chỉ công việc đã duyệt mới được giao.

2.2. Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo vòng đời cuộc họp ở Bảng 2.2. Thứ tự ưu tiên theo phân kỳ đã đặt: phần lõi từ thu nhận tới bảng công việc làm trước; tích hợp ngoài, cảnh báo nâng cao và hỏi đáp làm sau.

Bảng 2.2. *Yêu cầu chức năng và trạng thái hiện thực*

Nhóm	Chức năng	Trạng thái
Tài khoản, workspace	Đăng ký kèm tạo workspace, đăng nhập, đổi mật khẩu; mời thành viên với vai trò	Đã có
	Tạo dự án; quản lý thành viên dự án	Một phần
	Mẫu biên bản, lưu trữ, nhắc việc, nhật ký quản trị	Thiết kế
Cuộc họp	Tạo cuộc họp kèm agenda, người dự, nguồn thu nhận; danh sách theo trạng thái	Đã có
	Ghi chú cá nhân/chia sẻ; chủ ghi chú cho phép đưa vào biên bản	Đã có
	Kết nối lịch, bot tự vào họp theo lịch	Thiết kế
Thu nhận nội dung	Tải tệp ghi âm, chống trùng theo SHA-256, URL ký để tải lại	Đã có
	Bot ghi âm Google Meet; tiện ích Chrome đọc phụ đề	Đã có
	Ghi âm trên trình duyệt tại phòng họp, ghi khi mất mạng	Thiết kế
Transcript, người nói	Phiên âm tiếng Việt xen tiếng Anh, tách người nói, đánh dấu nói chồng	Đã có
	Transcript theo phiên bản; gán nhãn người nói, giữ "chưa xác định"	Đã có
	Sửa nội dung đoạn, gộp/tách nhãn, phát lại audio đồng bộ	Một phần
	Nhận diện qua mẫu giọng đã đăng ký	Thiết kế
Biên bản, duyệt	Biên bản nháp do AI sinh; nháp → chờ duyệt → đã duyệt	Đã có
	Hàng đợi duyệt: sửa, duyệt, từ chối, giữ bản gốc của AI	Đã có

Nhóm	Chức năng	Trạng thái
	Xuất biên bản ra tệp hoặc Notion/Google Docs	Thiết kế
Công việc	Duyệt đề xuất sinh công việc; Kanban 4 trạng thái; bình luận	Đã có
	Liên kết công việc với câu nguồn	Một phần
	Lịch sử thay đổi, phụ thuộc giữa công việc	Thiết kế
Tích hợp ngoài	Đồng bộ một chiều công việc đã duyệt sang Trello, thông báo Telegram (đã có); đồng bộ sang Jira (chưa có)	Một phần
Cảnh báo, hỏi đáp	Cảnh báo theo quy tắc: quá hạn, bị chặn, thiếu người phụ trách	Một phần
	Tìm kiếm trong transcript một cuộc họp	Đã có
	Hỏi đáp có trích nguồn (RAG), lọc quyền trước truy xuất	Thiết kế

Các mục "Một phần" có lý do cụ thể: sửa đoạn transcript đã có API nhưng chưa có giao diện; liên kết nguồn của công việc đi qua đề xuất gốc, bảng trích dẫn riêng chưa được ghi tự động; cảnh báo được tính ở phía giao diện, chưa có bộ quét phía máy chủ; tích hợp ngoài mới có Trello và Telegram, cấu hình qua biến môi trường, chưa có Jira. Nhóm chủ động làm trọn luồng lõi trước, vì nếu luồng này sai thì đồng bộ ra ngoài chỉ nhân bản sai sót.

2.3. Yêu cầu phi chức năng

Với hệ thống xử lý hội thoại nội bộ bằng mô hình AI có thể sai, yêu cầu phi chức năng quan trọng ngang chức năng; Bảng 2.3 tổng hợp các giải pháp.

Bảng 2.3. *Yêu cầu phi chức năng và giải pháp thiết kế*

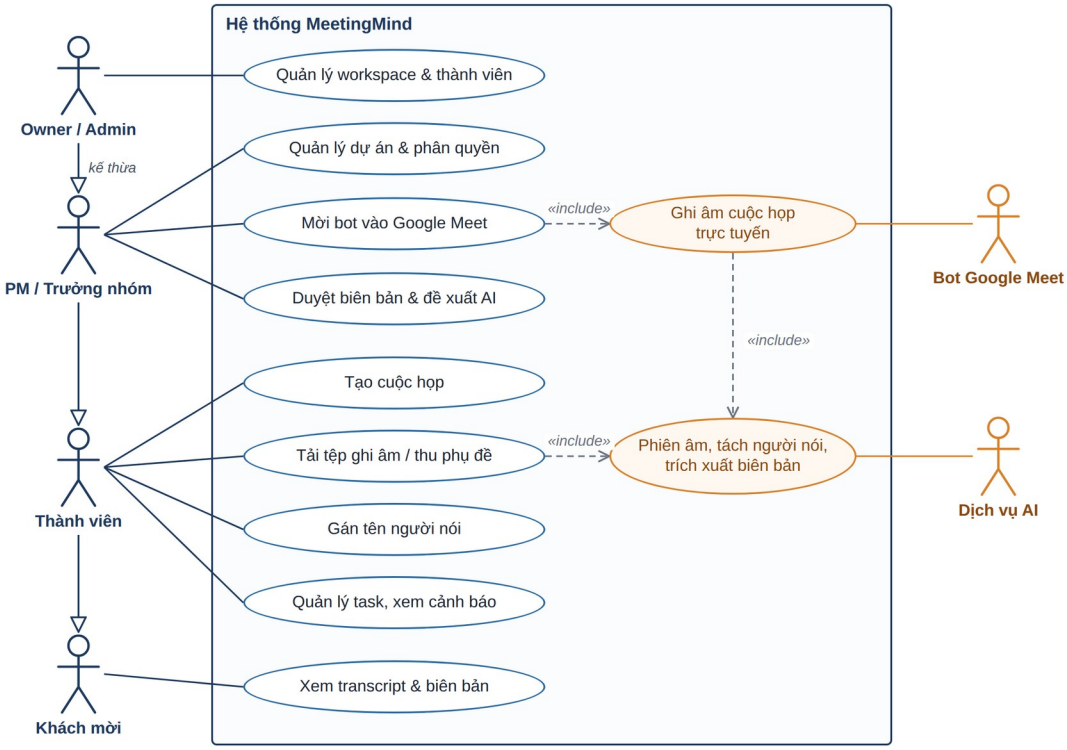
Nhóm	Giải pháp	Trạng thái
Bảo mật	JWT, mật khẩu băm bcrypt, phân quyền theo phạm vi tập trung ở Core; khoá API ở biến môi trường	Đã có
	Mã hoá dữ liệu hội thoại khi lưu	Chưa triển khai
Toàn vẹn dữ liệu	Transcript, biên bản theo phiên bản; đề xuất giữ bản gốc của AI	Đã có
Tin cậy	Outbox, bảng chống xử lý lặp, khoá idempotency cho job	Đã có
	Tự động thử lại job lỗi, hàng đợi thư chết	Thiết kế
Hiệu năng	Phiên âm, phân tích chạy nền qua hàng đợi; giao diện hiển thị trạng thái từng bước	Đã có
Mở rộng	Worker AI và bot tách container, nhận việc từ hàng đợi bền	Đã có
Thay thế mô hình AI	Giao diện Transcriber/Analyzer chọn bằng biến môi trường; mô hình chính và dự phòng	Đã có
Quan trắc	correlationId xuyên suốt; Prometheus, Grafana	Một phần

Về bảo mật, chúng tôi đã vá các lỗ hổng của mã khung ban đầu: route quản lý người dùng mở không cần đăng nhập, đổi mật khẩu không kiểm tra mật khẩu cũ, người dùng tự đặt chủ sở hữu workspace. Điểm thiếu lớn nhất là mã hoá khi lưu: transcript và audio hiện chỉ được bảo vệ bằng phân quyền và tài khoản cơ sở dữ liệu riêng.

Về tin cậy, RabbitMQ chỉ bảo đảm giao *ít nhất một lần* [24], nên thiết kế dùng mẫu *transactional outbox* [26] cùng chống lặp phía người nhận và khoá idempotency phía job [16]. Về thay thế mô hình, khi chuyển từ bản giả lập sang faster-whisper [29] và mô hình của FPT AI Factory [10], luồng điều phối ở Core không phải sửa.

2.4. Mô hình ca sử dụng

Hình 2.1 trình bày sơ đồ ca sử dụng tổng quát, trong đó PM tham gia nhiều ca nhất, còn bot và dịch vụ AI chỉ hành động theo lệnh của Core. Bảng 2.4, Bảng 2.5 và Bảng 2.6 đặc tả ba ca tiêu biểu về phối hợp giữa con người, bot và AI.



Hình 2.1. *Sơ đồ ca sử dụng tổng quát của hệ thống MeetingMind*

Bảng 2.4. *Đặc tả ca sử dụng "Mời bot vào cuộc họp"*

Mục	Nội dung
Tác nhân	PM, Owner/Admin, MANAGER hoặc người tạo cuộc họp; bot ghi âm
Tiền điều kiện	Cuộc họp có đường dẫn Google Meet; người dùng có quyền `meeting.recording.control`
Luồng chính	1. Bấm "Mời bot". 2. Core tạo phiên bot `queued`, ghi `bot.join.requested` vào outbox. 3. Bot vào phòng, báo `waiting_admission`; được cho vào thì báo `recording`. 4. Họp kết thúc hoặc bot ở một mình 60 s thì rời phòng, báo

Mục	Nội dung
	`left`. 5. Bot tải bản ghi lên MinIO, phát `recording.completed`. 6. Core đăng ký tệp, khởi động phiên âm
Luồng thay thế	Đã có phiên đang chạy: trả về phiên đó. Dừng trước khi vào phòng: `cancelled`. Bị từ chối hoặc chờ quá 300 s: `failed` kèm lý do

Bảng 2.5. *Đặc tả ca sử dụng "Duyệt đề xuất công việc"*

Mục	Nội dung
Tác nhân	PM, Owner/Admin, MANAGER (người tạo cuộc họp chỉ được sửa)
Tiền điều kiện	Đã có lô duyệt với các đề xuất `pending`
Luồng chính	1. Mở "Task cần duyệt"; đề xuất hiển thị cạnh câu nguồn, gợi ý người thực hiện, cụm hạn gốc. 2. Người duyệt sửa tiêu đề, chọn người thực hiện, ngày hạn, ưu tiên rồi bấm "Duyệt". 3. Core kiểm tra `review.approve`, tạo công việc `todo`, bật cờ cần xác nhận nếu chưa có ngày, chuyển đề xuất sang `approved`
Luồng thay thế	Từ chối kèm lý do: `rejected`. Đề xuất loại quyết định: tạo bản ghi quyết định. Duyệt lặp: ràng buộc duy nhất chặn công việc thứ hai

Bảng 2.6. *Đặc tả ca sử dụng "Gán tên người nói"*

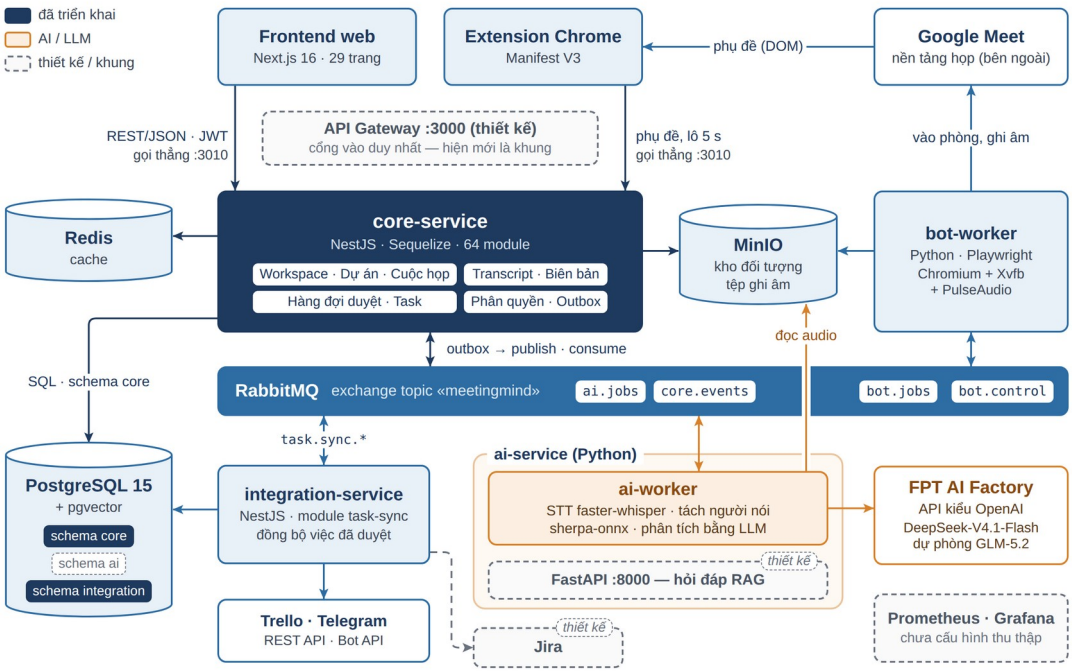
Mục	Nội dung
Tác nhân	PM, Owner/Admin, MANAGER/EDITOR hoặc người tạo cuộc họp
Tiền điều kiện	Có phiên bản transcript hiện hành với nhãn người nói tự động
Luồng chính	1. Mở tab transcript, chọn thành viên cho một nhãn. 2. Core kiểm tra `transcript.update` và tư cách thành viên. 3. Mọi đoạn mang nhãn chuyển sang `confirmed`; ghi `speaker_aliases` kèm người xác nhận. 4. Công việc của cuộc họp được đánh dấu "cần rà soát", không bị đổi người âm thầm
Luồng thay thế	Chọn "Giữ chưa xác định": không đổi. Người được chọn ngoài workspace: báo lỗi

Phân biệt được hai giọng không có nghĩa là biết họ là ai: đường bot và tải tệp chỉ trả nhãn ở trạng thái `unknown`; tên hiển thị trên Google Meet từ đường phụ đề có thể đặt tuỳ ý nên chỉ ở trạng thái `suggested` cho tới khi được xác nhận.

2.5. Kiến trúc tổng thể

2.5.1. Các thành phần

MeetingMind theo kiến trúc nhiều dịch vụ [21], giao tiếp đồng bộ qua HTTP và bất đồng bộ qua RabbitMQ. Hình 2.2 mô tả kiến trúc tổng thể, Bảng 2.7 liệt kê thành phần và trạng thái. Hệ thống chạy bằng Docker Compose với mười hai container; giao diện web chạy riêng.



Hình 2.2. *Kiến trúc tổng thể hệ thống MeetingMind*

Bảng 2.7. *Các thành phần của hệ thống*

Thành phần	Công nghệ	Trách nhiệm	Trạng thái
Giao diện web	Next.js	Cuộc họp, transcript, biên bản, duyệt, công việc	Đã có
Tiện ích Chrome	Manifest V3 [11]	Đọc phụ đề Google Meet	Đã có
api-gateway	NestJS	Cổng vào duy nhất, kiểm tra token	Khung
core-service	NestJS [20], Sequelize	Nghiệp vụ, phân quyền, điều phối	Đã có
ai-worker	Python, faster-whisper, sherpa-onnx	Phiên âm, tách người nói, phân tích LLM	Đã có
ai-service	FastAPI	API AI đồng bộ, dành cho hỏi đáp	Khung
bot-worker	Python, Playwright [19]	Vào Google Meet, ghi âm, tải lên	Đã có
integration-service	NestJS	Đồng bộ công việc sang Trello, báo Telegram; Jira, lịch (thiết kế)	Đã chạy (đồng bộ Trello/Telegram)
PostgreSQL, RabbitMQ, MinIO, Redis	pgvector; exchange topic; tương thích S3	Dữ liệu, hàng đợi, lưu audio, bộ đệm	Đã có
Prometheus, Grafana		Giám sát	Một phần

Cần nói rõ rằng api-gateway mới là khung NestJS chưa có nghiệp vụ; integration-service đã có mô-đun đồng bộ Trello và Telegram nhưng chưa có Jira và kết nối lịch; ai-service và ai-worker dùng chung mã nhưng toàn bộ phiên âm và phân tích chạy ở ai-worker.

2.5.2. Các nguyên tắc kiến trúc

Cổng vào duy nhất. Theo thiết kế, chỉ api-gateway được mở ra ngoài; gateway kiểm tra token và định tuyến nhưng không chứa logic phân quyền, để quyết định "ai được làm gì" chỉ nằm ở Core. Hiện nguyên tắc này mới là mục tiêu: trong môi trường phát triển, giao diện web và tiện ích Chrome gọi thẳng core-service.

Mỗi bảng một dịch vụ sở hữu. Chỉ dịch vụ sở hữu được ghi vào bảng; dịch vụ khác lấy dữ liệu qua API hoặc sự kiện; khoá ngoại chỉ dùng trong một schema. Nguyên tắc này xuất phát từ sự cố thật: khi bốn dịch vụ dùng chung một schema và một tài khoản, dịch vụ AI chiếm mất tên bảng `users` và các dịch vụ NestJS cùng sửa cấu trúc bảng lúc khởi động.

Core điều phối, AI chỉ đề xuất. Bot và AI không tự sửa bảng nghiệp vụ; chúng báo kết quả bằng sự kiện và Core quyết định có ghi thành dữ liệu chính thức hay không. Kết quả phân tích chỉ thành biên bản nháp và đề xuất chờ duyệt, như một cách đối phó với hiện tượng ảo giác của mô hình ngôn ngữ [14]: chấp nhận mô hình có thể sai và đặt con người vào điểm quyết định.

Thông điệp mang tham chiếu, trừ các ngoại lệ có chủ đích. Hợp đồng ban đầu quy định thông điệp chỉ mang định danh, số phiên bản và khoá đối tượng; lệnh phiên âm giữ đúng quy định, ai-worker tự tải tệp từ MinIO. Ở bước phân tích, quy định này mâu thuẫn với nguyên tắc sở hữu: transcript nằm trong schema `core` mà ai-worker không được đọc; cấp quyền đọc chéo thì phá ranh giới, gọi ngược API của Core thì tạo phụ thuộc đồng bộ. Chúng tôi quyết định cho `analysis.requested` mang kèm các đoạn transcript, vì văn bản nhỏ hơn nhiều so với audio, vì thông điệp tự đủ nên phân tích luôn chạy trên đúng phiên bản chụp lúc phát lệnh, và vì Core thay được nhãn tự động bằng tên đã gán. Lệnh đồng bộ `task.sync.requested` là ngoại lệ thứ hai với cùng lý do: integration-service không được đọc schema `core`, nên thông điệp mang ảnh chụp công việc tại thời điểm duyệt. Các ngoại lệ này được ghi rõ trong hợp đồng sự kiện.

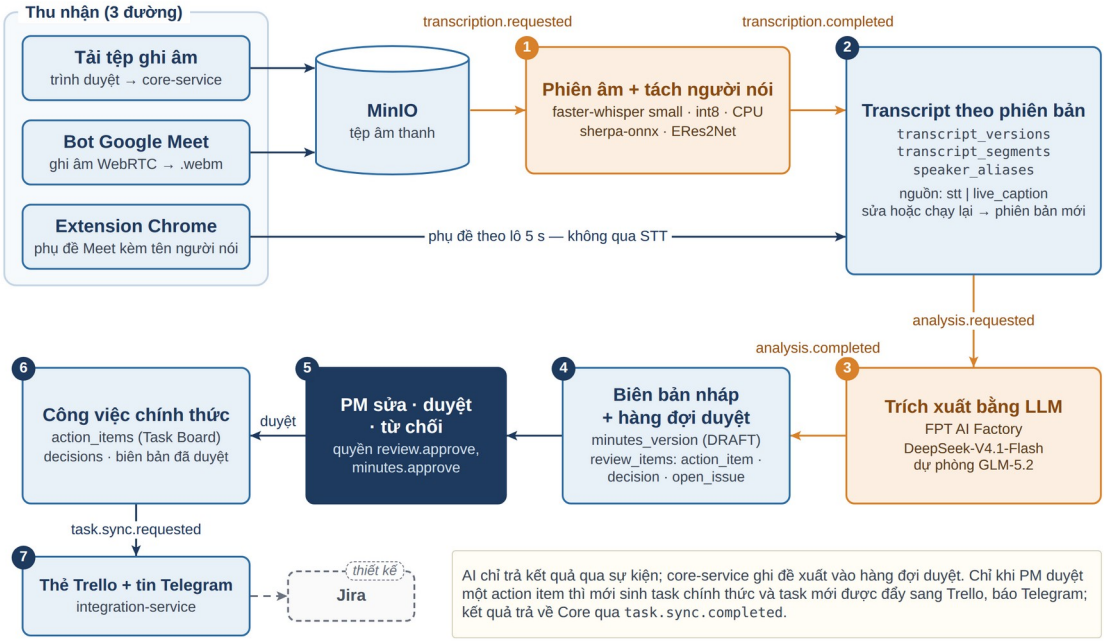
Outbox và idempotency. Core ghi sự kiện vào `outbox_events` thay vì gửi thẳng; bộ phát chạy mỗi 2 giây đẩy tối đa 50 sự kiện, thử lại tối đa 5 lần rồi đánh dấu lỗi, nên lệnh ghi nghiệp vụ không phụ thuộc RabbitMQ có đang sống hay không. Thiết kế yêu cầu sự kiện được ghi cùng giao dịch với thay đổi nghiệp vụ; dịch vụ outbox đã

nhận được giao dịch, nhưng các luồng hiện tại còn ghi job và sự kiện tuần tự, đây là điểm đang hoàn thiện. Phía nhận ghi cặp (mã thông điệp, bộ xử lý) vào `processed_messages` có ràng buộc duy nhất, trùng thì bỏ qua. Mỗi job có khoá idempotency duy nhất, ví dụ `{meetingId}:analyze:{transcriptVersionId}`, nên một phiên bản transcript chỉ được phân tích một lần.

2.6. Luồng xử lý từ âm thanh đến công việc

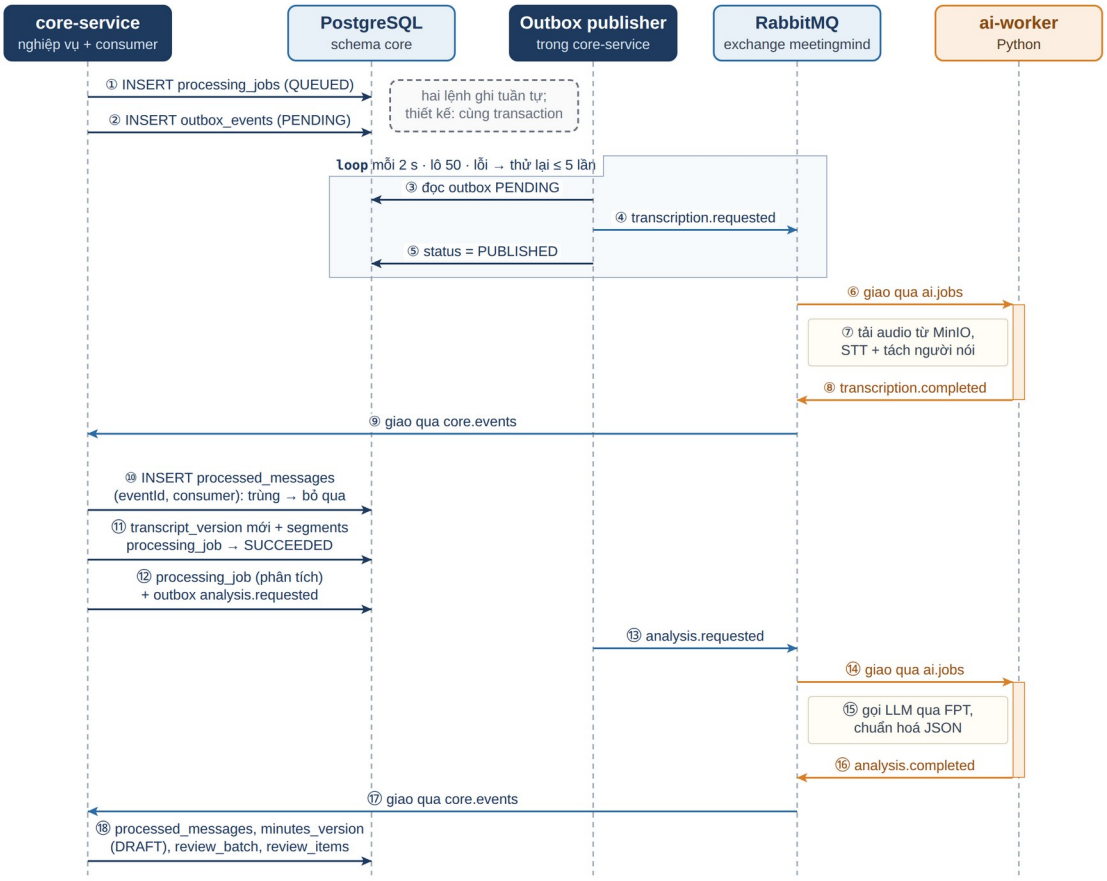
2.6.1. Luồng tổng quát

Hình 2.3 mô tả luồng xử lý, trong đó ba đường thu nhận hội tụ tại `transcript_versions`. Đường tải tệp và đường bot tạo tệp âm thanh trên MinIO: Core tạo job phiên âm, phát `transcription.requested`; ai-worker phiên âm bằng faster-whisper [25], tách người nói bằng vector giọng ERes2Net qua sherpa-onnx [7] [15], rồi phát `transcription.completed`. Đường phụ đề không có âm thanh: Core ghi từng lô phụ đề thành đoạn của một phiên bản nguồn `live_caption` và chốt khi người dùng kết thúc.



Hình 2.3. *Luồng xử lý từ âm thanh đến công việc*

Từ phiên bản transcript trở đi, ba đường dùng chung một luồng: Core phát `analysis.requested`; ai-worker gọi mô hình ngôn ngữ, phát `analysis.completed`; Core tạo biên bản nháp và một lô đề xuất gồm công việc, quyết định, vấn đề còn mở. Chỉ khi được duyệt, đề xuất mới thành bản ghi chính thức và mới được integration-service đẩy sang Trello, báo qua Telegram; đồng bộ Jira vẫn là thiết kế. Hình 2.4 chi tiết hoá trình tự sự kiện qua outbox và bước chống lặp.



Hình 2.4. *Sơ đồ tuần tự sự kiện giữa core-service và ai-worker*

2.6.2. Hợp đồng sự kiện

Mọi sự kiện có phần bao chung gồm `eventId` (để chống lặp), `eventType`, `eventVersion`, `occurredAt`, `workspaceId` và `correlationId` (xuyên suốt một cuộc họp để lần vết), đi qua exchange dạng topic `meetingmind`. Bot-worker có hàng đợi riêng cho lệnh dừng, vì phiên ghi âm kéo dài hàng giờ. Bảng 2.8 liệt kê các sự kiện; tám sự kiện phiên âm, phân tích và đồng bộ công việc được mô tả bằng JSON Schema dùng chung, bốn sự kiện của bot được định nghĩa đồng bộ trong mã TypeScript và Python.

Bảng 2.8. *Hợp đồng sự kiện giữa các dịch vụ*

Sự kiện	Bên phát → bên nhận	Nội dung chính
`transcription.requested`	Core → ai-worker	jobId, fileId, bucket, objectKey, language
`transcription.completed`	ai-worker → Core	runId, model, segments (nhãn, mốc thời gian, lời, cờ nói chồng)
`analysis.requested`	Core → ai-worker	jobId, transcriptVersionId, segments (số thứ tự, người nói, lời)
`analysis.completed`	ai-worker → Core	runId, model, summary, decisions, openIssues, actionItems
`transcription.failed`,	ai-worker →	jobId, meetingId, error

Sự kiện	Bên phát → bên nhận	Nội dung chính
`analysis.failed`	Core	
`bot.join.requested`, `bot.stop.requested`	Core → bot-worker	botSessionId; meetingId, joinUrl (lệnh vào phòng)
`bot.status.changed`	bot-worker → Core	botSessionId, status, detail, error
`recording.completed`	bot-worker → Core	botSessionId, bucket, objectKey, durationMs, endReason
`task.sync.requested`	Core → integration-service	actionItemId, meetingId, idempotencyKey, task (tiêu đề, mô tả, người thực hiện, hạn, câu hạn gốc, ưu tiên, tên cuộc họp, câu trích nguồn)
`task.sync.completed`	integration-service → Core	actionItemId, results (provider, status, externalId, externalUrl, error)

Trường `assigneeHint` là *tên như xuất hiện trong transcript*, không phải mã tài khoản; Core không tự ánh xạ, để hai người trùng tên không bị gán nhầm tài khoản.

2.6.3. Trạng thái xử lý

Bảng `processing_jobs` là nguồn sự thật cho từng job, còn `processingStatus` của cuộc họp là giá trị tổng hợp để hiển thị nhanh; cuộc họp và phiên bot có vòng đời riêng, tóm tắt ở Bảng 2.9.

Bảng 2.9. *Trạng thái của job, cuộc họp và phiên bot*

Đối tượng	Chuyển trạng thái đang dùng
Job xử lý	queued → succeeded; queued → failed khi nhận sự kiện lỗi
Xử lý cuộc họp	transcribing → summarizing → done; failed khi một job lỗi
Diễn ra cuộc họp	scheduled → in_progress → ended; tải tệp thì tạo thẳng ở ended
Phiên bot	queued → joining → waiting_admission → recording → left; failed; cancelled

Cuộc họp chỉ chuyển "hoàn tất xử lý" khi biên bản nháp và hàng đợi duyệt đã được tạo. Sự kiện trạng thái bot tới trễ bị bỏ qua nếu phiên đã kết thúc. Trạng thái `running` của job và `indexing` của cuộc họp đã khai báo nhưng chưa dùng, vì ai-worker chưa báo tiến độ trung gian và chưa có lập chỉ mục.

2.7. Thiết kế cơ sở dữ liệu

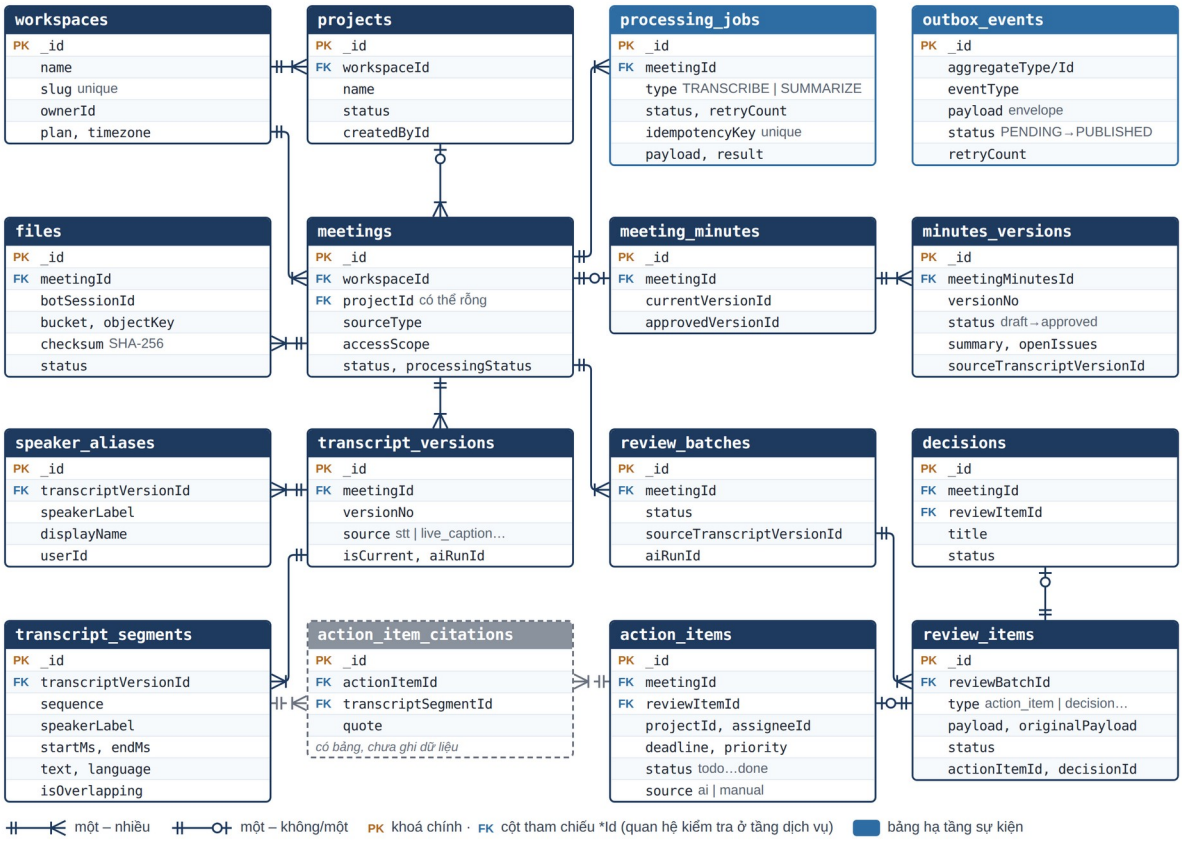
2.7.1. Tổ chức schema và tài khoản

Cơ sở dữ liệu dùng một máy chủ PostgreSQL có pgvector, chia ba schema ứng với ba tài khoản: `core` (`mm_core`), `integration` (`mm_integration`) và `ai` (`mm_ai`);

mỗi tài khoản chỉ ghi được vào schema của mình. Tài liệu thiết kế gồm 43 bảng: 35 bảng `core`, 4 bảng `integration` và 4 bảng `ai` (lần chạy AI, đoạn tài liệu có vector nhúng, hội thoại hỏi đáp); khi chi tiết hoá phân quyền, chúng tôi bổ sung bảng `meeting_access`. Schema `core` đã hiện thực đầy đủ; schema `integration` đã tạo đủ bốn bảng cùng mô-đun đồng bộ; schema `ai` mới ở mức thiết kế. Mọi bảng nghiệp vụ có `workspaceId` để tách dữ liệu theo workspace.

2.7.2. Các bảng chính

Hình 2.5 trình bày quan hệ các bảng chính của schema `core`, Bảng 2.10 mô tả các cột quan trọng. Cuộc họp là thực thể trung tâm: có nhiều phiên bản transcript và biên bản; mỗi lần phân tích tạo một lô duyệt; mỗi đề xuất đã duyệt sinh tối đa một công việc.



Hình 2.5. *Sơ đồ quan hệ các bảng chính của schema core*

Bảng 2.10. *Các bảng chính của schema core*

Bảng	Cột quan trọng
`meetings`	projectId, sourceType, accessScope, status, processingStatus, createdById
`transcript_versions`	versionNo, source, aiRunId, isCurrent; duy nhất (meetingId, versionNo)
`transcript_segments`	speakerLabel, speakerUserId, identityStatus, startMs, endMs, text, rawText, isOverlapping, sequence

Bảng	Cột quan trọng
speaker_aliases	speakerLabel, displayName, userId, confirmedById; duy nhất (transcriptVersionId, speakerLabel)
meeting_minutes, minutes_versions	currentVersionId, approvedVersionId; summary, openIssues, status, sourceTranscriptVersionId
review_batches, review_items	sourceTranscriptVersionId; type, originalPayload, payload, status, actionItemId
action_items	reviewItemId (duy nhất), assigneeId, deadline, deadlineRawText, deadlineNeedsConfirm, status, needsReviewReason
action_item_citations	transcriptSegmentId, transcriptVersionId, startMs, endMs, quote
processing_jobs	type, status, idempotencyKey (duy nhất), inputVersion, result
outbox_events, processed_messages	eventType, payload, status, retryCount; (messageId, consumer) duy nhất

2.7.3. Lưu phiên bản transcript và biên bản

Quyết định quan trọng nhất là lưu transcript và biên bản theo phiên bản, không ghi đè: nếu transcript bị sửa hoặc phân tích chạy lại, câu nguồn của công việc đã duyệt không được đổi nghĩa âm thầm. Mỗi lần nhận kết quả phiên âm, Core tạo phiên bản mới với số thứ tự tăng dần, ghi nguồn (`stt`, `live_caption`) và chuyển cờ `isCurrent` để đúng một phiên bản là hiện hành; mỗi đoạn lưu cả `text` hiện tại và `rawText` gốc. Biên bản đã duyệt không sửa trực tiếp mà phải mở phiên bản nháp mới rồi duyệt lại; chạy lại phân tích cũng chỉ sinh bản nháp mới. Tương tự, `review_items` lưu `originalPayload` (bản AI sinh) bên cạnh `payload` (sau khi sửa), vừa để truy vết vừa để đo mức người duyệt phải chỉnh sửa đầu ra của mô hình.

2.7.4. Nhãn người nói và bí danh

Mỗi đoạn giữ `identityStatus` với ba giá trị `unknown`, `suggested`, `confirmed`, bảo đảm nguyên tắc "không đủ căn cứ thì giữ chưa xác định". Bảng `speaker_aliases` ánh xạ nhãn sang tên hiển thị và tài khoản, kèm người và thời điểm xác nhận. Bí danh gắn với *phiên bản transcript* chứ không gắn với cuộc họp, vì tách người nói chạy lại có thể đánh số khác. Khi gửi transcript đi phân tích, Core thay nhãn đã có tên bằng tên đó. Khi gán lại người nói, hệ thống không sửa người thực hiện của công việc đã duyệt mà chỉ gắn lý do `speaker_changed` để rà soát.

2.7.5. Trích dẫn nguồn của công việc

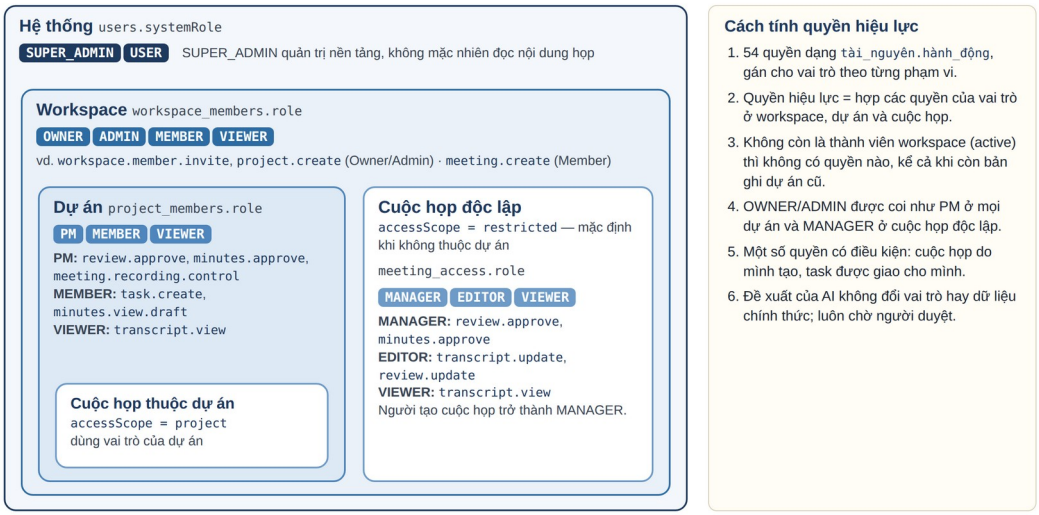
Bảng `action_item_citations` được thiết kế để lưu đoạn nguồn, *phiên bản transcript đã dùng lúc duyệt*, mốc thời gian và câu trích, để trích dẫn vẫn đúng khi

transcript có phiên bản mới. Hiện tại chuỗi truy vết đi qua đề xuất: công việc có `reviewItemId`, đề xuất thuộc lô duyệt có `sourceTranscriptVersionId`, và nội dung đề xuất chứa câu trích cùng `transcriptSequence`, đủ xác định duy nhất đoạn nguồn. Việc tự sinh bản ghi trích dẫn khi duyệt là phần đang hoàn thiện.

2.8. Thiết kế phân quyền

2.8.1. Vai trò theo phạm vi

MeetingMind dùng phân quyền theo vai trò (RBAC) [27] kết hợp phạm vi tài nguyên, bổ sung điều kiện theo thuộc tính bản ghi theo tinh thần ABAC [13]. Hình 2.6 minh hoạ bốn tầng: hệ thống (`SUPER_ADMIN`, `USER`), workspace (`OWNER`, `ADMIN`, `MEMBER`, `VIEWER`), dự án (`PM`, `MEMBER`, `VIEWER`) và cuộc họp độc lập (`MANAGER`, `EDITOR`, `VIEWER`). Cuộc họp thuộc dự án kế thừa quyền của dự án; cuộc họp độc lập mặc định ở phạm vi `restricted` và người tạo được gán `MANAGER`.



Hình 2.6. *Phân quyền theo vai trò kết hợp phạm vi tài nguyên*

Quyền được định nghĩa trong mã dạng `<tài nguyên>.<hành động>`, gồm 54 quyền; mỗi vai trò ánh xạ tới một tập quyền. Theo chính sách giai đoạn đầu, Owner/Admin workspace được coi như PM ở mọi dự án và MANAGER ở mọi cuộc họp độc lập. Ngược lại, `SUPER_ADMIN` không được đi vòng qua bộ kiểm tra quyền, vì quản trị nền tảng không đồng nghĩa với đọc nội dung khách hàng. Một số quyền không bao giờ được cấp qua vai trò, kể cả Owner: đọc hội thoại hỏi đáp hay lịch cá nhân của người khác, đọc khoá bí mật của kết nối tích hợp. Bảng 2.11 nêu các quyền tiêu biểu.

Bảng 2.11. *Một số quyền tiêu biểu và vai trò được phép*

Quyền	Vai trò được phép
`workspace.member.invite`, `project.create`, `integration.manage`	OWNER, ADMIN
`meeting.create`	MEMBER workspace trở lên

Quyền	Vai trò được phép
`meeting.recording.control`	PM, MANAGER; người tạo cuộc họp
`transcript.update`, `review.update`	PM, MANAGER, EDITOR; người tạo cuộc họp
`review.approve`, `minutes.approve`, `task.assign`	PM, MANAGER
`task.status.update`	PM, MANAGER; người được giao
`sync.request`	PM
`chat.query`	Mọi vai trò workspace

Các ô "người tạo cuộc họp" và "người được giao" là quyền có điều kiện, chỉ kiểm tra được khi đã biết bản ghi. Điều kiện "cuộc họp mình tạo" chỉ còn hiệu lực khi người đó vẫn xem được cuộc họp. Danh sách người tham dự chỉ ghi nhận ai có mặt, không cấp quyền truy cập.

2.8.2. Trình tự kiểm tra quyền

Việc kiểm tra tập trung trong `PermissionService` của Core, kích hoạt bởi `PermissionGuard` theo quyền khai báo trên từng route và luôn chạy sau bộ xác thực JWT. Trình tự gồm:

1. Xác thực token và nạp tài khoản.
2. Xác định phạm vi (workspace, dự án, cuộc họp) từ header, thân hoặc tham số request; request tải tệp phải truyền qua header vì thân chưa được đọc lúc bộ bảo vệ chạy.
3. Kiểm tra tư cách thành viên workspace ở trạng thái `active`; không có thì không có quyền nào, kể cả khi bản ghi thành viên dự án cũ còn.
4. Kiểm tra cuộc họp thuộc đúng workspace đang xét.
5. Tra vai trò dự án và vai trò cuộc họp độc lập, hợp các tập quyền, so với quyền yêu cầu.
6. Trong tầng dịch vụ, kiểm tra điều kiện theo bản ghi: người tạo, người được giao, chủ ghi chú.

So với thiết kế ban đầu, bước kiểm tra tài khoản hoặc workspace bị khoá chưa thành một bước riêng. Các danh sách "của tôi" cũng được lọc theo cùng tập quyền.

2.9. Thiết kế bộ phân tích AI

2.9.1. Quy tắc chống bịa trong prompt

Bộ phân tích dùng mô hình ngôn ngữ lớn [5] qua FPT AI Factory [10] với giao diện tương thích OpenAI; mô hình chính là DeepSeek-V4.1-Flash [8], dự phòng là

GLM-5.2 [31]. Mô hình chính lỗi thì chuyển sang dự phòng; cả hai lỗi thì phát `analysis.failed`. Transcript đưa vào dạng các dòng đánh số `[số thứ tự] Người nói: lời nói`, để mô hình chỉ ra bằng chứng bằng một con số kiểm tra được. Lời nhắc hệ thống đặt mô hình vào vai thư ký và nêu các quy tắc chống bịa [14]:

- Quyết định chỉ gồm điều cả nhóm đã chốt; đề xuất chưa ai đồng ý ("hay là…") không phải quyết định.
- Người thực hiện ghi đúng tên trong transcript; không rõ thì để trống.
- Hạn chót ghi nguyên văn ("thứ Sáu", "18/11"); không có thì để trống, không tự đoán ngày.
- Có đính chính (đổi hạn, đổi người, huỷ việc) thì lấy thông tin sau cùng; bỏ qua chuyện ngoài lề.
- Mỗi mục kèm câu trích nguyên văn trong đúng dòng được nêu; không có gì thì trả mảng rỗng.

2.9.2. Cấu trúc đầu ra

Mô hình chỉ trả một đối tượng JSON như Bảng 2.12. Chúng tôi không bật chế độ ép JSON của API, vì khi bật, DeepSeek-V4.1-Flash trả nội dung rỗng; bộ phân tích tự bóc đối tượng JSON từ câu trả lời và tự kiểm tra cấu trúc.

Bảng 2.12. *Cấu trúc JSON đầu ra của bộ phân tích*

Trường	Ý nghĩa
`summary`	Tóm tắt 3–5 câu
`decisions[]`	title, description, quote, transcriptSequence
`openIssues[]`	Vấn đề chưa có kết luận
`actionItems[].assigneeHint`	Tên người thực hiện như trong transcript, hoặc rỗng
`actionItems[].deadlineRawText`	Cụm chỉ thời hạn nguyên văn, hoặc rỗng
`actionItems[].priority`, `confidence`	high/medium/low; số trong [0, 1]
`actionItems[].quote`, `transcriptSequence`	Câu trích làm bằng chứng và số thứ tự dòng

Kết quả sau đó được chuẩn hoá: ưu tiên không hợp lệ đưa về `medium`; độ tin cậy ép vào [0, 1]; các chuỗi "null", "N/A" đưa về rỗng thật; mục thiếu tiêu đề bị loại. Quan trọng nhất là bước *đối chiếu bằng chứng*: mô hình đôi khi trích đúng câu nhưng ghi lệch số dòng, nên bộ phân tích so câu trích với dòng được nêu, không khớp thì dò lại toàn transcript, vẫn không thấy thì chỉ giữ số dòng nếu số đó tồn tại. Transcript rỗng thì không gọi mô hình.

2.9.3. Con người trong vòng lặp

Đầu ra không bao giờ thành dữ liệu chính thức ngay: Core chuyển nó thành biên bản nháp và các đề xuất giữ bản gốc; người duyệt xem đề xuất cạnh câu nguồn, chọn người thực hiện từ danh sách thành viên (`assigneeHint` chỉ để tham khảo), chọn ngày hạn rồi duyệt hoặc từ chối. Mô hình được coi như người soạn nháp nhanh nhưng có thể sai, vì một công việc giao nhầm người hay nhầm hạn gây hậu quả lớn hơn nhiều so với vài giây kiểm tra.

2.10. Thiết kế giao diện

Giao diện web xây dựng bằng Next.js với 29 trang, nhóm như Bảng 2.13. Giao diện có chế độ nối backend thật và chế độ dữ liệu mẫu để trình diễn các chức năng chưa có backend; cột cuối của bảng ghi rõ mức hoàn thiện.

Bảng 2.13. *Các màn hình chính của giao diện web*

Nhóm	Trang	Mức hoàn thiện
Xác thực	Đăng nhập, Đăng ký	Nối API
Tổng quan	Dashboard	Phần lớn nối API
Cuộc họp	Tất cả, Sắp tới, Chờ duyệt, Đang xử lý, Hoàn tất, Ghi âm cá nhân, Chi tiết cuộc họp	Nối API
	Ghi âm trực tiếp	Giao diện mô phỏng
Dự án	Danh sách, Chi tiết	Một phần
Công việc	Task cần duyệt, Task board, Task tồn, Cảnh báo	Nối API
Tri thức, Khách mời	Tri thức (hỏi đáp), Cuộc họp được mời	Giao diện mẫu
Cài đặt	Tài khoản, Thành viên và quyền	Nối API
	Mẫu giọng, Tích hợp, Mẫu biên bản, Lưu trữ, Nhật ký	Giao diện mẫu
Điều hướng	Trang gốc, /work, /settings	Chuyển hướng

Thanh điều hướng ẩn mục theo quyền chỉ để tiện dụng; kiểm soát thật vẫn ở backend. Luồng điển hình của PM: tạo cuộc họp qua hộp thoại; tải tệp thì phiên âm khởi động ngay, họp trực tuyến thì bấm "Mời bot" và theo dõi trạng thái bot; xử lý xong thì mở tab transcript, nơi đoạn nói chồng có nhãn "nên nghe lại", để gán tên người nói; tiếp theo duyệt biên bản và đề xuất; cuối cùng theo dõi công việc trên Task board Kanban bốn cột, mỗi thẻ mở được câu nguồn và bình luận. Thành viên chỉ thấy "Việc của tôi" để cập nhật tiến độ. Với tiện ích Chrome, người dùng bấm một nút trong phòng Google Meet; tiện ích tự tìm hoặc tạo cuộc họp theo đường dẫn phòng, gửi phụ đề theo lô 5 giây và chốt transcript khi kết thúc.

2.11. Thiết kế module đồng bộ công cụ ngoài và cảnh báo

2.11.1. Hiện trạng và dữ liệu

Module đồng bộ công việc sang Jira [1] hoặc Trello [2] và gửi cảnh báo qua Telegram [32] thuộc integration-service. Đến thời điểm nghiệm thu, phần Trello và Telegram đã được hiện thực ở mức MVP (trình bày ở chương Xây dựng hệ thống): integration-service nhận `task.sync.requested`, tạo thẻ Trello, gửi tin Telegram rồi phát `task.sync.completed`; bốn bảng của schema `integration` ở Bảng 2.14 đã được tạo. Phần còn ở mức thiết kế gồm đồng bộ Jira, màn hình kết nối theo từng workspace (trang "Tích hợp" vẫn là giao diện mẫu) và đồng bộ hai chiều trạng thái. Mọi trường trỏ về Core đều là tham chiếu logic, không có khoá ngoại xuyên schema.

Bảng 2.14. *Các bảng của schema integration*

Bảng	Cột quan trọng	Vai trò
`integrations`	provider, status, config, lastError	Một kết nối với một nhà cung cấp; `config` chỉ giữ thông tin không bí mật (danh sách Trello đã dò, cuộc trò chuyện Telegram)
`external_task_links`	integrationId, actionItemId, externalId, externalUrl, syncStatus	Liên kết công việc với thẻ ngoài; duy nhất (integrationId, actionItemId)
`sync_jobs`	type, status, idempotencyKey, payload, result, retryCount, error	Một lần đẩy công việc ra một nhà cung cấp; duy nhất idempotencyKey
`processed_messages`	messageId, consumer	Chống xử lý lặp thông điệp RabbitMQ; duy nhất (messageId, consumer)

Luồng thiết kế: khi công việc được duyệt, Core ghi `task.sync.requested` vào outbox; integration-service tạo job, gọi REST API của nền tảng đích, ghi liên kết ngoài rồi phát `task.sync.completed`. Khi hiện thực, nhóm điều chỉnh một điểm so với thiết kế ban đầu: thay vì để integration-service gọi ngược API nội bộ của Core để đọc công việc, thông điệp mang sẵn ảnh chụp công việc, tránh phụ thuộc đồng bộ giữa hai dịch vụ. Theo thiết kế đầy đủ, kết nối dùng OAuth 2.0 [12] nơi nhà cung cấp hỗ trợ, token được mã hoá khi lưu và không bao giờ được trả về qua API hay giao diện; bản MVP hiện tại lấy khoá API từ biến môi trường và không ghi khoá vào cơ sở dữ liệu.

2.11.2. Nguyên tắc chống tạo trùng và cảnh báo

Hai nguyên tắc chi phối đồng bộ. Thứ nhất, *chỉ đồng bộ công việc đã duyệt*, vì một thẻ hay issue sai trên công cụ ngoài khó thu hồi hơn một đề xuất bị từ chối. Thứ hai, *thử lại không tạo bản trùng*: chống lặp thông điệp nội bộ là chưa đủ, vì lời gọi tạo issue có thể đã thành công nhưng phản hồi bị mất, gửi lại sẽ tạo issue thứ hai. Với API không hỗ trợ khoá idempotency, thiết kế yêu cầu tra cứu đối soát trước khi thử lại;

ràng buộc duy nhất trên (kết nối, công việc) là chốt chặn cuối. Bản hiện thực cho Trello áp dụng đúng nguyên tắc này bằng một dòng đánh dấu trong mô tả thẻ; phạm vi đã hiện thực là đồng bộ một chiều (chỉ tạo thẻ).

Cảnh báo dựa trên quy tắc minh bạch thay vì phán đoán của AI. Bảng `risk_flags` đã có trong schema `core`, lưu quy tắc sinh cảnh báo (`ruleKey`), dữ liệu căn cứ (`evidence`) và khoá chống tạo trùng qua các lần quét. Hiện giao diện tính năm loại cảnh báo trên dữ liệu công việc (quá hạn, bị chặn, thiếu người phụ trách, phụ thuộc chưa xong, từ năm hạn dồn vào một ngày cho một người), mỗi cảnh báo kèm quy tắc tương ứng; chỉ kết luận "quá tải" khi có dữ liệu công sức và năng lực. Kênh Telegram hiện mới dùng để báo công việc mới được duyệt; bộ quét phía máy chủ và các cảnh báo, nhắc việc theo quy tắc (quá hạn, sắp đến hạn) qua Telegram chưa được hiện thực.

2.12. Kết luận chương

Chương này đã phân tích yêu cầu và trình bày thiết kế của MeetingMind. Từ khảo sát, chúng tôi rút ra quy tắc cốt lõi: phân biệt đề xuất với cam kết, không bịa người hay hạn, mọi công việc có nguồn, chỉ công việc đã duyệt mới được giao. Về kiến trúc, Core là bên điều phối và là nơi duy nhất quyết định quyền; bot và AI chỉ thực thi và báo kết quả qua RabbitMQ, với outbox, chống xử lý lặp và khoá idempotency. Cơ sở dữ liệu lưu transcript và biên bản theo phiên bản để công việc luôn truy được về đúng câu nguồn; bộ phân tích AI bị ràng buộc bằng quy tắc chống bịa và luôn qua bước duyệt của con người. Phần lõi từ thu nhận tới bảng công việc đã hiện thực, cùng đồng bộ một chiều sang Trello và thông báo công việc mới qua Telegram; đồng bộ Jira, cảnh báo theo quy tắc phía máy chủ, hỏi đáp có trích nguồn và mã hoá khi lưu còn ở mức thiết kế. Chương tiếp theo trình bày việc hiện thực các thiết kế này.

CHƯƠNG 3. XÂY DỰNG HỆ THỐNG

Chương này trình bày việc hiện thực hoá thiết kế ở chương Phân tích và thiết kế thành hệ thống MeetingMind chạy được: môi trường phát triển, dịch vụ lõi, ba đường thu nhận nội dung cuộc họp, ba mô-đun trí tuệ nhân tạo, luồng duyệt công việc, mô-đun đồng bộ Trello và cảnh báo Telegram, giao diện web, triển khai và kiểm thử. Mỗi phần nêu cả khó khăn khi chạy thật và cách xử lý. Những hạng mục mới ở mức thiết kế (đồng bộ Jira, hỏi đáp trên tri thức cuộc họp, phát hiện rủi ro tự động, mã hoá dữ liệu khi lưu) không được mô tả như đã chạy.

Nhóm nghiên cứu xây dựng theo kiểu "lát cắt dọc": trước hết làm thông toàn bộ đường đi của dữ liệu bằng các thành phần giả lập (stub), sau đó thay dần bằng thành phần thật. Lịch sử commit phản ánh đúng trình tự này: giữa tháng 9/2026 tách schema theo dịch vụ, áp ma trận phân quyền, làm thông luồng outbox → RabbitMQ → AI giả lập → biên bản nháp, thêm đường tải tệp và đường extension; cuối tháng 10/2026 sửa extension sau khi thử trên Google Meet thật, vá lỗ hổng bảo mật, nối giao diện; đầu tháng 11/2026 gắn Whisper, tách người nói, bot và LLM thật, sau đó hiện thực đồng bộ công việc đã duyệt sang Trello và Telegram.

3.1. Môi trường và công cụ phát triển

Hệ thống gồm năm dịch vụ backend, một ứng dụng web và một tiện ích mở rộng trình duyệt. Dịch vụ lõi (core-service), cổng API (api-gateway) và dịch vụ tích hợp (integration-service) viết bằng TypeScript trên NestJS [20]; api-gateway hiện mới là khung dựng từ mẫu NestJS, còn integration-service đã có mô-đun đồng bộ công việc sang Trello và Telegram. Phần xử lý AI (ai-worker) và bot ghi âm (bot-worker) viết bằng Python vì hệ sinh thái thư viện phù hợp. Bảng Bảng 3.1 liệt kê công cụ chính với phiên bản lấy từ `package.json`, `requirements.txt` và `Dockerfile`.

Bảng 3.1. *Ngôn ngữ, framework và thư viện chính dùng để xây dựng hệ thống*

Thành phần	Công nghệ và phiên bản	Vai trò
core-service	Node.js 22, TypeScript 5.8.3, NestJS 11.1.3	Khung ứng dụng, module, guard
core-service	Sequelize 6.37.7, sequelize-typescript 2.1.6	Truy cập PostgreSQL
core-service	@golevelup/nestjs-rabbitmq 6.0.1, minio 8.0.5	Sự kiện RabbitMQ; ghi object, ký URL
core-service	@nestjs/jwt 11.0.0, bcryptjs 3.0.2	Xác thực JWT, băm mật khẩu
integration-service	NestJS 11.1.3, @golevelup/nestjs-rabbitmq 6.0.1, Jest 30.0.0	Nhận sự kiện, gọi Trello REST API và Telegram Bot API, kiểm thử

Thành phần	Công nghệ và phiên bản	Vai trò
ai-worker	Python 3.12.3, aio-pika 9.5.5, httpx 0.26.0	Nhận việc từ RabbitMQ, gọi API LLM
ai-worker	faster-whisper 1.2.1, PyAV 18.1.0	Nhận dạng tiếng nói, giải mã âm thanh
ai-worker	sherpa-onnx 2.0.1, NumPy 2.2.5	Vector đặc trưng giọng, gom cụm
bot-worker	Playwright 1.64.1 (Chromium), Xvfb, PulseAudio	Vào Google Meet trên màn hình và loa ảo
Extension	Chrome Manifest V3, node:test	Đọc phụ đề Meet, kiểm thử
Frontend	Next.js 16.3.8, React 19.2.8, Tailwind CSS 4.3, shadcn/Radix UI	Ứng dụng web
Hạ tầng	PostgreSQL 15, RabbitMQ 3.11, Redis 7, MinIO	Lưu trữ, hàng đợi, bộ đệm, lưu trữ đối tượng
Hạ tầng	Docker Compose, Prometheus, Grafana	Triển khai, khung giám sát

Backend, extension, bot và AI nằm chung một kho Git; frontend là kho riêng. Quy mô hiện tại: core-service khoảng 29 nghìn dòng TypeScript trong 64 module, frontend khoảng 14,8 nghìn dòng với 29 trang, bot-worker khoảng 780 dòng, extension khoảng 700 dòng. Phần lớn module CRUD được sinh bằng công cụ `tools/generate-module.js` từ tệp đặc tả bảng, nên 43 bảng của thiết kế được hiện thực đồng nhất.

3.2. Xây dựng dịch vụ lõi core-service

3.2.1. Tổ chức mã nguồn và phân quyền

Core-service là nơi duy nhất sở hữu dữ liệu nghiệp vụ và quyết định quyền. Mỗi module theo khuôn cố định: thực thể, mô hình Sequelize, kho dữ liệu (repository), dịch vụ, bộ điều khiển và DTO kiểm tra đầu vào. `BaseService` và `SqlRepository` cung cấp sẵn thao tác CRUD, phân trang, xoá mềm; bộ điều khiển CRUD sinh từ `BaseControllerFactory`, mỗi module chỉ khai tuyến nào bật và cần quyền gì. Khung mẫu ban đầu có cả MongoDB; nhóm bỏ MongoDB, đưa toàn bộ dữ liệu về schema `core`. Thao tác nghiệp vụ nhiều bước (duyệt biên bản, duyệt đề xuất, mời bot) nằm ở các module riêng `workflow`, `meeting-processing`, `meeting-bot`, `me`.

Mô hình phân quyền vai trò kết hợp phạm vi [13, 27] (xem Hình 2.6) được hiện thực trong `PermissionService`, với danh mục 54 quyền dạng `<tài nguyên>.<hành động>` và ba bảng ánh xạ vai trò → quyền cho workspace, dự án và cuộc họp độc lập. Hàm `resolve` tính tập quyền hiệu lực theo thứ tự: chỉ thành viên `active` mới có quyền; cộng quyền vai trò workspace; nếu yêu cầu gắn với cuộc họp thì xác nhận cuộc họp thuộc đúng workspace rồi suy ra dự án; cộng quyền vai trò dự án; với cuộc họp `restricted` thì cộng quyền theo bảng `meeting_access`. OWNER và ADMIN được

coi như PM ở mọi dự án của workspace; SUPER_ADMIN không được đi vòng qua, vì đó là quyền quản trị nền tảng chứ không phải quyền đọc nội dung khách hàng. Hai điều kiện theo bản ghi được xét riêng: thành viên được sửa cuộc họp *do chính mình tạo* khi vẫn còn quyền xem nó (`canOnMeeting`), người được giao task được đổi trạng thái task đó (`canOnTask`).

`PermissionGuard` đọc quyền khai bằng `@RequirePermission` và xác định phạm vi từ header `x-workspace-id`, `x-project-id`, `x-meeting-id`, sau đó mới tới thân yêu cầu. Header là bắt buộc với yêu cầu multipart, vì multer phân tích thân yêu cầu *sau* khi guard chạy. Ban đầu guard xác thực và guard phân quyền gắn bằng hai decorator riêng; do decorator lớp áp dụng từ dưới lên, guard phân quyền đã chạy trước. Nhóm gộp cả hai vào decorator `Authorization()` để thứ tự luôn là xác thực JWT rồi mới phân quyền.

3.2.2. Outbox, publisher và chống xử lý lặp

Giao tiếp giữa core-service với ai-worker, bot-worker và integration-service đi qua RabbitMQ [24] theo mẫu transactional outbox [26] (xem Hình 2.4). Mọi sự kiện có chung phong bì (`eventId`, `eventType`, `correlationId`, `payload`…); hợp đồng viết một lần trong `contracts/events.json`, có bản dịch cho TypeScript và Python. ai-worker nghe hàng đợi `ai.jobs`; bot-worker nghe `bot.jobs` cho lệnh vào phòng và `bot.control` cho lệnh dừng — tách riêng vì phiên ghi âm kéo dài hàng giờ, dùng chung thì lệnh dừng phải xếp sau chính phiên cần dừng; integration-service nghe `integration.jobs.task-sync`.

`OutboxService.emit` không gửi thẳng mà ghi một dòng `outbox_events` trạng thái `pending`. `OutboxPublisherService` quét mỗi 2 giây, lấy tối đa 50 sự kiện cũ nhất, phát với `messageId` bằng `eventId` ở chế độ lưu bền rồi đánh dấu `published`; lỗi được đếm, quá 5 lần thì chuyển `failed` để người vận hành kiểm tra. Nhờ vậy khi RabbitMQ ngừng, lệnh ghi nghiệp vụ vẫn thành công và sự kiện chờ trong bảng [16]. Cần nói rõ hai hạn chế: `emit` nhận tham số `transaction` nhưng chưa nơi gọi nào truyền vào, nên job và sự kiện vẫn được ghi tuần tự, chưa cùng một giao dịch; hệ thống cũng chưa có hàng đợi thư chết (dead-letter queue) hay cơ chế tự chạy lại job lỗi. Đây là các việc cần hoàn thiện.

RabbitMQ chỉ bảo đảm giao tin ít nhất một lần (at-least-once), nên mọi consumer của Core phải chịu được tin lặp. `MessageDedupService.claim` ghi cặp `(messageId, consumer)` vào bảng `processed_messages` có chỉ mục duy nhất; lần thứ hai vi phạm ràng buộc và handler bỏ qua tin:

```
@RabbitSubscribe({
    exchange: MEETINGMIND_EXCHANGE,
    routingKey: EventType.TRANSCRIPTION_COMPLETED,
    queue: `${CORE_EVENTS_QUEUE}.transcription-completed`,
    queueOptions: { durable: true },
})
async onTranscriptionCompleted(envelope:
EventEnvelope<TranscriptionCompletedPayload>) {
    if (!(await this.dedup.claim(envelope.eventId, CONSUMER))) {
        return; // đã xử lý ở lần giao trước
    }
    await
this.meetingProcessingService.onTranscriptionCompleted(envelope.payload);
}
```

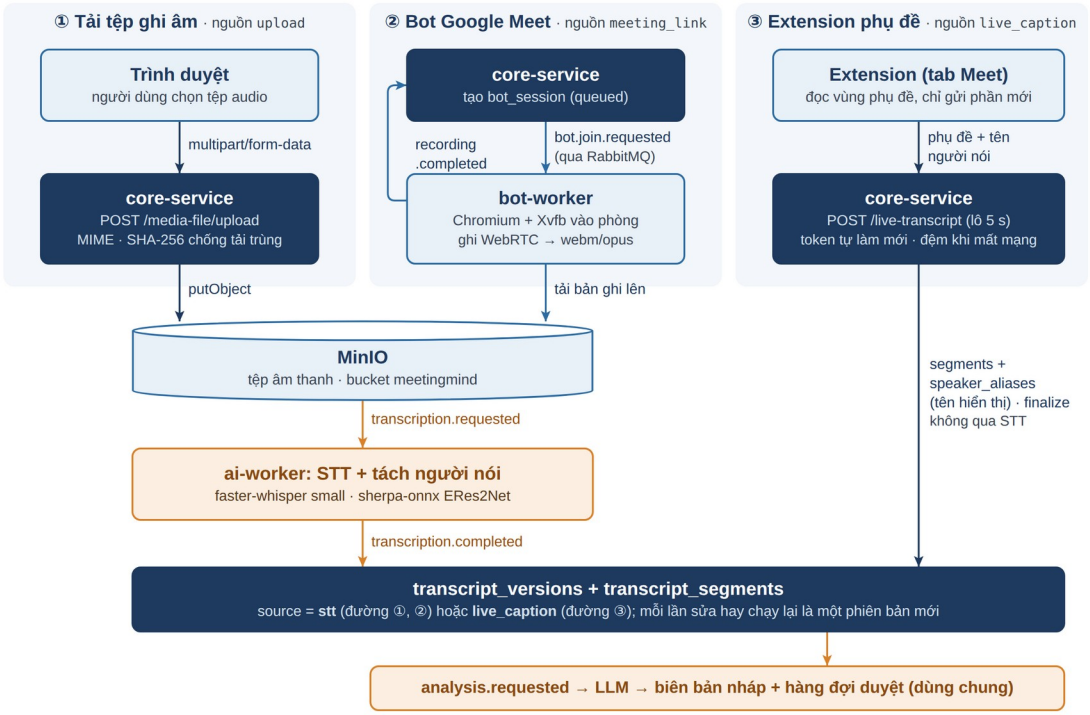
Ở chiều phát lệnh, mỗi `processing_jobs` mang khoá lũy đẳng (`idempotencyKey`), ví dụ `<meetingId>:transcribe:<objectKey>`; gọi lại với cùng đầu vào trả về job đã có. Nguyên tắc này áp dụng nhất quán: mời bot khi đang có phiên hoạt động thì trả về phiên đó, duyệt lại một đề xuất đã duyệt không sinh thêm task, sự kiện trạng thái bot đến trễ không làm lùi phiên đã kết thúc.

3.2.3. Điều phối luồng xử lý cuộc họp

`MeetingProcessingService` hiện thực luồng ở Hình 2.3 theo nguyên tắc Core ra lệnh và ghi kết quả, AI chỉ tính toán rồi trả về. Khi nhận transcript, dịch vụ tạo `transcript_versions` mới (không ghi đè bản cũ), ghi từng đoạn kèm nhãn người nói, mốc thời gian, độ tin cậy, ngôn ngữ và cờ nói chồng, rồi cập nhật ngôn ngữ cuộc họp theo ngôn ngữ nói nhiều nhất. Lệnh phân tích gửi kèm toàn bộ transcript, vì ai-worker không được đọc schema `core`. Khi nhận kết quả phân tích, dịch vụ tạo biên bản nháp, một lô duyệt và các đề xuất chờ duyệt.

3.3. Ba đường thu nhận nội dung cuộc họp

Hệ thống hỗ trợ ba cách đưa nội dung vào, ứng với ba bối cảnh: tải lên tệp ghi âm có sẵn (họp trực tiếp), mời bot ghi âm cuộc họp Google Meet (cách của Otter [22], Fireflies [9]), và extension đọc phụ đề do Meet sinh ra (cách của Tactiq [30]). Như Hình 3.1 minh hoạ, hai đường đầu đi qua nhận dạng tiếng nói, đường thứ ba thì không; cả ba hội tụ tại `transcript_versions`, từ đó phần phân tích và duyệt dùng chung.



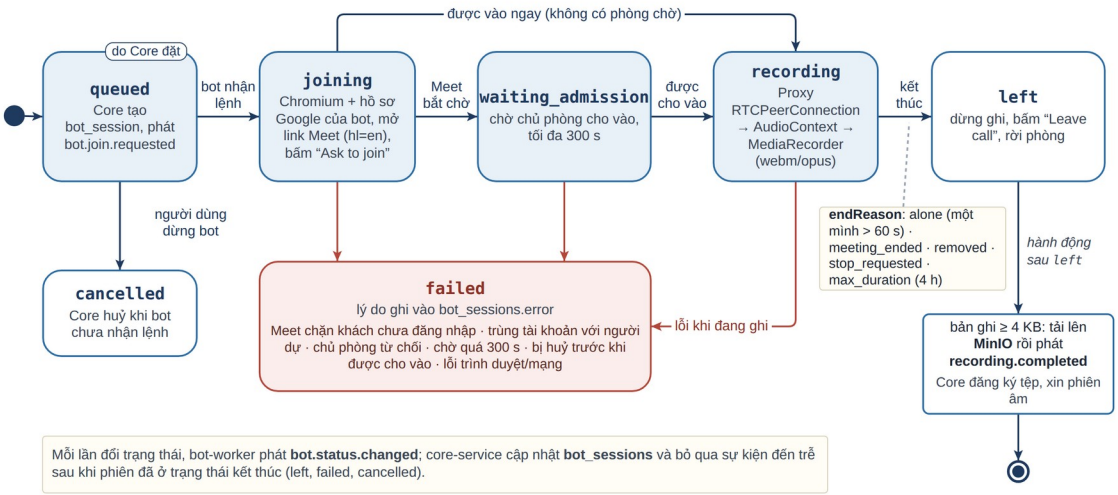
Hình 3.1. *Ba đường thu nhận nội dung cuộc họp hội tụ tại transcript_versions*

3.3.1. Tải tệp ghi âm lên MinIO

Bảng `files` chỉ lưu `bucket` và `objectKey`; khung mẫu cũ lưu tệp base64 trong cơ sở dữ liệu, không dùng được cho bản ghi một giờ. `MediaFileUploadService` kiểm tra kiểu MIME, tính băm SHA-256 và nếu workspace đã có tệp trùng băm thì dùng lại bản ghi cũ thay vì tạo object trùng. Tệp mới được đặt khoá `<workspaceId>/<meetingId>/<uuid>.<đuôi>`, chuyển trạng thái `uploading` → `ready` (hoặc `failed`). Để nghe lại, Core cấp URL ký có hạn 300 giây sau khi kiểm quyền `audio.listen`. Khi chạy thật, nhóm phát hiện chữ ký URL gắn chặt với tên máy: ký bằng tên nội bộ `minio` thì trình duyệt nhận lỗi 403; giải pháp là thêm `MINIO_PUBLIC_ENDPOINT` và một client riêng chỉ để ký. Với bot, Core chỉ đăng ký siêu dữ liệu của bản ghi bot tự đẩy lên MinIO, không làm trung gian cho tệp lớn.

3.3.2. Bot tham gia và ghi âm Google Meet

Ở core-service, `MeetingBotService` kiểm tra quyền `meeting.recording.control` và đường dẫn đúng dạng `https://meet.google.com/...`, tạo `bot_sessions` rồi phát `bot.join.requested`; khi nhận `recording.completed`, Core đăng ký tệp và gọi đúng hàm bắt đầu phiên âm của đường tải tệp. Bot-worker mở hai kênh RabbitMQ: kênh việc nhận một phiên mỗi lúc (một hồ sơ Chromium không mở được hai lần) và kênh điều khiển để nhận lệnh dừng. Lệnh vào phòng được xác nhận (ack) ngay, vì phiên dài hơn `consumer_timeout` 30 phút sẽ bị broker đóng kênh.



Hình 3.2. *Trạng thái một phiên bot và các lý do kết thúc*

Vòng đời một phiên ở Hình 3.2: `queued` khi Core tạo, `joining` khi mở trang, `waiting_admission` khi chờ chủ phòng duyệt, `recording` khi đã vào và bắt đầu ghi. Phiên kết thúc ở `left` khi chỉ còn một mình quá 60 giây, cuộc họp kết thúc, bị mời ra, có lệnh dừng hoặc vượt 4 giờ; thất bại ở `failed` khi Meet từ chối khách, chủ phòng từ chối hay chờ duyệt quá 300 giây; bị dừng khi còn `queued` thì thành `cancelled`. Tải lên không phải một trạng thái riêng: sau `left`, bản ghi dưới 4 KB (không ai nói) bị bỏ, còn lại được bot tải lên MinIO rồi phát `recording.completed`.

Bot dùng Playwright [19] điều khiển Chromium có giao diện trên màn hình ảo Xvfb, vì Meet hay từ chối trình duyệt không giao diện. Bot không bật micro, camera; âm thanh được ghi ngay trong trang bằng script chèn trước mọi script của Meet. Script bọc hàm tạo `RTCPeerConnection` [34] để bắt mọi track âm thanh từ xa, trộn vào một `AudioContext` rồi ghi bằng `MediaRecorder` dạng webm/opus 64 kbps, mỗi khúc 10 giây được đẩy về Python theo đúng thứ tự.

```
const Original = window.RTCPeerConnection;
window.RTCPeerConnection = new Proxy(Original, {
    construct(target, args) {
        const pc = Reflect.construct(target, args);
        // Mỗi track âm thanh từ xa được trộn vào AudioContext chung.
        pc.addEventListener("track", (event) => attach(event.track));
        return pc;
    },
});
```

Phần tốn công nhất là các tình huống thực tế của Google Meet:

– **Google chặn khách ẩn danh dùng trình duyệt tự động** ("You can't join this video call"), nên bot bắt buộc dùng tài khoản Google riêng. Người vận hành đăng nhập một lần qua noVNC bằng `login.sh` (Chromium thường, vì trang đăng nhập Google cũng từ chối trình duyệt tự động); hồ sơ kèm cookie lưu ở volume

`bot-profile`. Nếu hồ sơ bot trùng tài khoản với người trong phòng, Meet đề nghị "Switch here" — bấm vào sẽ đẩy người thật ra — nên bot nhận ra nút này và dừng với thông báo lỗi.

- **Giao diện Meet theo ngôn ngữ tài khoản**, không theo locale trình duyệt, nên với tài khoản tiếng Việt nút hiện "Yêu cầu tham gia". Nhóm ép giao diện tiếng Anh bằng tham số `hl=en` trên đường dẫn, giữ nhãn tiếng Việt làm dự phòng.
- **Nhãn nút dài, dễ khớp nhầm.** Nút thật tên "Ask to join without microphone & camera", còn so khớp lỏng "Join" lại trúng "Other ways to join"; nhóm chuyển sang so khớp tiền tố (`^Ask to join`).
- **Phòng chờ kiểu mới** cũng có nút "Leave call", nên bot chỉ coi là đã vào khi dòng "Asking to be let in" biến mất.
- **Thiếu thiết bị âm thanh.** Trong container không có loa, âm thanh WebRTC của người khác không chảy và bot chỉ ghi được khoảng lặng; nhóm khởi động PulseAudio với loa ảo (null sink). Ngoài ra, do một lỗi đã biết của Chromium, mỗi track từ xa phải gắn vào một thẻ `<audio>` tắt tiếng thì mới vào được Web Audio.
- **Hồ sơ trình duyệt của bot phải sạch.** Hồ sơ lưu phiên đăng nhập của bot không được kéo extension, tab hay lịch sử đồng bộ từ thiết bị khác, vì các thành phần lạ chèn vào trang có thể làm thay đổi giao diện Meet mà bot đang dò theo. Chromium của bot được chạy với `--disable-sync` cả khi đăng nhập lẫn khi vào phòng.
- **Hạ tầng:** `xvfb-run` treo khi là tiến trình số 1 nên Xvfb được khởi động trực tiếp; `/dev/shm` mặc định 64 MB làm tab sập nên container được cấp 1 GB.

Bot không vượt rào kiểm soát của Meet: hiện đúng tên, phải được chủ phòng duyệt và dừng ngay khi bị từ chối.

3.3.3. Tiện ích mở rộng đọc phụ đề

Extension Chrome theo Manifest V3 [11] không có bot và không ghi âm, chỉ đọc phụ đề Meet đã hiển thị trên màn hình của chính người dùng, nên có sẵn tên người nói. Người dùng bấm "Bắt đầu" trong popup; Core tìm cuộc họp theo đường dẫn phòng, chưa có thì tạo mới; khi kết thúc, Core chốt phiên bản transcript và gọi lại đúng bước phân tích của hai đường kia. Tên hiển thị trên Meet chỉ được lưu ở mức `suggested`, vì đó không phải bằng chứng danh tính.

`content.js` quét DOM mỗi 400 ms thay vì gắn `MutationObserver` cố định, vì Meet gỡ và dựng lại vùng phụ đề sau mỗi quãng im lặng. Vùng phụ đề được tìm theo thuộc tính trợ năng `[role="region"]` có `aria-label` chứa "Captions" hoặc "Phụ

đề", không dựa vào tên lớp CSS bị làm rối; khối con có từ hai vùng chữ trở lên là một lượt nói, vùng đầu là tên. Thử trên phòng Meet thật ngày 29/10/2026 cho thấy ba lỗi. Thứ nhất là **lặp chồng**: Meet giữ một khối cho cùng người nói và nối câu mới vào sau, trong khi mã cũ gửi cả khối mỗi khi chữ đứng yên, nên transcript thành "A", "A B", "A B C". Mỗi khối giờ nhớ phần đã gửi và chỉ gửi phần mới, xử lý cả khi Meet cắt dòng đầu hay sửa từ đã gửi:

```
function unsentPart(sent, live) {
    if (live.startsWith(sent)) return live.slice(sent.length).trim(); // nói
tiếp
    const tail = sent.slice(-TAIL_LEN);                                // 24
ký tự cuối
    const at = tail.length === TAIL_LEN ? live.lastIndexOf(tail) : -1;
    if (at >= 0) return live.slice(at + TAIL_LEN).trim();              // Meet
cắt dòng đầu
    // Meet sửa từ đã gửi: gửi lại từ đầu từ bị sửa, không gửi nửa từ.
    const p = commonPrefixLength(sent, live);
    const atBoundary = p === 0 || /\s/.test(live[p - 1]) || /\s/.test(live[p]
?? " ");
    const from = atBoundary ? p : live.lastIndexOf(" ", p) + 1;
    return live.slice(from).replace(/^[\s.,!?…]+/, "").trim();
}
```

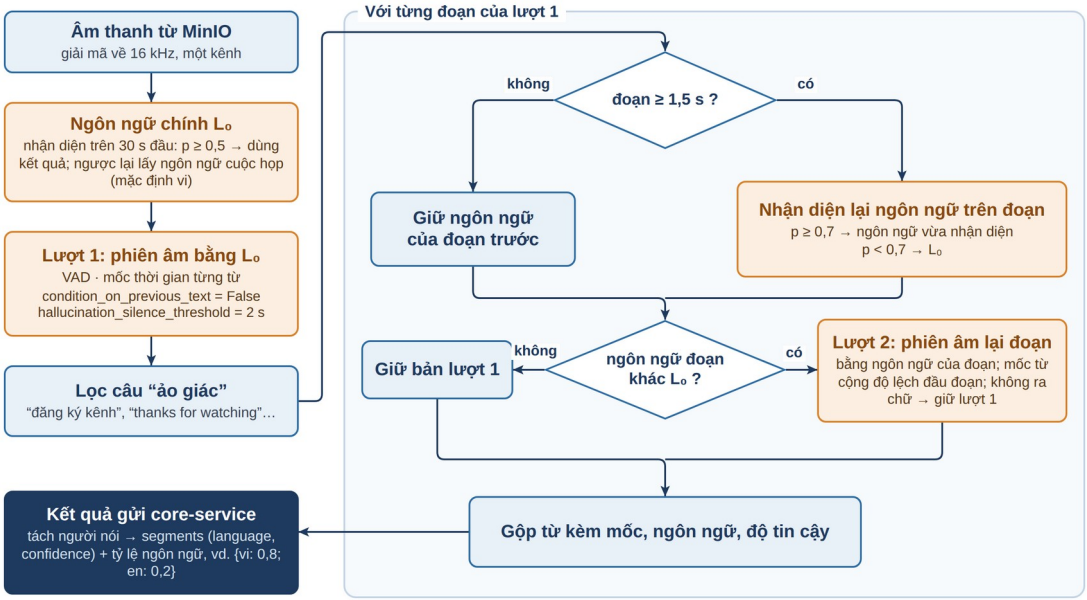
Thứ hai là **bắt nhầm rác lúc im lặng**: khi vùng phụ đề rỗng, mã cũ rơi xuống bộ chọn dự phòng và đọc nhầm menu cài đặt thành phụ đề; các bộ chọn dự phòng đã bị bỏ. Thứ ba là **khai báo trùng biến**: các content script của cùng extension chung một phạm vi toàn cục, nên `unsentPart` khai ở cấp ngoài cùng của `caption-text.js` đụng `const { unsentPart }` trong `content.js`, làm `content.js` lỗi cú pháp và không chạy, trong khi popup chỉ báo "Không gọi được tab Meet". Nhóm bọc `caption-text.js` trong hàm tự gọi (IIFE) và viết kiểm thử nạp các content script theo thứ tự manifest vào chung một ngữ cảnh.

Service worker MV3 bị Chrome tắt sau khoảng 30 giây không có sự kiện, làm mất biến trong bộ nhớ và `setInterval`, nên phụ đề từng nằm mãi trong bộ đệm. Bộ đệm được chuyển vào `chrome.storage.local`, việc gửi theo lô 5 giây được hẹn bằng `setTimeout` mỗi khi có đoạn mới; gửi lỗi thì lô trả về bộ đệm, gặp mã 401 thì làm mới token rồi gửi lại. Hạn chế: người dùng phải bật phụ đề trong Meet và extension phụ thuộc cấu trúc DOM của Meet.

3.4. Mô-đun nhận dạng tiếng nói

Mô-đun nhận dạng tiếng nói (Speech-to-Text, STT) dùng faster-whisper 1.2.1 [29], bản cài đặt Whisper [25] trên CTranslate2 (xem Hình 1.1), mặc định mô hình `small`, chạy CPU lượng tử hoá int8, beam search độ rộng 5; mô hình nạp một lần mỗi tiến trình. Phiên âm chạy trong luồng phụ (`asyncio.to_thread`), vì chặn vòng lặp sự kiện sẽ làm lỡ heartbeat và RabbitMQ cắt kết nối giữa chừng.

Whisper chỉ nhận diện ngôn ngữ một lần trên 30 giây đầu; cuộc họp nói lẫn Việt – Anh thì câu tiếng Anh bị giải mã như tiếng Việt thành chữ vô nghĩa hoặc bị dịch. Nhóm thiết kế phiên âm hai lượt (Hình 3.3). Lượt một chọn ngôn ngữ chính (xác suất dưới 0,5 thì theo ngôn ngữ cuộc họp do Core gửi) và phiên âm có lọc khoảng lặng (VAD). Lượt hai nhận diện lại ngôn ngữ trên từng đoạn từ 1,5 giây; nếu khác ngôn ngữ chính với xác suất từ 0,7 thì phiên âm lại đoạn đó bằng ngôn ngữ của nó và cộng bù mốc thời gian. Đoạn ngắn hơn giữ ngôn ngữ đoạn trước, vì người nói hiếm khi đổi thứ tiếng giữa một câu ngắn.



Hình 3.3. *Phiên âm hai lượt theo ngôn ngữ của từng đoạn*

```
DECODE = {"condition_on_previous_text": False, "word_timestamps": True,
          "hallucination_silence_threshold": 2.0}

def choose_segment_language(base, previous, duration_s, guess, settings):
    if duration_s < settings.min_detect_seconds or guess is None:
        return previous   # đoạn dưới 1,5 s: giữ ngôn ngữ đoạn trước
    language, probability = guess
    if probability >= settings.segment_min_prob:
        return language   # nhận diện lại đủ chắc (>= 0,7)
    return base           # không chắc: về ngôn ngữ chính
```

Tham số giải mã nhằm hạn chế "ảo giác" (sinh câu không có trong âm thanh) [14]: `condition_on_previous_text=False` tránh vòng lặp lặp câu; `hallucination_silence_threshold=2.0` bỏ đoạn bịa giữa khoảng lặng dài, cần `word_timestamps=True` — vốn cũng là đầu vào của bước tách người nói. Một biểu thức chính quy lọc thêm các câu Whisper hay bịa do học từ phụ đề video như "đăng ký kênh", "thanks for watching". Mỗi đoạn mang độ tin cậy và ngôn ngữ riêng; bản ghi kèm tỷ lệ thời lượng theo ngôn ngữ.

Hai lỗi phát sinh khi chạy thật đáng ghi lại. PyAV bản 19 bỏ tham số `metadata_errors` mà faster-whisper 1.2.1 vẫn truyền vào hàm mở tệp, khiến mọi lần

giải mã lỗi; nhóm ghim `av==18.1.0`. Mốc thời gian của faster-whisper là số kiểu NumPy, nên cờ nói chồng tính từ chúng có kiểu `numpy.bool` không chuyển được sang JSON, kết quả không gửi về được và job ở Core kẹt ở trạng thái chờ; nhóm thêm hàm chuyển kiểu khi tuần tự hoá và đưa lệnh gửi vào khối bắt lỗi để mọi lỗi đều thành `transcription.failed`. Độ chính xác được đánh giá ở chương Thực nghiệm và đánh giá (Hình 4.1).

3.5. Mô-đun tách người nói

Với cuộc họp trực tiếp ghi bằng một micro, mô-đun tách người nói (speaker diarization) [23] trả lời câu hỏi "ai nói khi nào". Nhóm thử trước mô hình phân đoạn pyannote [4], nhưng trên dữ liệu thử mô hình coi cả cửa sổ 10 giây là một người và gộp nhầm lời hai người, trong khi vector đặc trưng giọng tách người rất rõ (độ giống cosine khoảng 0,6 giữa hai khúc cùng người, khoảng 0,2 giữa hai người khác nhau). Vì vậy nhóm chuyển sang gom cụm trực tiếp các khúc lời (Hình 1.2).

Lời Whisper được cắt thành khúc khoảng 1,5 giây theo mốc từ, cắt thêm khi gặp khoảng lặng trên 0,4 giây hoặc sang đoạn Whisper mới. Mỗi khúc được tính vector đặc trưng giọng bằng mô hình ERes2Net của bộ 3D-Speaker [7] qua sherpa-onnx [15] trên CPU — ý tưởng biểu diễn người nói bằng vector nhúng cố định độ dài kế thừa từ x-vector [28]. Các vector được gom cụm phân cấp theo liên kết trung bình (average linkage): liên tục ghép hai cụm có độ tương đồng cosine cao nhất, chừng nào độ tương đồng đó còn từ 0,3 trở lên, nên số người được suy ra chứ không khai trước. Thu âm ồn hay sinh "người nói" ảo chỉ vài giây, nên cụm có tổng thời gian nói dưới 2 giây được gộp vào cụm lớn giống nhất; đánh đổi là người chỉ nói một câu "ok" có thể bị gộp vào người khác.

Để đánh dấu nói chồng, mô-đun xét độ giống của mỗi khúc với tâm cụm *đứng thứ hai*: khi hai người nói chồng, giọng át vẫn trội nhưng độ giống với người kia nhích lên. Mức "bình thường" khác nhau theo bản ghi, nên ngưỡng được tính thích nghi bằng trung vị cộng hai lần độ lệch tuyệt đối trung vị (MAD), với sàn 0,3. Từ các khúc đã gán người, mô-đun ghép ngược với lời Whisper: mỗi từ thuộc lượt nói phủ nó nhiều nhất, một đoạn Whisper trùm hai người được cắt đôi đúng chỗ đổi người, nhãn "Người nói 1, 2…" đặt theo thứ tự lên tiếng, và đoạn có phần chồng lấn từ 0,3 giây (hoặc từ 30% thời lượng) được gắn cờ `isOverlapping`. Sau khi thực nghiệm cho thấy cơ chế này báo nhầm nhiều (chương Thực nghiệm và đánh giá), cờ được tắt mặc định và chỉ bật khi đặt biến `AI_DIARIZATION_OVERLAP=1` để nghiên cứu tiếp.

Tách người nói là phần bổ sung, không được làm hỏng phiên âm: toàn bộ bước này nằm trong khối bắt lỗi, lỗi chỉ được ghi log và mọi đoạn nhận chung nhãn "Người nói 1"; biến `AI_DIARIZATION=0` tắt hẳn. Giới hạn bản chất là với một micro, lời người

nói chồng thường bị Whisper bỏ mất, nên cờ nói chồng chỉ là gợi ý "nên nghe lại"; nhãn "Người nói N" cũng không phải danh tính, người dùng gán tên ở màn hình transcript.

3.6. Mô-đun phân tích biên bản bằng LLM

Mô-đun phân tích nhận transcript và trả về tóm tắt, quyết định, vấn đề còn mở và công việc, mỗi mục kèm câu trích và số dòng làm bằng chứng (Hình 1.3). Worker gọi dịch vụ Serverless Inference của FPT AI Factory [10] qua API kiểu OpenAI tại `https://token-api.fpt.ai/v1`, nhiệt độ 0,2, tối đa 32.000 token đầu ra vì mô hình có bước suy luận tiêu tốn nhiều token. Nhóm không bật `response_format` JSON vì khi bật, mô hình chính trả nội dung rỗng; prompt yêu cầu JSON và worker tự bóc tách, kể cả khi mô hình bọc kết quả trong khối mã hay nói thêm.

Transcript được đưa vào dạng các dòng `[số thứ tự] Người nói: lời nói`. Lời nhắc hệ thống, dựa trên khả năng làm theo chỉ dẫn của LLM [5], quy định viết biên bản bằng ngôn ngữ chính của cuộc họp và nêu các quy tắc rút từ tình huống thật: đề xuất chưa ai đồng ý ("hay là…") không phải quyết định; người thực hiện ghi đúng tên trong transcript; thời hạn giữ nguyên văn ("thứ Sáu"), không tự đoán ngày; có đính chính thì lấy thông tin cuối; bỏ chuyện ngoài lề; không bịa thông tin [14]; câu trích phải nguyên văn từ đúng dòng được dẫn.

Mô hình được chọn bằng chấm thử có đáp án trên một cuộc họp tiếng Việt mẫu chứa đủ hiện tượng khó (đề xuất khác quyết định, hạn bị đính chính, đổi người làm giữa chừng, chuyện ngoài lề, câu tiếng Anh xen kẽ, người nói chỉ có nhãn tự động); mỗi mô hình ứng viên chạy nhiều lần, chấm theo đáp án và đo thời gian. Chỉ xét mô hình do FPT tự vận hành để dữ liệu hội thoại không ra ngoài hạ tầng FPT. Kết quả, DeepSeek-V4.1-Flash (họ DeepSeek [8]) được chọn làm mô hình chính vì chính xác nhất dù chậm hơn, GLM-5.2 (họ GLM [31]) nhanh hơn nhiều làm dự phòng; số liệu ở chương Thực nghiệm và đánh giá (Hình 4.2).

Lời gọi LLM dùng chế độ truyền luồng (stream): khi gọi thường, cổng dịch vụ trả lỗi hết thời gian chờ (HTTP 524) nếu mô hình suy luận quá lâu, còn ở chế độ luồng các token suy luận chảy về liên tục nên kết nối không bị cắt; phần suy luận bị bỏ qua, chỉ ghép phần nội dung, và tổng thời gian vẫn bị giới hạn. Worker thử mô hình chính rồi mô hình dự phòng; một mô hình bị coi là không dùng được khi lỗi HTTP, trả nội dung rỗng, hết token trước khi xong JSON hoặc JSON hỏng; cả hai lỗi thì phát `analysis.failed` kèm lỗi của cả hai. Kết quả được chuẩn hoá: bỏ mục không tiêu đề, quy "null", "N/A" về rỗng, `priority` ngoài {high, medium, low} quy về medium, `confidence` cắt vào [0, 1]. Số dòng bằng chứng được đối chiếu lại với câu trích, vì mô hình đôi khi trích đúng câu nhưng ghi lệch dòng:

```
def ground(quote, sequence, texts):
    needle = _norm(quote).strip(" .,\"'“”…") if quote else ""
    if needle:
        if sequence in texts and needle in _norm(texts[sequence]):
            return sequence                 # số dòng model đưa là đúng
        for seq, text in texts.items():     # dò lại theo nội dung câu trích
            if needle in _norm(text):
                return seq
    return sequence if sequence in texts else None
```

Khi transcript rỗng, worker không gọi LLM mà trả kết quả rỗng, tránh để mô hình bịa biên bản. Cấu hình đọc từ biến môi trường; khoá API đặt trong tệp `.env` đã đưa vào `.gitignore`, thiếu khoá thì worker tự lùi về bộ phân tích giả lập. Tên người thực hiện (`assigneeHint`) chỉ là tên trong transcript; Core không tự ánh xạ sang tài khoản mà để người duyệt chọn.

3.7. Luồng duyệt và quản lý công việc

Kết quả AI không trở thành dữ liệu chính thức nếu chưa có người duyệt. Biên bản quản lý theo phiên bản với ba trạng thái `draft` → `pending_approval` → `approved`; người có quyền `minutes.approve` duyệt hoặc trả về nháp. Biên bản đã duyệt không sửa trực tiếp: thao tác "tạo nháp mới" sinh phiên bản kế tiếp chép nội dung bản đã duyệt, bản cũ giữ nguyên; chạy lại AI cũng chỉ tạo nháp mới.

Đề xuất của AI nằm trong `review_items` với ba loại: công việc, quyết định, vấn đề còn mở; mỗi mục giữ song song bản AI sinh và bản người dùng sửa. Duyệt một đề xuất công việc sinh đúng một `action_items` nguồn `ai`, ghi người duyệt, độ tin cậy, mức ưu tiên, người thực hiện do người duyệt chọn; nếu chỉ có câu gốc như "thứ Sáu" mà chưa chốt ngày thì task được đánh dấu cần xác nhận hạn. Duyệt đề xuất quyết định sinh một `decisions`. Nguồn gốc của task được truy qua đề xuất đã sinh ra nó: câu trích và số dòng nằm trong nội dung đề xuất, lô duyệt giữ định danh phiên bản transcript nguồn; bảng `action_item_citations` đã có nhưng thao tác duyệt chưa ghi vào — phần này đang hoàn thiện. Sau khi duyệt, PM/Admin hoặc người được giao đổi được trạng thái task, còn giao người, đổi hạn, đổi ưu tiên chỉ người có quyền `task.assign` làm được; dịch vụ chỉ coi là "lên kế hoạch" khi giá trị thật sự khác giá trị đang lưu, vì giao diện gửi lại cả trường không đổi.

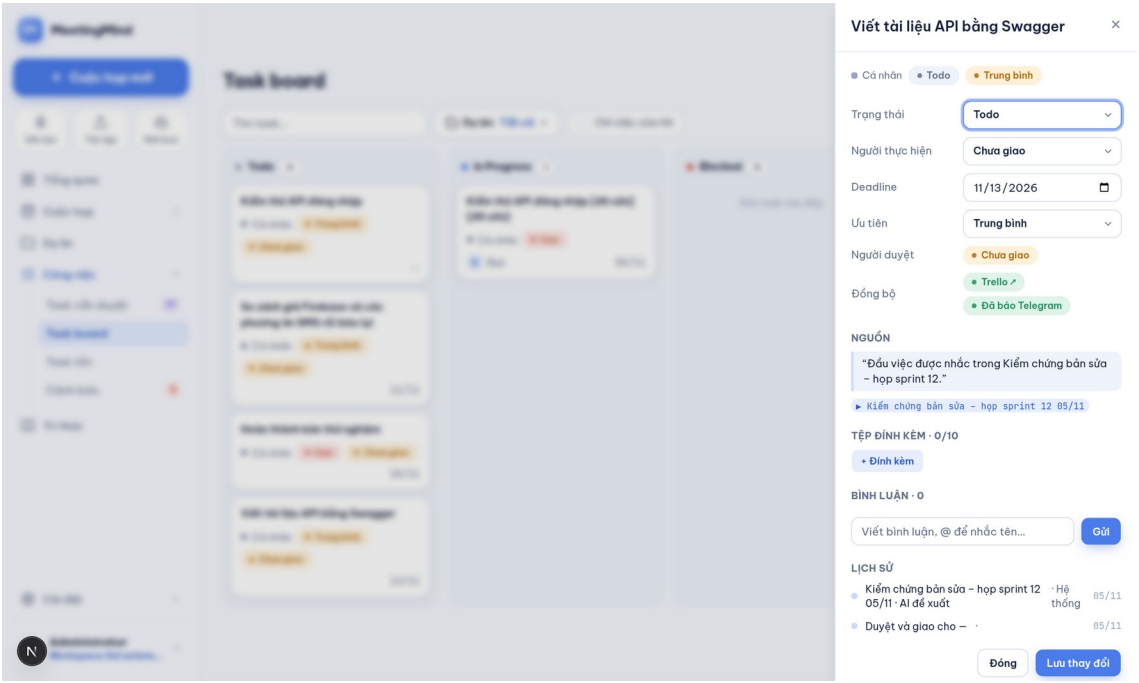
Gán tên người nói (speaker alias) là thao tác có chủ đích của người dùng. Khi gắn nhãn "Người nói 2" với một thành viên, các đoạn transcript chuyển sang danh tính `confirmed` và `speaker_aliases` ghi người xác nhận. Thao tác này **không** tự đổi người được giao của task đã duyệt; thay vào đó, task của cuộc họp được đánh dấu `needsReviewReason = speaker_changed` để PM rà soát.

Vì phân tích AI chạy ngay khi có transcript, tức là trước khi người dùng kịp gán tên, hệ thống cung cấp thao tác **phân tích lại** (`POST /workflow/meetings/:id/reanalyze`, nút "Phân tích lại" trong khung người nói). Khi gán tên, bí danh lưu tên hiển thị của thành viên; khi phân tích lại, transcript gửi sang AI mang tên thật thay cho nhãn "Người nói N", nhờ đó người thực hiện trong đề xuất mới là tên thành viên. Mỗi lần phân tích lại là một job mới với khoá chống lặp riêng, sinh biên bản nháp và lô đề xuất mới; biên bản, công việc đã duyệt giữ nguyên.

3.8. Mô-đun đồng bộ Trello và cảnh báo Telegram

Mô-đun này hiện thực phần đồng bộ trong thiết kế ở chương Phân tích và thiết kế: đưa công việc *đã được duyệt* sang bảng Trello [2] của nhóm và báo tin qua Telegram [32]. Mô-đun nằm trong integration-service — dịch vụ NestJS trước đó chỉ là khung — dưới tên `task-sync`, và được làm ở mức MVP: mỗi nhà cung cấp một kết nối cấu hình qua biến môi trường, đồng bộ một chiều (chỉ tạo thẻ, chưa cập nhật trạng thái ngược lại); đồng bộ Jira chưa được hiện thực.

Luồng xử lý. Khi người duyệt chấp nhận một đề xuất công việc, cùng với việc tạo `action_items`, Core ghi sự kiện `task.sync.requested` vào outbox. Vì integration-service không được đọc schema `core`, thông điệp mang ảnh chụp công việc tại lúc duyệt: tiêu đề, mô tả, tên người thực hiện, hạn dạng ISO 8601 và câu hạn gốc, mức ưu tiên, tên cuộc họp, câu trích nguồn, cùng khoá `<actionItemId>:push`. Bộ phát outbox đẩy sự kiện lên exchange topic `meetingmind`; integration-service, nay đã bật kết nối RabbitMQ, nhận nó qua hàng đợi bền `integration.jobs.task-sync` rồi xử lý lần lượt Trello và Telegram. Sau khi ghi kết quả vào cơ sở dữ liệu, dịch vụ phát `task.sync.completed` gồm `actionItemId` và kết quả của từng nhà cung cấp (trạng thái `synced`, `failed` hoặc `skipped`, mã và đường dẫn đối tượng ngoài, thông báo lỗi). Core lưu danh sách này vào trường `externalLinks` của công việc; ngăn chi tiết công việc trên giao diện có thêm dòng "Đồng bộ" với liên kết "Trello" dẫn tới thẻ, nhãn "Đã báo Telegram" hoặc lỗi của từng bên (Hình 3.4).



Hình 3.4. *Ngăn chi tiết công việc hiển thị kết quả đồng bộ Trello và Telegram*

Hành vi theo nhà cung cấp được tóm tắt ở Bảng 3.2. Danh sách "Việc từ cuộc họp" được dịch vụ tự tìm hoặc tạo trên bảng Trello đã cấu hình, mã danh sách được lưu lại cho các lần sau. Thẻ mang tiêu đề công việc và trường hạn nếu ngày đã chốt; mô tả thẻ ghi người thực hiện, hạn hoặc câu hạn gốc, ưu tiên, tên cuộc họp, câu trích nguồn và kết thúc bằng dòng đánh dấu `MeetingMind #<actionItemId>`. Tin Telegram "Công việc mới từ cuộc họp" được gửi qua một bot riêng của MeetingMind và kèm đường dẫn thẻ Trello nếu bước trước thành công — vì vậy Telegram chạy sau Trello. Phương thức `getUpdates` chỉ được dùng để dò cuộc trò chuyện đã nhắn cho bot khi chưa cấu hình mã cuộc trò chuyện.

Bảng 3.2. *Hành vi của mô-đun đồng bộ theo từng nhà cung cấp*

Tiêu chí	Trello	Telegram
Thao tác	Tạo thẻ trong danh sách "Việc từ cuộc họp" (tự tìm hoặc tạo trên bảng cấu hình)	Gửi tin `sendMessage` (HTML) qua bot riêng của MeetingMind
Nội dung	Tiêu đề, người thực hiện, hạn hoặc câu hạn gốc, ưu tiên, cuộc họp, câu trích, dòng đánh dấu	Tiêu đề, người thực hiện, hạn, ưu tiên, cuộc họp, đường dẫn thẻ Trello
Chống trùng	Job theo khoá; tra `external_task_links`; dò thẻ mang dòng đánh dấu trước khi tạo	Job theo khoá; không tra cứu được tin đã gửi
Nơi lưu kết quả	`external_task_links` (externalId, externalUrl, syncStatus)	`sync_jobs` (mã tin nhắn)
Thiếu khoá cấu hình	`skipped`, không gọi API	`skipped`, không gọi API

Chống tạo trùng. Tài liệu thiết kế cơ sở dữ liệu đã cảnh báo rằng chống lặp thông điệp nội bộ chưa đủ cho lời gọi ra hệ thống ngoài, nên mô-đun xử lý ở ba lớp. Ở lớp thông điệp, consumer ghi cặp (eventId, tên consumer) vào `processed_messages` trước mọi thao tác khác; chỉ lỗi vi phạm khoá duy nhất mới được coi là "đã xử lý", lỗi cơ sở dữ liệu khác được ném ra để tin được giao lại — bộ chống lặp của Core trước đó nuốt mọi lỗi và có thể làm mất tin, nên được sửa cùng lúc. Ở lớp job, mỗi nhà cung cấp có một bản ghi `sync_jobs` với khoá duy nhất `<actionItemId>:push:<provider>`; job đã thành công thì kết quả cũ được dùng lại, không gọi API lần nữa. Ở lớp nền tảng ngoài, Trello không hỗ trợ khoá idempotency nên lời gọi tạo thẻ có thể đã thành công nhưng mất phản hồi; vì vậy, trước khi tạo, dịch vụ tra `external_task_links` rồi dò trong danh sách đích thẻ mang dòng đánh dấu của công việc và dùng lại nếu tìm thấy:

```
const idempotencyKey = `${payload.idempotencyKey}:${provider}`; //
<id>:push:trello
const job = await this.syncJobService.getOne(null, { idempotencyKey });
if (job?.status === SyncJobStatus.SUCCEEDED) return job.result; // không gọi
API lại
// ...
// Trello không có khoá idempotency: dò dòng đánh dấu trước khi tạo thẻ mới.
const marker = trelloMarker(payload.actionItemId);             //
"MeetingMind #<id>"
let card = await this.trello.findCardByMarker(listId, marker);
if (!card) {
    card = await this.trello.createCard({ listId, name: payload.task.title,
        desc: buildTrelloDescription(payload), due });
}
```

Telegram không có cơ chế tương tự: Bot API không cho tra cứu tin đã gửi, nên nếu `sendMessage` thành công nhưng mất phản hồi, lần thử lại có thể gửi trùng một tin. Nhóm chấp nhận giới hạn này vì một tin báo trùng ít hại hơn một thẻ công việc trùng.

Xử lý lỗi. Mỗi nhà cung cấp chạy độc lập: lỗi API hay cấu hình sai của Trello chỉ làm kết quả Trello là `failed`, tin Telegram vẫn được gửi (không kèm đường dẫn thẻ), và ngược lại. Thiếu khoá cấu hình thì nhà cung cấp đó nhận trạng thái `skipped` chứ không bị coi là lỗi, nên hệ thống vẫn chạy khi chưa kết nối công cụ nào. Với lỗi hạ tầng (cơ sở dữ liệu, RabbitMQ), consumer trả lại bản ghi chống lặp, chờ một nhịp rồi để tin quay lại hàng đợi. Sự kiện `task.sync.completed` được phát thẳng sau khi ghi cơ sở dữ liệu, không qua outbox; điều này chấp nhận được vì Core xử lý lũy đẳng theo `actionItemId` (ghi đè danh sách liên kết), còn nếu phát lỗi thì tin gốc được giao lại và các job đã thành công chỉ phát lại kết quả cũ. Hệ thống chưa có hàng đợi thư chết cho các lần đồng bộ lỗi.

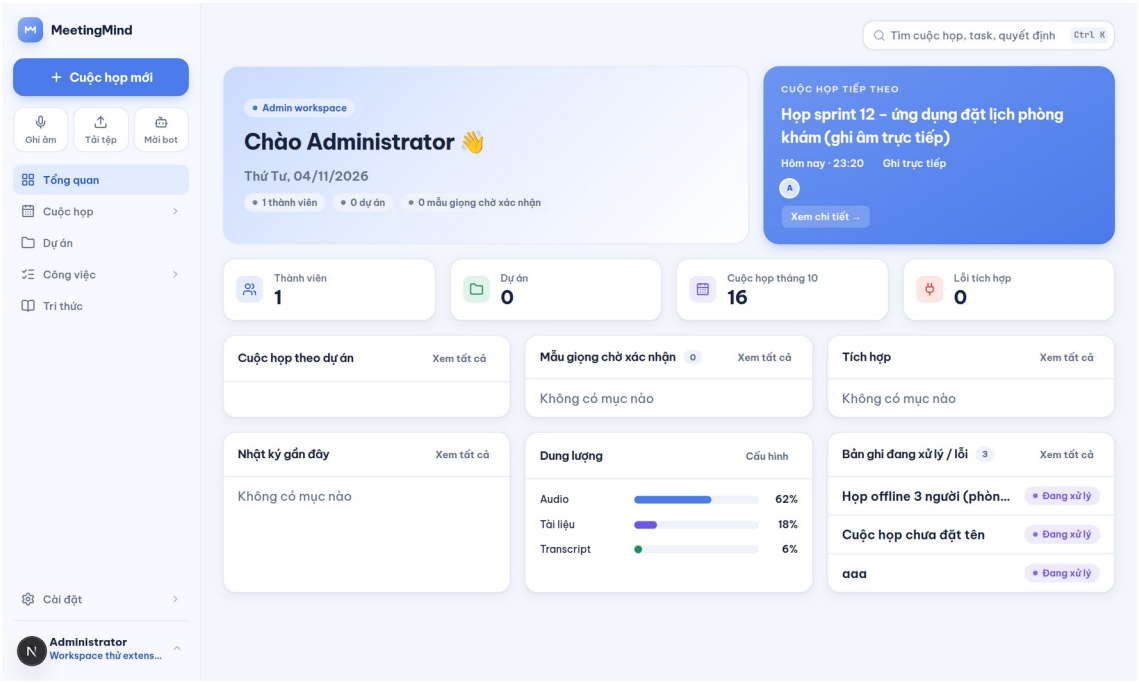
Bảo mật khoá. Khoá API và token Trello, token bot Telegram chỉ được đọc từ biến môi trường (tệp `.env` đã nằm trong `.gitignore`), không ghi vào cơ sở dữ liệu; bảng `integrations` chỉ giữ thông tin không bí mật. Vì khoá Trello nằm trên chuỗi truy vấn còn token Telegram nằm ngay trên đường dẫn URL, mọi thông báo lỗi đều đi qua một hàm xoá bí mật trước khi ghi log, ghi cơ sở dữ liệu hay gửi về Core; nhóm đã kiểm tra log và thông báo lỗi không chứa khoá.

Lỗi khung mẫu. Khi bật RabbitMQ cho integration-service, nhóm gặp một lỗi kế thừa từ khung mẫu: các interceptor và bộ lọc ngoại lệ HTTP được gắn toàn cục nên cũng chạy với handler RabbitMQ, vốn không có đối tượng yêu cầu và phản hồi HTTP; handler bị sập và tin bị trả lại hàng đợi khoảng 1.500 lần mỗi giây. Các thành phần này được sửa để bỏ qua ngữ cảnh không phải HTTP.

Kết quả kiểm thử. Mô-đun có 20 ca kiểm thử đơn vị Jest, tổng hợp ở mục Kiểm thử phần mềm. Trong kiểm thử đầu-cuối trên Docker Compose với tài khoản Trello và bot Telegram thật, duyệt công việc "Viết tài liệu API bằng Swagger" thì sau 3,4 giây thẻ được tạo trên bảng "MeetingMind", trong danh sách "Việc từ cuộc họp" do hệ thống tự tạo; tin Telegram kèm đường dẫn thẻ được gửi; Core lưu cả hai kết quả và giao diện hiển thị như Hình 3.4. Một lần duyệt trước đó, khi chưa cấu hình khoá Trello, cho kết quả Trello `skipped` và tin Telegram đến sau khoảng 2 giây, đúng thiết kế bỏ qua nhà cung cấp thiếu cấu hình.

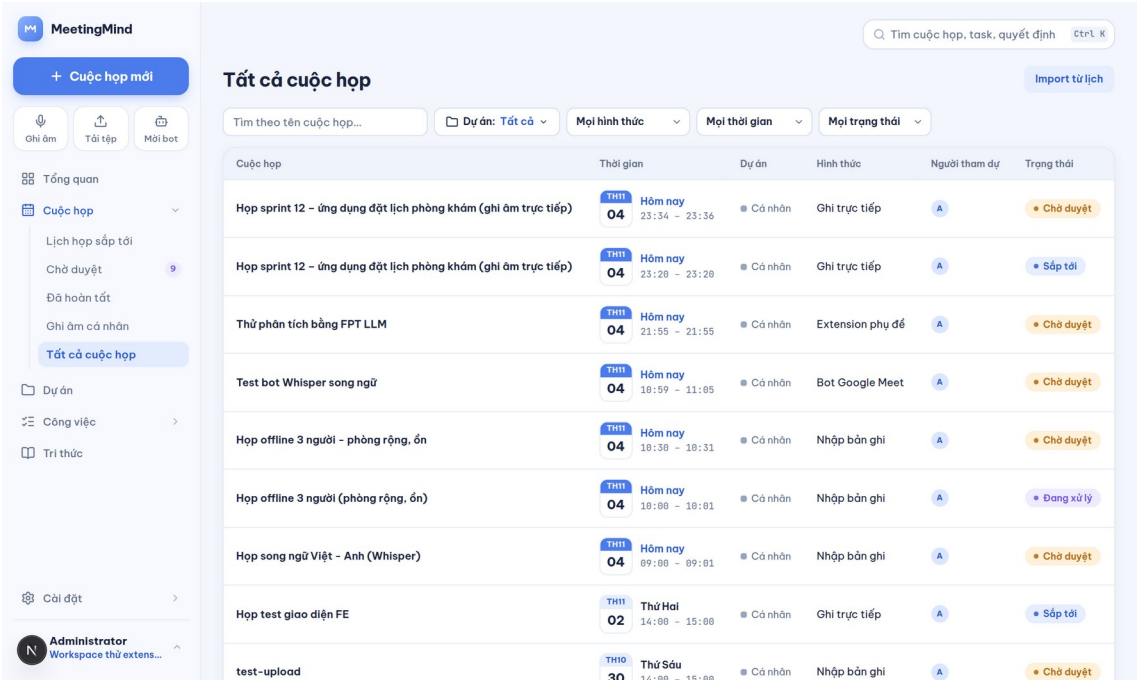
3.9. Giao diện web

Giao diện xây dựng bằng Next.js 16 (App Router), React 19, Tailwind CSS 4 và shadcn/Radix UI. Trình duyệt chỉ gọi `/api/...` cùng nguồn gốc, Next.js chuyển tiếp thẳng tới core-service (cổng 3010) nên không phải cấu hình CORS; cũng như extension, giao diện hiện chưa đi qua api-gateway. Giao diện được thiết kế trước dưới dạng bản mẫu, sau đó mới nối backend qua một lớp chuyển đổi: `live.ts` gom mọi lời gọi API, `adapters.ts` chuyển dữ liệu backend sang kiểu giao diện, nên các thành phần không phải viết lại. Có phiên đăng nhập thì chạy chế độ *live*; chưa đăng nhập, nút "Xem demo" mở chế độ *demo* với dữ liệu mẫu. Ở chế độ live, sau mỗi thao tác giao diện nạp lại từ backend thay vì tự sửa trạng thái, vì backend quyết định kết quả. Các trang mà backend chưa có API — ghi âm trực tiếp trên web, hỏi đáp tri thức (RAG), khách mời, mẫu giọng, tích hợp, mẫu biên bản, lưu trữ, nhật ký — mới là giao diện mẫu, chỉ có dữ liệu ở chế độ demo.



Hình 3.5. *Màn hình tổng quan của người quản lý dự án*

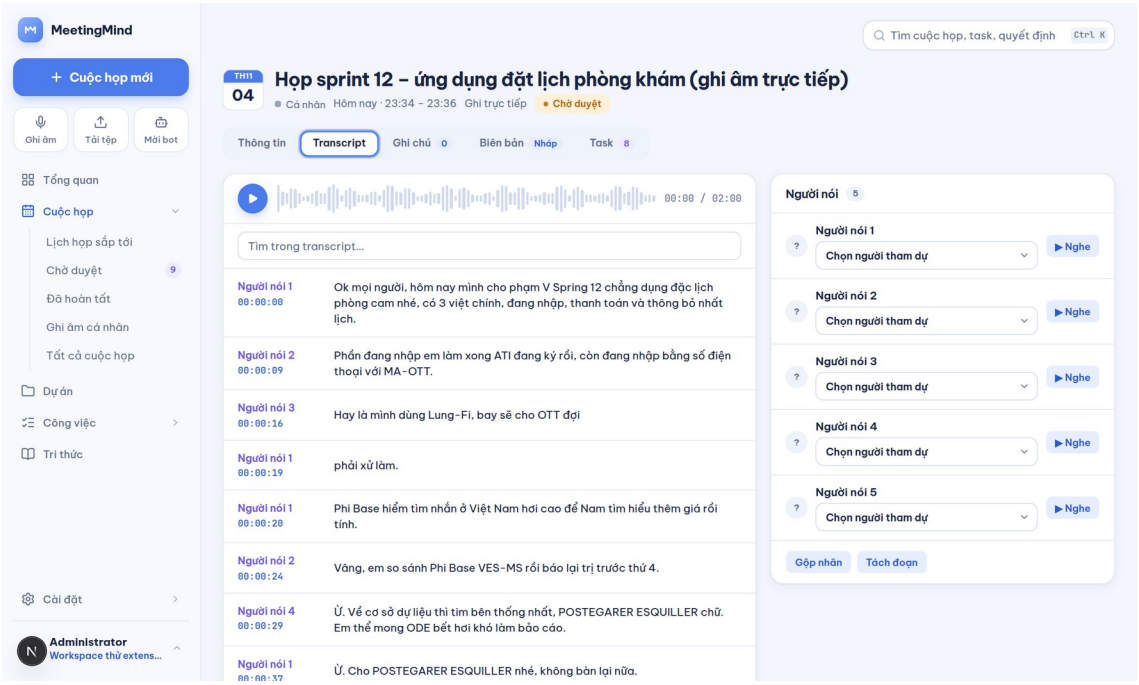
Màn hình tổng quan (Hình 3.5) thay đổi theo vai trò. Với PM, trang hiển thị chỉ số nhanh (cuộc họp trong tuần, mục chờ duyệt, task đang mở, task quá hạn), lịch họp sắp tới, danh sách chờ duyệt và việc của tôi. Ô cảnh báo được tính ở giao diện theo quy tắc đơn giản (quá hạn, bị chặn, chưa có người thực hiện); phát hiện rủi ro tự động ở backend chưa triển khai.



Hình 3.6. *Danh sách cuộc họp với bộ lọc và giai đoạn xử lý*

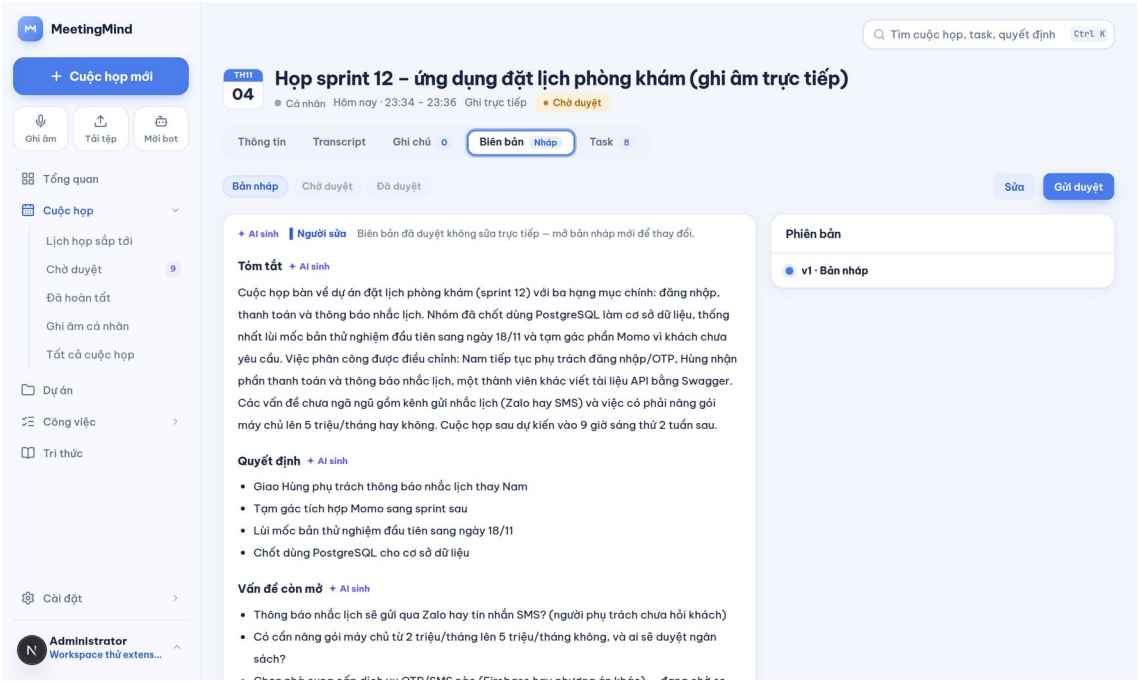
Danh sách cuộc họp (Hình 3.6) cho phép tìm kiếm, lọc theo dự án, hình thức (tải tệp, bot, extension, họp trực tiếp), thời gian, trạng thái. Mỗi cuộc họp mang giai đoạn

hiển thị do backend gộp từ trạng thái cuộc họp, job xử lý gần nhất và trạng thái biên bản: đã lên lịch, đang xử lý, lỗi, chờ duyệt, hoàn tất.



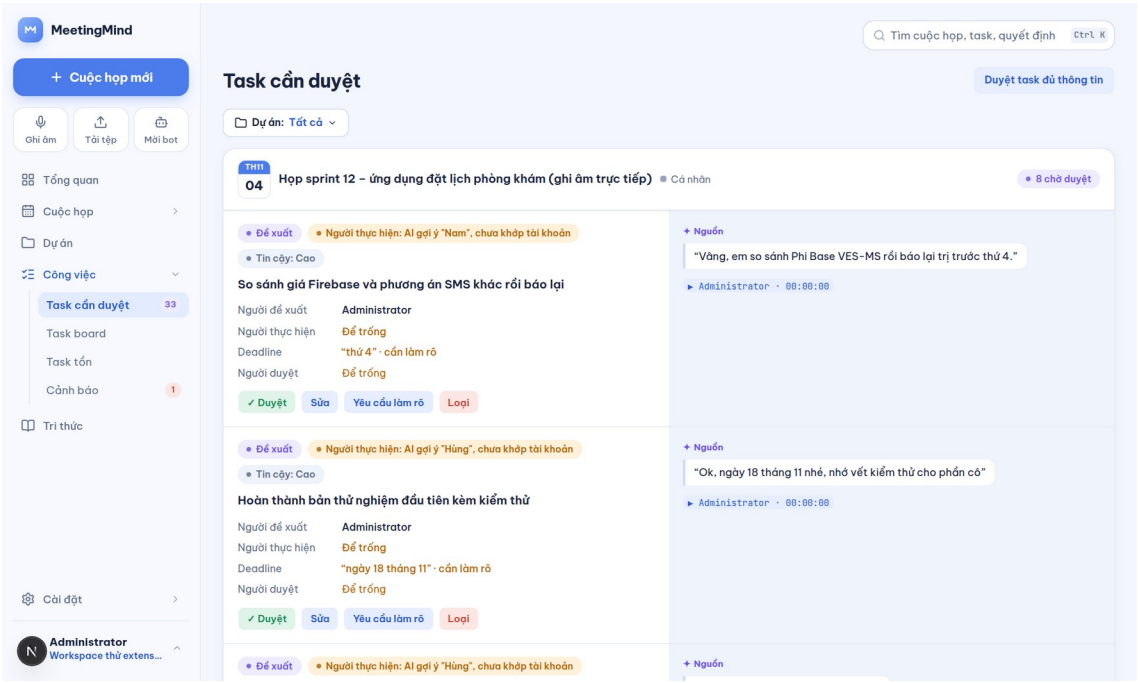
Hình 3.7. *Màn hình transcript với nhãn ngôn ngữ, cảnh báo nói chồng và gán người nói*

Màn hình transcript (Hình 3.7) hiển thị từng đoạn với người nói, mốc thời gian và ô tìm kiếm. Khi cuộc họp có từ hai thứ tiếng, mỗi đoạn có nhãn mã ngôn ngữ (VI, EN); đoạn có cờ nói chồng mang nhãn "Nói chồng · nên nghe lại". Cột bên phải liệt kê người nói, cho phép gán nhãn tự động với thành viên. Thanh phát âm thanh hiện mới mô phỏng; API cấp URL ký đã có nhưng giao diện chưa nối vào.



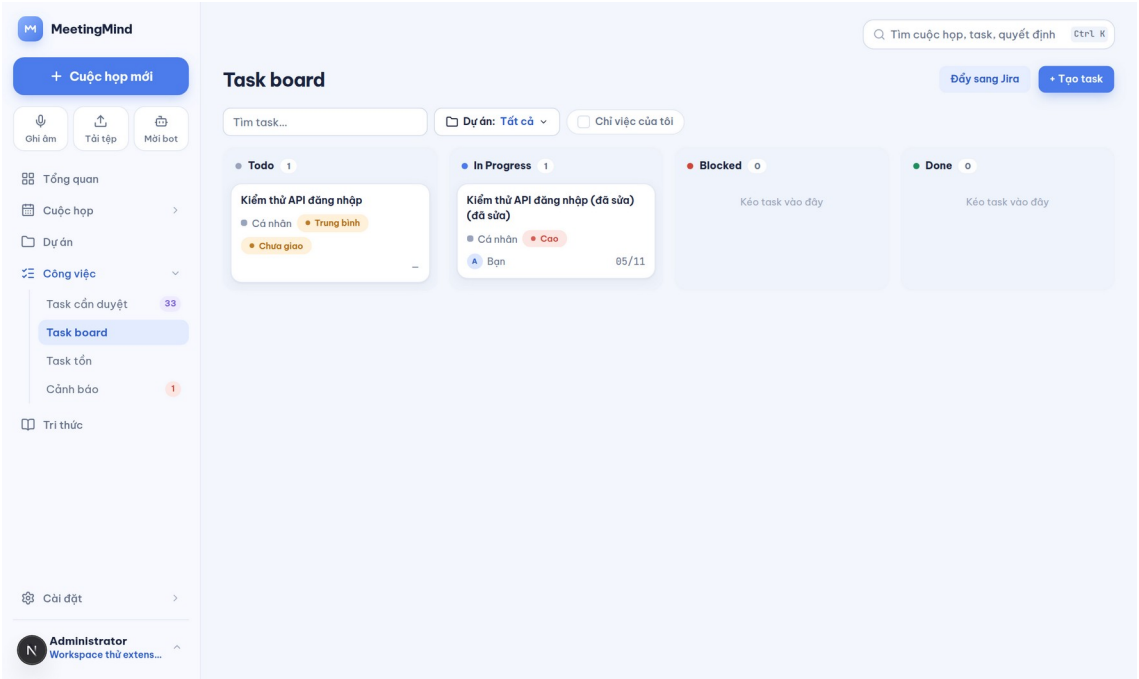
Hình 3.8. *Màn hình biên bản và vòng duyệt theo phiên bản*

Màn hình biên bản (Hình 3.8) hiển thị tóm tắt, quyết định, vấn đề còn mở, công việc và phiên bản; nút thao tác đổi theo trạng thái: "Gửi duyệt" với bản nháp, "Duyệt biên bản" và "Từ chối" với bản chờ duyệt, "Mở bản nháp mới" với bản đã duyệt.



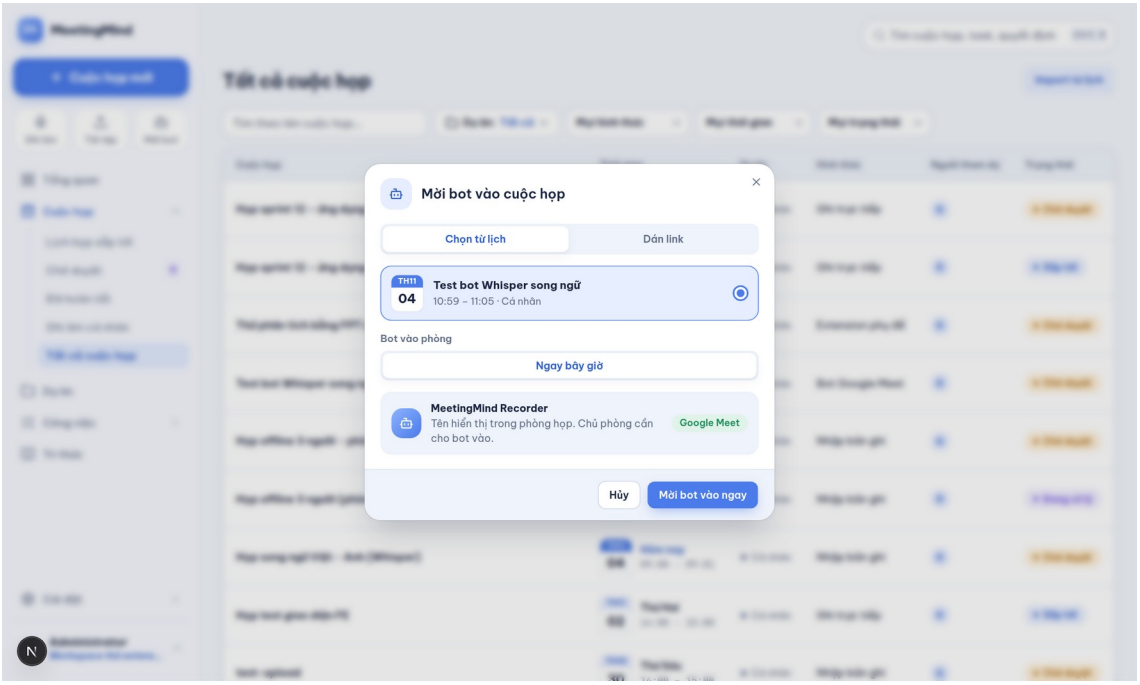
Hình 3.9. *Hàng đợi duyệt đề xuất công việc do AI trích xuất*

Hàng đợi duyệt (Hình 3.9) nhóm đề xuất theo cuộc họp. Mỗi thẻ có dấu hiệu nội dung do AI sinh, mức tin cậy, các trường sửa được (người thực hiện, hạn) và câu trích nguyên văn làm bằng chứng; người duyệt chọn "Duyệt", "Sửa" hoặc "Loại". Đây là điểm kiểm soát của con người: chỉ sau khi duyệt, đề xuất mới thành task chính thức.



Hình 3.10. *Bảng công việc theo trạng thái*

Bảng công việc (Hình 3.10) trình bày task theo bốn cột Todo, In Progress, Blocked, Done kèm dự án, hạn, ưu tiên; mở một task sẽ hiện ngăn chi tiết với bình luận.

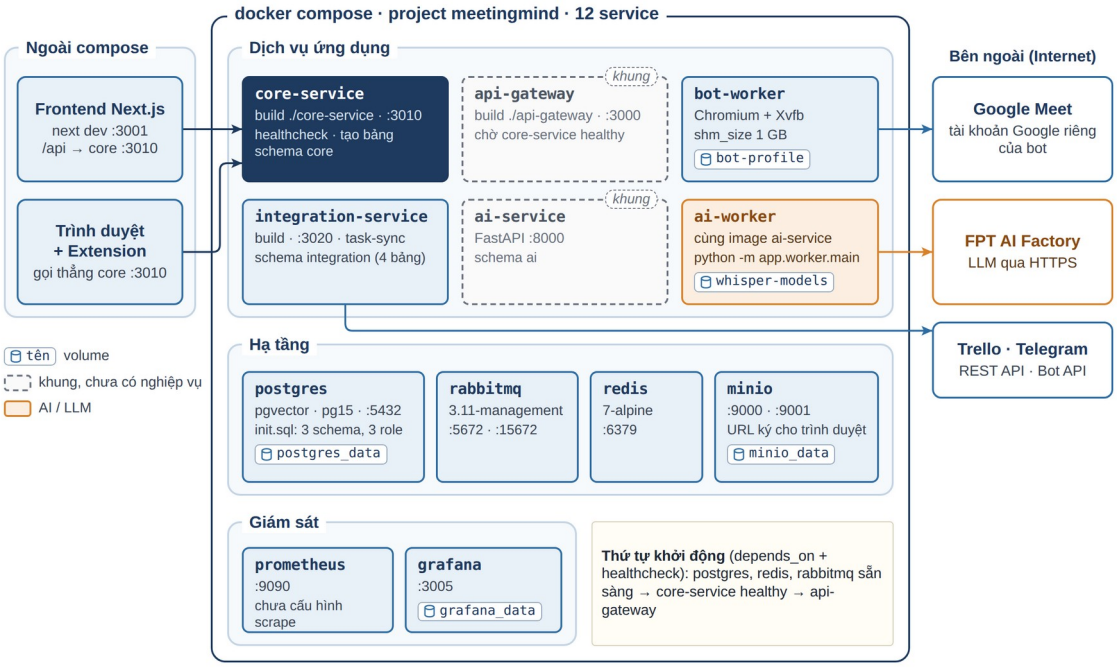


Hình 3.11. *Hộp thoại mời bot ghi âm vào cuộc họp Google Meet*

Hộp thoại mời bot (Hình 3.11) cho chọn cuộc họp trực tuyến trong lịch hoặc dán đường dẫn Google Meet mới; ở chế độ live chỉ có lựa chọn "Ngay bây giờ" vì Core chưa có bộ hẹn giờ cho bot. Sau khi mời, trang cuộc họp hỏi trạng thái bot mỗi 4 giây, hiển thị "Chờ chủ phòng cho vào", "Bot đang ghi âm" hoặc lý do thất bại cùng nút "Dừng bot".

3.10. Triển khai và vận hành

Backend được mô tả trong một tệp `docker-compose.yml` gồm 12 dịch vụ (Hình 3.12): PostgreSQL 15, Redis 7, RabbitMQ 3.11, MinIO, api-gateway, core-service, integration-service, ai-service, ai-worker, bot-worker, Prometheus và Grafana; frontend chạy riêng. ai-worker dùng chung ảnh với ai-service nhưng chạy tiến trình nhận việc từ RabbitMQ thay vì máy chủ HTTP. Thứ tự khởi động ràng buộc bằng kiểm tra sức khoẻ; chỉ dịch vụ sở hữu schema được bật tự sinh bảng, tránh nhiều tiến trình cùng sửa bảng lúc khởi động. Tệp `init.sql` tạo ba schema `core`, `integration`, `ai` cùng ba tài khoản riêng và thu hồi quyền chéo giữa các schema.



Hình 3.12. *Triển khai hệ thống bằng Docker Compose: các container và volume*

Dữ liệu bền giữ trong năm volume: `postgres_data`, `minio_data`, `grafana_data`, `whisper-models` (mô hình Whisper và ERes2Net, tải lần đầu) và `bot-profile` (hồ sơ Chromium đã đăng nhập tài khoản bot). Cấu hình đi qua biến môi trường: `AI_STT`, `AI_WHISPER_MODEL`, `AI_DIARIZATION` cho phiên âm; `AI_LLM`, `AI_LLM_MODEL`, `AI_LLM_FALLBACK_MODEL` cho phân tích; `BOT_ADMISSION_TIMEOUT_S`, `BOT_ALONE_TIMEOUT_S` cho bot; `TRELLO_API_KEY`, `TRELLO_TOKEN`, `TRELLO_BOARD_ID` (hoặc `TRELLO_LIST_ID`), `TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID` cho đồng bộ. Tệp `.env` chứa khoá LLM, khoá Trello, token Telegram và khoá ký gói extension (`*.pem`) đều bị `.gitignore` loại; các mật khẩu mặc định trong `docker-compose.yml` chỉ dành cho môi trường phát triển.

Về giám sát, Prometheus và Grafana đã được khai báo nhưng chưa có cấu hình thu thập số liệu, các dịch vụ cũng chưa xuất điểm cuối số liệu, nên giám sát mới ở mức khung. Khi vận hành và gỡ lỗi, nhóm dựa vào log của từng dịch vụ, trang quản trị RabbitMQ để xem hàng đợi, giao diện MinIO để kiểm tra object, trạng thái trong `processing_jobs`, `bot_sessions`, và ảnh chụp từng bước của bot.

3.11. Kiểm thử phần mềm

3.11.1. Kiểm thử đơn vị

Nguyên tắc giúp kiểm thử là tách phần quyết định thành hàm thuần, độc lập với mô hình và mạng: ở ai-worker, các hàm chọn ngôn ngữ, cắt khúc, gom cụm, phát hiện nói chồng, dựng prompt, bóc JSON, đối chiếu bằng chứng đều là hàm thuần; phần gọi

LLM được kiểm thử bằng `httpx.MockTransport` giả lập lỗi 402, nội dung rỗng, JSON bị cắt. Bộ kiểm thử chạy trong khoảng một giây, không cần tải mô hình. Ở integration-service, các lời gọi Trello và Telegram được thay bằng bản giả lập để kiểm thử từng nhánh: tạo thẻ rồi báo tin, dùng lại thẻ đã có dòng đánh dấu, job đã thành công không gọi API lại, thiếu cấu hình thì bỏ qua, lỗi của một bên không chặn bên kia, khoá không lộ trong lỗi, log, cơ sở dữ liệu và sự kiện. Tất cả các ca trong bảng Bảng 3.3 đều đạt ở lần chạy gần nhất.

Bảng 3.3. *Tổng hợp kiểm thử đơn vị*

Thành phần	Công cụ	Số ca	Nội dung chính
ai-worker – nhận dạng tiếng nói	pytest	8	Họp nói lẫn Việt – Anh, đoạn ngắn giữ ngôn ngữ, tỷ lệ ngôn ngữ
ai-worker – tách người nói	pytest	15	Cắt khúc, gom cụm, gộp cụm nhỏ, nói chồng, lọc câu bịa
ai-worker – phân tích LLM	pytest	21	Prompt, bóc JSON, đối chiếu số dòng, mô hình dự phòng, gọi theo luồng, quá giờ
bot-worker	pytest	3	Ép `hl=en`, so khớp tiền tố nhãn nút
Extension	node: test	7	Phần chưa gửi của phụ đề, nạp content script chung phạm vi
integration-service	Jest	20	Đồng bộ Trello/Telegram, chống lặp, ẩn khoá

Core-service chưa có bộ kiểm thử đơn vị do nhóm viết (các tệp `*.spec.ts` hiện có thuộc khung mẫu); luồng nghiệp vụ của nó được kiểm chứng bằng kiểm thử đầu-cuối.

3.11.2. Kiểm thử tích hợp đầu-cuối

Kiểm thử đầu-cuối chạy trên toàn hệ thống bằng Docker Compose, gọi API như frontend và extension rồi đối chiếu cơ sở dữ liệu. Lát cắt đầu tiên dùng STT và LLM giả lập để xác nhận đường đi của sự kiện tới biên bản nháp. Với đường tải tệp, nhóm tải lên tệp WAV 64 KB, tải lại qua URL ký từ ngoài mạng Docker thấy khớp đủ 64.044 byte, rồi chạy tiếp ra transcript và biên bản nháp. Extension được thử trên phòng Google Meet thật ngày 29/10/2026; giao diện được chạy với backend thật ngày 30/10/2026 qua các thao tác từ đăng nhập, tạo cuộc họp, tải tệp tới duyệt đề xuất, duyệt biên bản và bảng công việc. Cuối cùng, bot và các mô-đun AI được chạy với phòng Google Meet và bản ghi thật, luồng đồng bộ được chạy với tài khoản Trello và bot Telegram thật. Đánh giá định lượng độ chính xác và thời gian xử lý trình bày ở chương Thực nghiệm và đánh giá (Hình 4.1, Hình 4.2, Hình 4.3).

3.11.3. Lỗi phát hiện và cách khắc phục

Ngoài các lỗi chức năng đã nêu ở từng mục, kiểm thử phát hiện một số lỗi bảo mật và tính đúng đắn ở core-service, phần lớn kế thừa từ khung mẫu ban đầu (bảng Bảng 3.4).

Bảng 3.4. *Các lỗi bảo mật và tính đúng đắn phát hiện trong quá trình kiểm thử*

Lỗi phát hiện	Hậu quả	Cách khắc phục
Bộ điều khiển `/user/*` khai `authorize: false`	Gọi được không cần đăng nhập, lấy được danh sách người dùng kèm băm mật khẩu, tạo được tài khoản SUPER_ADMIN	CRUD người dùng chỉ cho SUPER_ADMIN; tự đăng ký qua `/auth/register`, luôn là tài khoản thường
Đổi mật khẩu truyền `null` thay người dùng, ghi mật khẩu cũ dạng rõ	Chức năng luôn lỗi, nguy cơ lưu mật khẩu không băm	Lấy người dùng từ token, so mật khẩu cũ với băm, lưu băm bcrypt mật khẩu mới
Dòng băm mật khẩu khi tạo người dùng bị chú thích mất	Mật khẩu lưu dạng rõ	Bật lại băm bcrypt
`/user/me` thiếu guard và trả cả trường mật khẩu	Lỗi 500, lộ băm mật khẩu	Gắn `Authorization()`, bỏ trường mật khẩu khỏi phản hồi
Tạo workspace nhận `ownerId` từ client	Tạo được workspace đứng tên người khác	Người gọi luôn là chủ sở hữu
Cuộc họp không thuộc dự án không ai truy cập được	Người tạo cũng không xem được	Luôn `restricted`, người tạo nhận vai trò MANAGER
Tuyến CRUD `/note` cho người xem cuộc họp đọc ghi chú riêng của người khác	Lộ ghi chú cá nhân	Khoá CRUD chung cho SUPER_ADMIN; người dùng đi qua tuyến nghiệp vụ đã lọc ghi chú riêng
Cuộc họp tải tệp ghi âm vẫn ở trạng thái "đã lên lịch" sau khi có transcript	Giao diện ẩn tab transcript	Chuyển cuộc họp sang "đã kết thúc" khi nhận transcript từ bản ghi
Tải cùng nội dung cho cuộc họp khác trả về bản ghi tệp của cuộc họp cũ	Tệp gắn sai cuộc họp	Khử trùng lặp trong phạm vi từng cuộc họp
Bộ chống lặp coi mọi lỗi khi ghi `processed_messages` là tin trùng	Lỗi cơ sở dữ liệu tạm thời làm mất tin	Chỉ lỗi vi phạm khoá duy nhất mới coi là trùng; lỗi khác được ném ra để tin được giao lại

Bài học rút ra là mọi tuyến kế thừa từ khung mẫu phải được rà theo ma trận quyền, và thông tin danh tính (chủ sở hữu, người tạo) phải lấy từ token chứ không nhận từ client. Hạn chế bảo mật đã biết còn lại là dữ liệu hội thoại chưa được mã hoá khi lưu mà chỉ được bảo vệ bằng phân quyền.

3.12. Kết luận chương

Chương này đã trình bày việc xây dựng MeetingMind từ thiết kế thành hệ thống chạy được. Core-service tập trung phân quyền ở `PermissionService` và giao tiếp bất đồng bộ qua outbox, RabbitMQ với cơ chế chống xử lý lặp. Ba đường thu nhận — tải tệp, bot ghi âm Google Meet, extension đọc phụ đề — đã hoạt động và hội tụ tại phiên bản transcript. Ba mô-đun AI đã được gắn thật: phiên âm hai lượt theo ngôn ngữ từng đoạn, tách người nói bằng gom cụm vector giọng, phân tích biên bản bằng LLM có mô hình dự phòng và đối chiếu bằng chứng; kết quả AI luôn qua hàng đợi duyệt trước khi thành dữ liệu chính thức. Công việc đã duyệt được integration-service đẩy sang Trello và báo qua Telegram, có chống tạo trùng ở ba lớp và đã chạy với tài khoản thật.

Phần khó nhất nằm ở các tình huống thực tế hơn là ở việc gọi mô hình. Phần chưa hoàn thiện gồm bọc giao dịch cho outbox, sinh bản ghi trích dẫn khi duyệt, nối trình phát âm thanh, cấu hình giám sát, kiểm thử tự động cho core-service, mã hoá dữ liệu khi lưu; tích hợp Trello, Telegram mới ở mức MVP (cấu hình qua biến môi trường, đồng bộ một chiều, chưa có hàng đợi thư chết) và đồng bộ Jira còn ở mức thiết kế. Chương Thực nghiệm và đánh giá tiếp theo sẽ đo định lượng độ chính xác và thời gian xử lý.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày các thực nghiệm định lượng nhằm trả lời câu hỏi: hệ thống MeetingMind nhận dạng, tách người nói và trích xuất công việc chính xác đến đâu, chịu ảnh hưởng của nhiễu như thế nào, xử lý một cuộc họp mất bao lâu và tốn bao nhiêu chi phí. Mọi con số trong chương được sinh ra từ các tập lệnh đánh giá chạy trên chính mã nguồn của hệ thống; kết quả thô (transcript, đầu ra JSON của LLM, thời gian từng bước) được lưu lại để có thể kiểm tra và chạy lại.

4.1. Mục tiêu và câu hỏi thực nghiệm

Theo đề cương, nội dung kiểm thử gồm đánh giá độ chính xác của AI và đo lường thời gian tiết kiệm được. Nhóm cụ thể hoá thành sáu câu hỏi thực nghiệm:

1. Mô-đun phiên âm (Whisper) đạt tỷ lệ lỗi từ bao nhiêu trên hội thoại cuộc họp tiếng Việt có thuật ngữ kỹ thuật, và suy giảm thế nào khi có nhiễu?
2. Từ điển thuật ngữ và mô hình lớn hơn cải thiện được bao nhiêu?
3. Mô-đun tách người nói phân biệt người nói đúng đến đâu, có phát hiện được đoạn nói chồng không, và có sinh "câu bịa" ở đoạn chỉ có nhiễu không?
4. Trong các LLM khả dụng, mô hình nào trích xuất biên bản đúng nhất trên các tình huống dễ gây sai (đề xuất khác quyết định, đính chính, đổi người làm, chuyện ngoài lề)?
5. Khi ghép cả chuỗi từ âm thanh đến công việc, lỗi nhận dạng ảnh hưởng thế nào đến chất lượng trích xuất?
6. Xử lý một cuộc họp mất bao lâu trên phần cứng phổ thông, chi phí bao nhiêu, và ước tính tiết kiệm được bao nhiêu thời gian của con người?

4.2. Môi trường và bộ dữ liệu thực nghiệm

4.2.1. Môi trường

Toàn bộ thực nghiệm chạy trên một máy tính xách tay cá nhân, không dùng GPU, để phản ánh điều kiện triển khai thực tế của một nhóm nhỏ. Mô-đun AI chạy trong đúng container ai-worker của hệ thống, nên kết quả phản ánh chính mã nguồn được triển khai. Cấu hình được tóm tắt trong Bảng 4.1.

Bảng 4.1. *Môi trường thực nghiệm*

Hạng mục	Cấu hình
CPU	Intel Core i7-10750H (6 nhân, 12 luồng, 2,6 GHz)
Bộ nhớ	23 GB RAM, không có GPU
Hệ điều hành, đóng gói	Linux, Docker Compose (container ai-worker của hệ thống)

Hạng mục	Cấu hình
Nhận dạng tiếng nói	faster-whisper 1.2.1, mô hình small và medium, int8 trên CPU, beam size 5
Tách người nói	sherpa-onnx 2.0.1, ERes2Net (3D-Speaker), ngưỡng tương đồng 0,3
LLM	FPT AI Factory Serverless Inference, nhiệt độ 0,2
Giọng tổng hợp dựng dữ liệu	FPT.AI-VITs (các giọng banmai, leminh, giahuy, ngoclam, kimngan, huyphong, minhquan)

4.2.2. Kịch bản cuộc họp có đáp án

Do không có sẵn bộ dữ liệu cuộc họp tiếng Việt công khai kèm nhãn công việc, nhóm tự xây dựng một kịch bản họp lập kế hoạch sprint của một nhóm phát triển ứng dụng đặt lịch phòng khám, gồm 30 lượt lời của 4 người (Lan – trưởng nhóm, Nam, Hùng và một thành viên chỉ xuất hiện dưới nhãn "Người nói 4" nhưng được gọi tên là Phương ở cuối buổi). Kịch bản được thiết kế có chủ đích để chứa các tình huống mà một trợ lý biên bản dễ làm sai:

- **Đề xuất chưa được chấp nhận:** "Hay là mình dùng luôn Firebase cho OTP?" không phải quyết định, mà dẫn tới một việc so sánh giá và một vấn đề còn mở.
- **Đính chính thời hạn:** hạn tích hợp thanh toán được nói là ngày 15 tháng 11 rồi sửa ngay thành ngày 18.
- **Đổi người thực hiện:** việc nhắc lịch "ban đầu định giao Nam" nhưng cuối cùng giao cho Hùng.
- **Việc thiếu thời hạn:** nhiều việc không nêu hạn, hệ thống không được tự điền.
- **Chuyện ngoài lề:** hai lượt lời bàn về bóng đá không được biến thành công việc.
- **Người nói chưa có tên:** việc viết tài liệu API của "Người nói 4" phải được gán cho Phương nhờ câu tổng kết.
- **Thuật ngữ tiếng Anh:** sprint, API, OTP, Firebase, eSMS, PostgreSQL, MongoDB, VNPay, MoMo, callback, Zalo, SMS, Swagger.

Đáp án gồm 8 công việc (kèm người thực hiện và thời hạn đúng), 2 quyết định (chốt PostgreSQL, chưa làm MoMo trong sprint này) và 3 vấn đề còn mở (chọn Firebase hay eSMS, gửi nhắc lịch qua Zalo hay SMS, ngân sách máy chủ).

4.2.3. Dựng âm thanh bằng giọng tổng hợp

Từ kịch bản, mỗi lượt lời được tổng hợp thành tiếng nói bằng dịch vụ FPT.AI-VITs, mỗi người dùng một giọng khác nhau (hai giọng nam, hai giọng nữ). Các câu được ghép với khoảng nghỉ ngẫu nhiên 0,3–0,9 giây thành một bản ghi dài 151 giây, với mốc thời gian và người nói chuẩn của từng câu được lưu lại. Từ bản sạch, nhóm

tạo thêm các điều kiện khó hơn như trong Bảng 4.2. Bản ghi tham chiếu có 443 tiếng (âm tiết) và 24 lần xuất hiện thuật ngữ tiếng Anh.

Bảng 4.2. *Các điều kiện âm thanh thử nghiệm*

Điều kiện	Cách tạo	Mục đích
Sạch	Ghép các câu tổng hợp, không thêm nhiễu	Mức chuẩn
Nhiễu trắng 20 / 10 / 5 dB	Cộng nhiễu Gauss theo tỷ số tín hiệu trên nhiễu (SNR)	Micro kém, quạt, điều hoà
Đông người 10 dB	Cộng tiếng nói chuyện nền của 3 giọng khác (6 câu không liên quan, lặp lại chồng nhau)	Phòng họp mở, nhiều người nói xung quanh
Nói chồng + đoạn nhiễu	4 chỗ người sau chen vào 1,2 giây cuối câu người trước; chèn 8 giây chỉ có nhiễu nền; nhiễu nhẹ 25 dB	Kiểm tra phát hiện nói chồng và hiện tượng ảo giác

Cách dựng dữ liệu bằng giọng tổng hợp cho phép kiểm soát hoàn toàn đáp án và điều kiện nhiễu, nhưng có hạn chế là giọng tổng hợp đều và rõ hơn giọng nói tự nhiên, đồng thời đọc thuật ngữ tiếng Anh theo cách phát âm tiếng Việt. Hạn chế này được bàn ở cuối chương.

4.3. Độ đo đánh giá

Tỷ lệ lỗi từ (WER) và tỷ lệ lỗi ký tự (CER). WER được tính bằng khoảng cách chỉnh sửa giữa chuỗi tiếng của bản nhận dạng và bản chuẩn [17], sau khi đưa về chữ thường và bỏ dấu câu. Do Whisper viết số bằng chữ số ("thứ 4", "12") còn bản chuẩn viết bằng chữ ("thứ Tư", "mười hai"), nhóm báo cáo thêm WER sau khi chuẩn hoá cách viết số (đọc số 0–99 thành chữ, thứ trong tuần thành chữ). Đây là bước chuẩn hoá văn bản thông thường khi đánh giá nhận dạng tiếng nói, vì hai cách viết mang cùng nội dung. Trong chương này, "WER" được hiểu là WER đã chuẩn hoá số, trừ khi ghi khác.

Độ đúng thuật ngữ. Số lần thuật ngữ trong danh sách ở trên được nhận dạng đúng nguyên dạng, trên tổng 24 lần xuất hiện.

Độ đúng người nói. Với mỗi đoạn transcript đầu ra, tính thời lượng trùng với từng câu chuẩn; tìm cách ghép một-một tốt nhất giữa nhãn máy ("Người nói 1, 2…") và người thật; độ đúng là tỷ lệ thời lượng được gán đúng người trên tổng thời lượng trùng. Độ đo này tương ứng với phần "nhầm người" của DER, không tính phần bỏ sót hay báo thừa lời nói (vốn do Whisper quyết định).

Phát hiện nói chồng. Số vùng nói chồng chuẩn có ít nhất một đoạn được gắn cờ trùng lên (độ nhạy), và số đoạn bị gắn cờ trên các bản ghi không có nói chồng (báo nhầm).

Điểm trích xuất biên bản. Bộ chấm tự động gồm 15 tiêu chí: 8 công việc với người thực hiện và thời hạn đúng (kể cả việc đã đổi người, hạn đã đính chính), 1 tiêu chí gán đúng tên Phương cho "Người nói 4", 2 quyết định, 3 vấn đề còn mở và tóm tắt viết bằng tiếng Việt. Ngoài ra có 7 "bẫy", mỗi bẫy dính bị trừ 1 điểm: giữ hạn 15/11 chưa đính chính, giao nhắc lịch cho Nam, biến MoMo hoặc chuyện bóng đá thành công việc, coi đề xuất Firebase hoặc nâng gói máy chủ là quyết định, tự bịa thời hạn. Bộ chấm khớp theo từ khoá nên khá nghiêm: một công việc đúng nhưng ghi "thứ 4" thay vì "thứ Tư" vẫn bị coi là thiếu; với đầu ra từ transcript do máy nhận dạng, nhóm báo cáo cả điểm nghiêm ngặt và điểm sau khi chuẩn hoá cách viết thứ trong tuần.

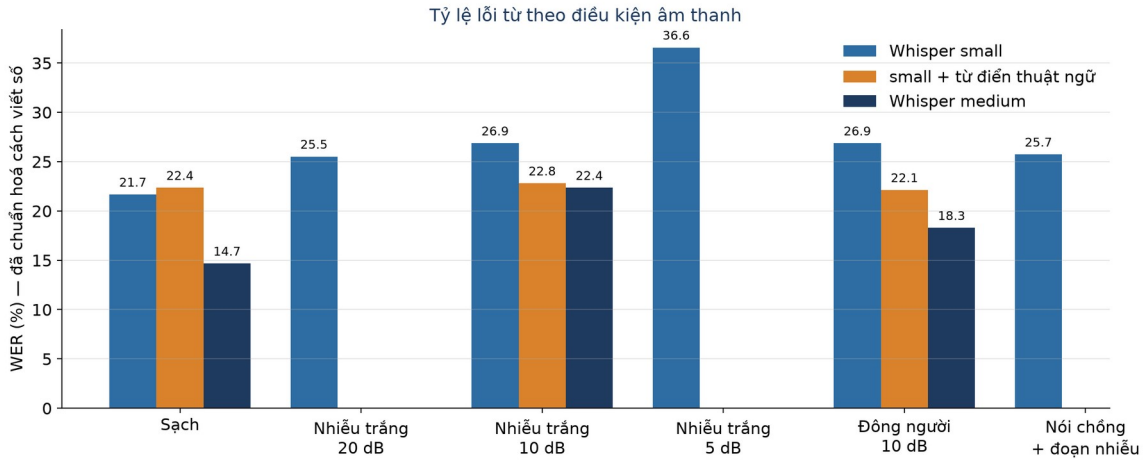
Thời gian và chi phí. Thời gian thực thi từng bước và hệ số thời gian thực (RTF, thời gian xử lý chia thời lượng âm thanh); chi phí LLM tính theo số token và bảng giá của nhà cung cấp.

4.4. Kết quả nhận dạng tiếng nói

Bảng 4.3 và Hình 4.1 trình bày kết quả nhận dạng của Whisper small (cấu hình mặc định của hệ thống), Whisper small có từ điển thuật ngữ, và Whisper medium.

Bảng 4.3. *Kết quả nhận dạng tiếng nói theo điều kiện âm thanh*

Điều kiện	Cấu hình	WER thô (%)	WER chuẩn hoá số (%)	CER (%)	Thuật ngữ đúng
Sạch	small	26,6	21,7	15,8	2/24
Sạch	small + từ điển	26,9	22,4	18,8	11/24
Sạch	medium	18,7	14,7	10,9	7/24
Nhiễu trắng 20 dB	small	30,0	25,5	16,8	3/24
Nhiễu trắng 10 dB	small	31,1	26,9	19,7	7/24
Nhiễu trắng 10 dB	small + từ điển	27,1	22,8	17,9	16/24
Nhiễu trắng 10 dB	medium	26,0	22,4	14,2	10/24
Nhiễu trắng 5 dB	small	40,9	36,6	25,5	6/24
Đông người 10 dB	small	31,6	26,9	18,5	3/24
Đông người 10 dB	small + từ điển	26,6	22,1	16,2	12/24
Đông người 10 dB	medium	22,6	18,3	14,3	7/24
Nói chồng + đoạn nhiễu	small	30,9	25,7	19,4	2/24



Hình 4.1. *Tỷ lệ lỗi từ (đã chuẩn hoá số) theo điều kiện âm thanh và cấu hình*

Với cấu hình mặc định (small, int8, CPU), WER ở điều kiện sạch là 21,7% và CER là 15,8%. Mức lỗi tăng dần theo nhiễu: khoảng 25–27% ở SNR 20–10 dB và ở nhiễu đông người, lên 36,6% khi nhiễu rất mạnh (SNR 5 dB). Nghĩa là mô hình chịu được nhiễu vừa phải mà không sụp đổ, nhưng chất lượng giảm rõ khi tín hiệu kém.

Phân tích lỗi cho thấy nguồn lỗi lớn nhất là **thuật ngữ tiếng Anh**: ở điều kiện sạch, Whisper small chỉ nhận đúng 2/24 lần xuất hiện thuật ngữ. "PostgreSQL" bị nhận thành "POSTEGARER-ESQUILLER", "Swagger" thành "so gơ", "eSMS" thành "EAS-MS", "MoMo" thành "mô mô". Một phần nguyên nhân nằm ở dữ liệu: giọng tổng hợp đọc thuật ngữ theo cách phát âm tiếng Việt, khác cách nói của người thật. Nguồn lỗi thứ hai là **dấu thanh và các từ gần âm** ("chốt" thành "chất", "đăng nhập" thành "đang nhập", "cuối tuần này" thành "cuối tôm nay"), điển hình của mô hình đa ngôn ngữ cỡ nhỏ với tiếng Việt.

Từ điển thuật ngữ (đưa danh sách thuật ngữ của dự án vào bước giải mã qua tham số hotwords của faster-whisper) cải thiện mạnh độ đúng thuật ngữ: từ 2 lên 11/24 ở điều kiện sạch, từ 7 lên 16/24 ở nhiễu trắng 10 dB và từ 3 lên 12/24 ở nhiễu đông người. Ở hai điều kiện nhiễu, WER giảm khoảng 4–5 điểm phần trăm. Tuy nhiên ở điều kiện sạch, WER tổng thể không giảm (22,4% so với 21,7%) và CER tăng nhẹ, do từ điển đôi khi kéo cả những đoạn không chứa thuật ngữ về phía các từ trong danh sách. Kết luận rút ra là từ điển thuật ngữ đáng dùng khi mục tiêu là giữ đúng tên công nghệ, sản phẩm trong biên bản, và nên được cấu hình theo từng dự án thay vì một danh sách chung.

Mô hình medium giảm WER ở điều kiện sạch từ 21,7% xuống 14,7% (giảm khoảng một phần ba) và CER từ 15,8% xuống 10,9%; ở nhiễu trắng 10 dB, WER giảm từ 26,9% xuống 22,4% và ở nhiễu đông người từ 26,9% xuống 18,3%, nhưng chạy chậm hơn khoảng 2,7 lần trên CPU (xem mục thời gian xử lý). Đây là đánh đổi điển

hình giữa độ chính xác và tài nguyên tính toán; với máy chủ có GPU, mô hình medium hoặc lớn hơn là lựa chọn hợp lý hơn.

4.5. Kết quả tách người nói và phát hiện nói chồng

Bảng 4.4. *Kết quả tách người nói (Whisper small)*

Điều kiện	Số người tìm được / thực tế	Độ đúng người nói (%)	Đoạn bị gắn cờ nói chồng
Sạch	4/4	99,5	8 / 35 đoạn (không có nói chồng thật)
Nhiễu trắng 20 dB	4/4	98,9	8 / 37
Nhiễu trắng 10 dB	5/4	94,8	11 / 44
Nhiễu trắng 5 dB	7/4	88,0	10 / 40
Đông người 10 dB	7/4	84,9	8 / 48
Nói chồng + đoạn nhiễu	5/4	93,4	1 / 42 (không trùng vùng nói chồng nào trong 4 vùng)

Kết quả ở Bảng 4.4 cho thấy mô-đun tách người nói hoạt động tốt ở điều kiện sạch và nhiễu nhẹ: tìm đúng 4 người và gán đúng người cho 99,5% thời lượng lời nói. Khi nhiễu tăng, mô-đun có xu hướng **tách thừa** (5–7 cụm cho 4 người): một số cửa sổ nhiễu nặng tạo vector đặc trưng lệch khỏi cụm của người nói và hình thành cụm nhỏ riêng. Nhờ bước gộp cụm nhỏ dưới 2 giây và cách ghép tối ưu, độ đúng theo thời gian vẫn giữ ở mức 85–95%, tức là phần lớn lời nói vẫn được gán đúng người; các "người thừa" chiếm ít thời lượng và người dùng có thể gộp lại khi gán tên.

Kết quả **phát hiện nói chồng là không đạt**. Trên bản ghi có 4 vùng nói chồng, mô-đun không gắn cờ đúng vùng nào; ngược lại, trên các bản ghi không hề có nói chồng, 8–11 đoạn vẫn bị gắn cờ. Nguyên nhân nằm ở thiết kế ngưỡng thích nghi (trung vị cộng hai lần độ lệch tuyệt đối trung vị của độ tương đồng với người nói thứ hai): ngưỡng tương đối luôn đẩy phần đuôi của phân bố lên thành cờ, kể cả khi không có nói chồng; trong khi các vùng nói chồng ngắn (1,2 giây) bị cửa sổ 1,5 giây làm loãng. Đây là phát hiện quan trọng của thực nghiệm: chức năng này cần thiết kế lại bằng mô hình phát hiện nói chồng chuyên dụng hoặc ngưỡng tuyệt đối được hiệu chỉnh trên dữ liệu thật, và trước mắt không nên hiển thị cờ "nói chồng" cho người dùng như một tín hiệu tin cậy. Theo kết quả này, cờ đã được tắt mặc định trong hệ thống.

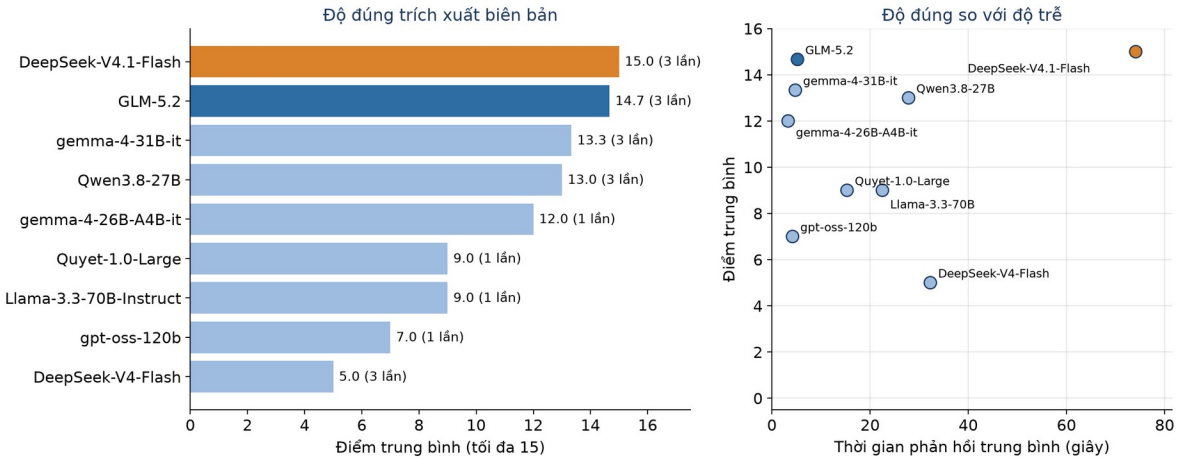
Về **hiện tượng ảo giác**, trong đoạn 8 giây chỉ có nhiễu nền, hệ thống không sinh ra đoạn văn bản nào. Kết quả này phù hợp với các biện pháp đã áp dụng (lọc khoảng lặng bằng VAD, bỏ đoạn bịa giữa khoảng lặng dài, không mượn ngữ cảnh câu trước, lọc câu rác thường gặp).

4.6. Lựa chọn mô hình ngôn ngữ lớn

Nhóm thử tất cả các LLM mà khoá API được cấp quyền trên FPT AI Factory. Tám mô hình thuộc nhóm "bán lại" (Azure gpt-5.6, Google Gemini, Alibaba Qwen/DeepSeek Pro, Z.ai GLM-5.3, Minimax) trả lỗi 402 do tài khoản chưa nạp tiền cho nhóm này; các mô hình này đồng thời gửi dữ liệu ra ngoài hạ tầng FPT nên không được ưu tiên. Chín mô hình do FPT tự vận hành được đánh giá trên transcript chuẩn của kịch bản, cùng một prompt hệ thống, nhiệt độ 0,2. Năm mô hình tốt nhất ở lượt đầu được chạy lại ba lần để đánh giá độ ổn định. Kết quả ở Bảng 4.5 và Hình 4.2.

Bảng 4.5. *So sánh các LLM trên bài trích xuất biên bản (transcript chuẩn, tối đa 15 điểm)*

Mô hình	Số lần chạy	Điểm từng lần	Trung bình	Thời gian TB (s)	Chi phí/lần (USD)
DeepSeek-V4.1-Flash	3	15, 15, 15	15,0	74,1	0,0052
GLM-5.2	3	15, 14, 15	14,7	5,2	0,0063
gemma-4-31B-it	3	14, 13, 13	13,3	4,8	0,0006
Qwen3.8-27B	3	13, 12, 14	13,0	27,8	0,0036
gemma-4-26B-A4B-it	1	12	12,0	3,3	0,0006
Quyet-1.0-Large	1	9	9,0	15,3	0,0001
Llama-3.3-70B-Instruct	1	9	9,0	22,5	0,0006
gpt-oss-120b	1	7	7,0	4,2	0,0007
DeepSeek-V4-Flash	3	0, 0, 15	5,0	32,3	0,0024



Hình 4.2. *Độ đúng trích xuất và độ trễ của các LLM trên FPT AI Factory*

DeepSeek-V4.1-Flash là mô hình duy nhất đạt trọn 15/15 điểm ở cả ba lần chạy, không dính bẫy nào: nhận ra đề xuất Firebase chưa phải quyết định, lấy đúng hạn 18/11 sau đính chính, giao việc nhắc lịch cho Hùng thay vì Nam, bỏ qua chuyện bóng đá và suy ra "Người nói 4" là Phương. Đổi lại, đây là mô hình suy luận nên mất 45–90 giây mỗi lần. GLM-5.2 đạt gần tương đương (14,7) với thời gian chỉ khoảng 5 giây; lỗi duy nhất là một lần bỏ sót việc bị đổi người thực hiện. Các mô hình nhỏ hơn (gemma, Qwen 27B) thường bỏ sót các quyết định dạng phủ định ("chưa làm MoMo") hoặc công việc gắn với người chủ trì. Các mô hình đa năng thế hệ trước (Llama 3.3 70B, gpt-oss-120b) và mô hình tiếng Việt Quyet-1.0-Large bỏ sót nhiều công việc.

Thực nghiệm cũng phát hiện hai vấn đề kỹ thuật quan trọng cho việc tích hợp. Thứ nhất, khi bật chế độ ép đầu ra JSON (response_format), DeepSeek-V4.1-Flash trả về nội dung rỗng (toàn bộ đầu ra nằm ở phần suy luận) và DeepSeek-V4-Flash trả về cấu trúc gọi hàm lặp lại transcript; vì vậy hệ thống không dùng chế độ này mà tự bóc và kiểm tra JSON. Thứ hai, DeepSeek-V4-Flash không ổn định: hai trong ba lần chạy rơi vào vòng lặp sinh văn bản đến khi hết giới hạn token.

Từ kết quả trên, hệ thống chọn **DeepSeek-V4.1-Flash làm mô hình chính** vì độ chính xác và độ ổn định cao nhất, chi phí khoảng 0,005 USD (khoảng 130 đồng) mỗi cuộc họp; độ trễ một phút không ảnh hưởng vì phân tích chạy nền sau bước phiên âm. **GLM-5.2 làm mô hình dự phòng** khi mô hình chính lỗi, quá thời gian hoặc trả JSON hỏng. Cả hai đều do FPT tự vận hành, dữ liệu cuộc họp không rời hạ tầng trong nước.

4.7. Đánh giá đầu-cuối từ âm thanh đến công việc

Ở thực nghiệm này, transcript do Whisper nhận dạng và mô-đun tách người nói gán nhãn được đưa thẳng vào bộ phân tích với mô hình đã chọn, đúng như luồng chạy thật. Mỗi transcript được phân tích hai lần: với nhãn tự động ("Người nói 1…4") và với tên thật được gán theo cách ghép người nói tốt nhất, mô phỏng thao tác "gán tên người nói" của người dùng trên giao diện. Bảng 4.6 trình bày điểm nghiêm ngặt và điểm sau chuẩn hoá cách viết thứ trong tuần.

Bảng 4.6. *Điểm trích xuất khi đầu vào là transcript do máy nhận dạng (tối đa 15)*

Điều kiện	Cấu hình STT	Nhãn tự động	Đã gán tên	Mô hình phân tích
Transcript chuẩn (đối chứng)	—	—	15 / 15	DeepSeek-V4.1-Flash
Sạch	small	10 / 11	12 / 13	DeepSeek-V4.1-Flash
Sạch	small + từ điển	11 / 12	9 / 10	DeepSeek-V4.1-Flash
Sạch	medium	11 / 11	15 / 15	DeepSeek-V4.1-

Điều kiện	Cấu hình STT	Nhãn tự động	Đã gán tên	Mô hình phân tích
				Flash
Nhiễu trắng 20 dB	small	10 / 11	10 / 11	DeepSeek-V4.1-Flash
Nhiễu trắng 10 dB	small	8 / 9	12 / 13	DeepSeek-V4.1-Flash
Nhiễu trắng 10 dB	small + từ điển	6 / 6	13 / 13	DeepSeek-V4.1-Flash
Nhiễu trắng 10 dB	medium	12 / 12	14 / 14	DeepSeek-V4.1-Flash
Nhiễu trắng 5 dB	small	5 / 5	8 / 8	GLM-5.2 (dự phòng)
Đông người 10 dB	small	8 / 9	11 / 12	DeepSeek-V4.1-Flash
Đông người 10 dB	small + từ điển	8 / 8	13 / 14	DeepSeek-V4.1-Flash
Đông người 10 dB	medium	12 / 12	13 / 13	DeepSeek-V4.1-Flash
Nói chồng + đoạn nhiễu	small	6 / 7	13 / 14	DeepSeek-V4.1-Flash

Ghi chú: mỗi ô dạng "nghiêm ngặt / sau chuẩn hoá cách viết thứ"; mỗi điều kiện chạy một lần.

Có ba nhận xét chính. Thứ nhất, so với 15/15 trên transcript chuẩn, điểm trên transcript máy giảm còn khoảng 8–13 điểm với Whisper small: lỗi nhận dạng lan sang bước trích xuất, đặc biệt ở các công việc gắn với thuật ngữ bị nghe sai (hạn "cuối tuần này" bị nghe thành "cuối tôm nay" nên công việc viết tài liệu mất thời hạn đúng). Thứ hai, **gán tên người nói giúp tăng điểm rõ rệt** (trung bình khoảng 3 điểm ở các điều kiện có nhiễu): khi chỉ có nhãn "Người nói 1…4", mô hình khó xác định ai nhận việc nếu câu giao việc không nêu tên. Điều này khẳng định giá trị của bước gán tên người nói; từ kết quả này, hệ thống được bổ sung thao tác "phân tích lại" để chạy lại AI sau khi người dùng gán tên. Thứ ba, chất lượng nhận dạng tốt hơn kéo theo trích xuất tốt hơn: với Whisper medium ở điều kiện sạch, hệ thống đạt lại 15/15 điểm khi đã gán tên.

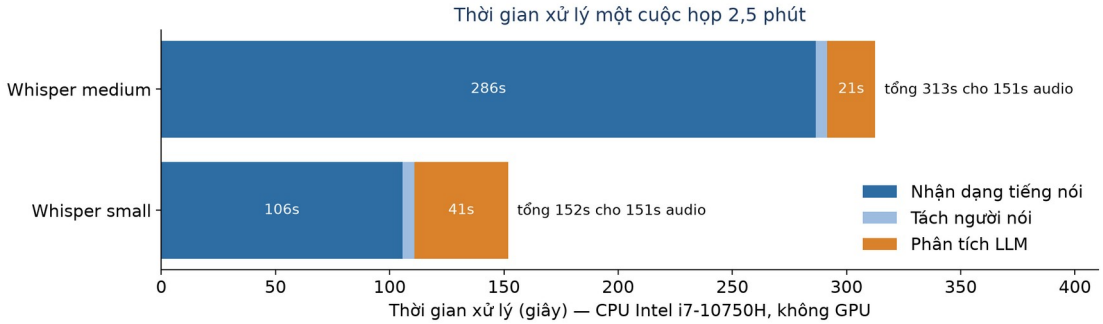
Một quan sát đáng chú ý là LLM tự sửa được nhiều lỗi nhận dạng nhờ ngữ cảnh: transcript ghi "phai bây sơ", "EAS-MS", "so gơ", nhưng công việc được trích xuất lại ghi đúng "Firebase", "ESMS", "Swagger". Ngược lại, LLM không thể khôi phục thông tin đã mất hẳn (thời hạn bị nghe sai). Ở điều kiện nhiễu 5 dB, mô hình chính suy luận quá lâu nên cổng dịch vụ trả lỗi hết thời gian chờ (HTTP 524) và hệ thống tự chuyển sang GLM-5.2, chứng tỏ cơ chế dự phòng hoạt động đúng; đồng thời cho thấy cần

chuyển lời gọi LLM sang chế độ truyền luồng. Sau thực nghiệm, lời gọi đã được chuyển sang chế độ luồng và kiểm chứng lại với cả hai mô hình trên API thật.

Các ô của Bảng 4.6 chỉ chạy một lần nên chịu ảnh hưởng của tính ngẫu nhiên của LLM (ví dụ ô "small + từ điển, sạch, đã gán tên" chỉ đạt 9 điểm dù nhận dạng thuật ngữ tốt hơn); cần nhiều lần chạy hơn để so sánh các cấu hình STT với độ tin cậy thống kê.

4.8. Thời gian xử lý, chi phí và hiệu quả tiết kiệm thời gian

4.8.1. Thời gian xử lý



Hình 4.3. *Thời gian xử lý từng bước cho bản ghi 151 giây (CPU, không GPU)*

Hình 4.3 và Bảng 4.7 cho thấy với cấu hình mặc định, hệ thống xử lý bản ghi 151 giây trong khoảng 150 giây: phiên âm chiếm phần lớn (khoảng 106 giây), tách người nói chỉ khoảng 5 giây, phân tích LLM khoảng 41–45 giây. Hệ số thời gian thực của bước phiên âm và tách người nói là 0,73, tức là trên laptop không GPU, hệ thống xử lý nhanh hơn thời gian thực một chút. Với mô hình medium, riêng bước phiên âm mất 287 giây (RTF khoảng 1,9). Ngoại suy tuyến tính, một cuộc họp 60 phút cần khoảng 44 phút xử lý âm thanh với mô hình small hoặc khoảng 2 giờ với mô hình medium trên phần cứng này; trên máy chủ có GPU, thời gian này giảm nhiều lần. Vì toàn bộ xử lý chạy nền và bất đồng bộ, người dùng không phải chờ trước màn hình.

Bảng 4.7. *Thời gian xử lý bản ghi 151 giây ở điều kiện sạch*

Bước	Whisper small	Whisper medium
Nạp mô hình (một lần mỗi tiến trình)	1,8 s	5,5 s
Phiên âm (2 lượt, nhận diện ngôn ngữ)	105,6 s	286,5 s
Tách người nói	5,3 s	5,0 s
Phân tích LLM (DeepSeek-V4.1-Flash)	40,9–44,6 s	21,1–33,9 s
Hệ số thời gian thực (phiên âm + tách người nói)	0,73	1,93

4.8.2. Chi phí

Chi phí LLM cho một lần phân tích bản họp thử nghiệm (khoảng 1.700 token đầu vào và 2.900–5.600 token đầu ra, phần lớn là token suy luận) là khoảng 0,004–0,007 USD. Với cuộc họp 60 phút (khoảng 15.000 token transcript), chi phí ước tính dưới 0,03 USD mỗi cuộc họp. Phiên âm và tách người nói chạy trên máy chủ của tổ chức nên không phát sinh chi phí API.

4.8.3. Ước tính hiệu quả tiết kiệm thời gian

Đề tài chưa có điều kiện đo trên người dùng thật, vì vậy hiệu quả tiết kiệm thời gian dưới đây là **ước tính dựa trên giả định**, cần được kiểm chứng bằng nghiên cứu người dùng ở giai đoạn sau. Giả định cho một cuộc họp 60 phút của nhóm 4–6 người: không có hệ thống, thư ký vừa họp vừa ghi chép, sau họp mất khoảng 30 phút soạn biên bản và khoảng 10 phút tạo, giao công việc trên công cụ quản lý. Có hệ thống, máy tự xử lý nền; trưởng nhóm mất khoảng 10 phút rà soát biên bản nháp và duyệt các đề xuất, mỗi đề xuất đã có sẵn câu trích dẫn để đối chiếu nhanh.

Bảng 4.8. *Ước tính thời gian công sức con người cho một cuộc họp 60 phút*

Công việc	Không có hệ thống	Có MeetingMind
Ghi chép trong lúc họp	Một người ghi liên tục	Không cần (máy ghi âm hoặc đọc phụ đề)
Soạn biên bản sau họp	~30 phút	~5 phút rà soát bản nháp
Tạo và giao công việc	~10 phút	~5 phút duyệt đề xuất
Tổng thời gian sau họp	~40 phút	~10 phút

Theo giả định này, mỗi cuộc họp tiết kiệm khoảng 30 phút công sức sau họp, và người ghi chép được giải phóng để tham gia thảo luận. Với nhóm họp 3 buổi mỗi tuần, mức tiết kiệm khoảng 1,5 giờ mỗi tuần. Thực nghiệm ở mục trước cũng cho thấy bản nháp không hoàn hảo (đặc biệt khi âm thanh kém), nên bước rà soát của con người là bắt buộc chứ không phải hình thức; thiết kế hàng đợi duyệt với câu trích dẫn đi kèm giúp bước này nhanh hơn so với đọc lại toàn bộ transcript.

4.9. Kiểm thử chức năng hệ thống

Bên cạnh đánh giá độ chính xác, nhóm kiểm thử các kịch bản chức năng trên môi trường Docker Compose với dịch vụ thật (PostgreSQL, RabbitMQ, MinIO, ai-worker, LLM qua FPT AI Factory, integration-service với tài khoản Trello và bot Telegram). Kết quả tóm tắt ở Bảng 4.9.

Bảng 4.9. *Kết quả các kịch bản kiểm thử chức năng*

Kịch bản	Các bước kiểm tra	Kết quả
Đường phụ đề	Mở phiên, đẩy 30 đoạn phụ đề, chốt	Đạt: biên bản nháp phiên bản 1, 16

Kịch bản	Các bước kiểm tra	Kết quả
trực tiếp	phiên → phân tích LLM → biên bản nháp và hàng đợi duyệt	đề xuất (8 công việc, 5 quyết định, 3 vấn đề mở), đúng các tình huống bẫy
Đường tải tệp ghi âm	Tạo cuộc họp, tải tệp WAV, phiên âm Whisper + tách người nói, phân tích LLM	Đạt (xem mô tả bên dưới)
Duyệt đề xuất	Duyệt, sửa, từ chối đề xuất trên giao diện → sinh công việc, quyết định chính thức	Đạt
Đồng bộ Trello và Telegram	Duyệt công việc "Viết tài liệu API bằng Swagger" → integration-service tạo thẻ Trello, gửi tin Telegram → Core lưu kết quả → giao diện	Đạt: sau 3,4 s có thẻ trên bảng "MeetingMind" (danh sách "Việc từ cuộc họp" do hệ thống tự tạo) và tin Telegram kèm đường dẫn thẻ; giao diện hiện liên kết "Trello" và nhãn "Đã báo Telegram". Khi chưa có khoá Trello: Trello `skipped`, Telegram vẫn gửi sau khoảng 2 s
Bot Google Meet	Mời bot bằng tài khoản Google riêng → vào phòng → chờ duyệt → ghi âm → tải lên	Một phần: bot vào phòng và nhận đúng trạng thái chờ duyệt; kịch bản ghi âm dài sau khi được cho vào chưa chạy trọn trong thời gian đề tài
Mô hình dự phòng	Mô hình chính bị cắt kết nối (lỗi 524)	Đạt: tự chuyển sang GLM-5.2, vẫn trả kết quả
Phân tích lại theo tên người nói	Gán "Người nói 1" cho một thành viên → phân tích lại	Đạt: lô đề xuất mới ghi người thực hiện bằng tên thành viên thay cho nhãn, mất khoảng 21–25 s
Gọi LLM theo luồng	Chạy lại bài trích xuất với mô hình chính và dự phòng ở chế độ luồng	Đạt: cả hai mô hình trả kết quả đúng định dạng
Phân quyền	Truy cập API không đăng nhập, đổi mật khẩu sai mật khẩu cũ, tự đặt chủ sở hữu workspace, đọc ghi chú riêng qua tuyến CRUD	Đạt sau khi vá các lỗi phát hiện trong quá trình kiểm thử (tài khoản thường gọi `/note` nhận 403)
Kiểm thử đơn vị	pytest (ai-service, bot-worker), node test (extension), Jest (integration-service)	Đạt: 44 + 3 + 7 + 20 ca

Kịch bản tải tệp ghi âm được chạy với bản ghi "nói chồng + đoạn nhiễu" (152 giây) qua đúng API của hệ thống: tạo cuộc họp trực tiếp, tải tệp lên MinIO, Core phát lệnh phiên âm, ai-worker phiên âm và tách người nói, Core lưu transcript phiên bản 1 (42 đoạn, 5 nhãn người nói) rồi phát lệnh phân tích, ai-worker gọi LLM, Core tạo biên bản nháp cùng hàng đợi duyệt gồm 8 công việc, 4 quyết định và 3 vấn đề còn mở. Hai lần chạy mất 171 giây và 148 giây từ lúc tải tệp đến khi có biên bản nháp. Lần chạy đầu phát hiện một lỗi: cuộc họp trực tiếp tạo bằng đường tải tệp vẫn ở trạng thái "đã lên lịch" sau khi có transcript, khiến giao diện ẩn tab transcript; lỗi được sửa bằng cách

chuyển cuộc họp sang "đã kết thúc" khi nhận transcript từ bản ghi. Kết quả phản ánh đúng các hạn chế đã đo: vài lỗi nhận dạng còn trong transcript, và vì người nói chưa được gán tên nên hai công việc mang nhãn "Người nói 1", "Người nói 4" thay vì tên thật, đúng như phân tích ở mục đánh giá đầu-cuối. Dữ liệu của kịch bản này được dùng cho các ảnh chụp giao diện ở chương Xây dựng hệ thống.

Kịch bản đồng bộ được chạy với tài khoản Trello và bot Telegram thật của nhóm. Ở lần duyệt đầu, khi chưa cấu hình khoá Trello, integration-service trả về Trello `skipped` (không gọi API) và Telegram `synced`, tin đến sau khoảng 2 giây; Core lưu kết quả và giao diện hiển thị đúng trạng thái. Sau khi cấu hình khoá Trello, duyệt công việc "Viết tài liệu API bằng Swagger" thì sau 3,4 giây thẻ được tạo trên bảng "MeetingMind", trong danh sách "Việc từ cuộc họp" do hệ thống tự tạo, tin Telegram kèm đường dẫn thẻ được gửi, Core lưu Trello `synced` (có đường dẫn thẻ) và Telegram `synced`. Kịch bản này xác nhận trọn chuỗi duyệt → outbox → RabbitMQ → integration-service → Trello, Telegram → Core → giao diện; các tình huống mất phản hồi và lỗi của một bên được kiểm tra bằng kiểm thử đơn vị.

4.10. Hạn chế của thực nghiệm

Kết quả trên cần được đọc cùng các hạn chế sau. Thứ nhất, dữ liệu được dựng bằng giọng tổng hợp từ một kịch bản: giọng tổng hợp rõ và đều hơn giọng thật nhưng đọc thuật ngữ tiếng Anh theo phát âm tiếng Việt, nên WER trên cuộc họp thật có thể cao hơn ở phần tiếng Việt và thấp hơn ở phần thuật ngữ. Thứ hai, quy mô nhỏ: một kịch bản, bốn người nói, mỗi điều kiện âm thanh chạy một lần, nên chênh lệch vài điểm giữa các cấu hình chưa có ý nghĩa thống kê. Thứ ba, bộ chấm trích xuất khớp theo từ khoá, có thể chấm thiếu những đầu ra đúng nghĩa nhưng diễn đạt khác. Thứ tư, chưa có giọng tiếng Anh tự nhiên để đánh giá nhận diện ngôn ngữ trên hội thoại xen Việt – Anh; cơ chế này mới được kiểm tra bằng kiểm thử đơn vị. Thứ năm, hiệu quả tiết kiệm thời gian mới là ước tính. Hướng khắc phục là thu thập một bộ ghi âm cuộc họp thật có sự đồng ý của người tham gia, gán nhãn thủ công và lặp lại toàn bộ quy trình đánh giá đã xây dựng.

4.11. Kết luận chương

Thực nghiệm cho thấy các thành phần AI của MeetingMind đạt mức dùng được cho mục đích hỗ trợ ghi biên bản, với con người rà soát ở bước cuối. LLM được chọn (DeepSeek-V4.1-Flash) trích xuất đúng hoàn toàn trên transcript chuẩn và ổn định qua các lần chạy, với chi phí không đáng kể. Tách người nói hoạt động tốt ở điều kiện sạch và chấp nhận được khi có nhiễu. Khâu yếu nhất là nhận dạng tiếng nói với mô hình nhỏ trên CPU, đặc biệt với thuật ngữ tiếng Anh; lỗi ở khâu này lan sang khâu trích

xuất. Các giải pháp đã được kiểm chứng định lượng gồm dùng mô hình lớn hơn (WER giảm từ 21,7% xuống 14,7%), từ điển thuật ngữ theo dự án (thuật ngữ đúng tăng từ 2 lên 11/24) và gán tên người nói trước khi phân tích. Chức năng đánh dấu nói chồng chưa đạt và cần thiết kế lại. Về chức năng, các kịch bản đầu-cuối đều đạt, trừ kịch bản ghi âm dài của bot chưa chạy trọn; luồng đồng bộ Trello và Telegram đã chạy với tài khoản thật.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết luận

Đề tài đặt mục tiêu xây dựng một hệ thống trợ lý cuộc họp ứng dụng nhận dạng tiếng nói và mô hình ngôn ngữ lớn, tự động hoá việc ghi biên bản, trích xuất công việc kèm người thực hiện và thời hạn, và đưa công việc vào quy trình quản lý của nhóm. Sau sáu tháng thực hiện, nhóm nghiên cứu đã hoàn thành sản phẩm MeetingMind ở dạng ứng dụng web chạy được trọn luồng từ âm thanh cuộc họp đến công việc đã được duyệt và được đẩy sang Trello, báo qua Telegram, cùng bộ số liệu thực nghiệm đánh giá từng thành phần AI. Các kết quả chính có thể tóm lược như sau.

Thứ nhất, về nghiên cứu tổng quan và thiết kế, đề tài đã hệ thống hoá các hướng tiếp cận nhận dạng tiếng nói, tách người nói và trích xuất thông tin bằng LLM, từ đó chọn kiến trúc nhiều đường thu nhận đổ về một luồng xử lý chung. Thiết kế hướng sự kiện với hàng đợi thông điệp, mẫu outbox và cơ chế chống xử lý lặp cho phép thay bộ phiên âm và bộ phân tích giả lập bằng mô hình thật ở cuối đề tài mà gần như không phải sửa luồng điều phối. Cơ sở dữ liệu lưu transcript và biên bản theo phiên bản, nên công việc đã duyệt luôn truy được về đúng câu nói gốc. Phân quyền theo vai trò kết hợp phạm vi tài nguyên (workspace, dự án, cuộc họp) với 54 quyền đã được hiện thực ở dịch vụ lõi.

Thứ hai, về thu nhận và xử lý âm thanh, hệ thống có ba đường đưa nội dung cuộc họp vào: tải tệp ghi âm, bot tự tham gia Google Meet để ghi âm, và tiện ích Chrome đọc phụ đề Google Meet mà không cần bot. Module phiên âm dùng Whisper (bản faster-whisper chạy CPU) với cơ chế nhận diện ngôn ngữ hai lượt cho hội thoại xen Việt – Anh và các biện pháp hạn chế ảo giác ở đoạn lặng hoặc nhiễu. Module tách người nói dựa trên vector đặc trưng giọng ERes2Net và gom cụm phân cấp. Trên bộ dữ liệu thực nghiệm, hệ thống phân biệt đúng 4/4 người nói với độ đúng theo thời gian 99,5% ở điều kiện sạch, giữ trên 84% ở các điều kiện nhiễu, và không sinh câu bịa trong đoạn chỉ có nhiễu nền.

Thứ ba, về phân tích biên bản, đề tài đã xây dựng bộ phân tích dùng LLM qua dịch vụ suy luận FPT AI Factory, có mô hình dự phòng, chuẩn hoá đầu ra và đối chiếu bằng chứng với transcript. Mô hình được chọn bằng thực nghiệm có kiểm soát trên chín mô hình: DeepSeek-V4.1-Flash đạt trọn 15/15 điểm ở cả ba lần chạy trên bài kiểm tra có các tình huống bẫy (đề xuất khác quyết định, đính chính thời hạn, đổi người thực hiện, chuyện ngoài lề), với chi phí khoảng 0,005 USD mỗi cuộc họp. Kết quả AI không được ghi thẳng thành dữ liệu chính thức mà đi vào hàng đợi duyệt; chỉ khi trưởng nhóm duyệt, công việc và quyết định mới được tạo. Công việc đã duyệt

được dịch vụ tích hợp tự động tạo thành thẻ Trello kèm người thực hiện, hạn và câu trích nguồn, đồng thời báo qua bot Telegram; cơ chế chống tạo trùng ba lớp bảo đảm thử lại không sinh thẻ thứ hai.

Thứ tư, về sản phẩm, hệ thống được đóng gói bằng Docker Compose gồm 12 dịch vụ, có giao diện web Next.js với bảng điều khiển, danh sách và chi tiết cuộc họp, transcript theo người nói, biên bản, hàng đợi duyệt và bảng công việc. Các kiểm thử đơn vị (44 ca ở dịch vụ AI, 3 ca ở bot, 7 ca ở tiện ích, 20 ca ở dịch vụ tích hợp) và các kịch bản kiểm thử đầu-cuối trên môi trường Docker đều chạy thành công, trong đó luồng đồng bộ tạo được thẻ Trello và tin Telegram thật sau 3,4 giây kể từ khi duyệt.

Đối chiếu với đề cương, cả năm nội dung đều đạt yêu cầu. Ở nội dung 4, ngoài tác tử AI tạo biên bản và trích xuất công việc, yêu cầu đồng bộ công việc lên "Jira/Trello" được đáp ứng bằng Trello và yêu cầu cảnh báo công việc mới qua Telegram đã được hiện thực, cả hai đều đã kiểm thử với tài khoản thật; đồng bộ Jira chưa được làm và phần tích hợp mới ở mức MVP, như nêu ở mục Hạn chế.

2. Hạn chế

Bên cạnh các kết quả trên, đề tài còn những hạn chế cần nêu rõ:

- Độ chính xác nhận dạng tiếng nói với mô hình Whisper small trên CPU còn hạn chế: tỷ lệ lỗi từ khoảng 22% ở điều kiện sạch và tăng lên khoảng 37% khi nhiễu mạnh, chủ yếu do thuật ngữ tiếng Anh (tên công nghệ, sản phẩm) và một số lỗi dấu thanh. Lỗi này lan sang bước trích xuất: điểm trích xuất trên transcript do máy nhận dạng thấp hơn đáng kể so với trên transcript chuẩn.
- Cơ chế đánh dấu nói chồng dùng ngưỡng tương đối nên báo nhầm nhiều đoạn trên bản ghi không có nói chồng và không bắt được các đoạn nói chồng ngắn trong bộ thử; chức năng này đã được tắt mặc định và cần thiết kế lại trước khi dùng thực tế.
- Bộ dữ liệu thực nghiệm được dựng bằng giọng tổng hợp (TTS) từ một kịch bản cuộc họp, nên chưa phản ánh đầy đủ giọng nói tự nhiên, giọng địa phương, tiếng vọng phòng họp và cách nói ngập ngừng. Hiệu quả tiết kiệm thời gian mới được ước tính, chưa đo trên người dùng thật.
- Phần tích hợp mới ở mức MVP: mỗi nền tảng một kết nối cấu hình qua biến môi trường, chưa có màn hình kết nối OAuth và chọn bảng theo từng workspace; đồng bộ một chiều (tạo thẻ, chưa cập nhật trạng thái hai chiều); tin Telegram có thể bị gửi trùng khi mất phản hồi; chưa có đồng bộ Jira.

- Chưa có hỏi đáp RAG trên kho cuộc họp và bộ phát hiện rủi ro phía máy chủ; dữ liệu hội thoại chưa được mã hoá khi lưu trữ mà mới được bảo vệ bằng phân quyền.
- Một số điểm thiết kế chưa hiện thực đầy đủ: ghi outbox cùng giao dịch với dữ liệu nghiệp vụ, đường đi qua API Gateway, cơ chế tự thử lại job lỗi và hàng đợi thư chết (dead-letter queue).

3. Hướng phát triển

Từ các hạn chế trên, nhóm đề xuất các hướng phát triển theo thứ tự ưu tiên:

1. Hoàn thiện tích hợp: màn hình kết nối OAuth 2.0 và chọn bảng theo từng workspace thay cho cấu hình bằng biến môi trường; đồng bộ hai chiều trạng thái công việc qua webhook; bổ sung Jira theo cùng cơ chế job và chống tạo trùng đã có; nhắc việc sắp đến hạn qua Telegram.
2. Nâng chất lượng nhận dạng: chạy mô hình Whisper lớn hơn trên GPU, thử các bản Whisper tinh chỉnh cho tiếng Việt, và đưa từ điển thuật ngữ riêng của từng dự án vào bước giải mã.
3. Thay cơ chế đánh dấu nói chồng bằng mô hình phát hiện nói chồng chuyên dụng, và cho người dùng gán tên người nói một lần để dùng lại ở các cuộc họp sau (nhận dạng người nói theo giọng đã đăng ký).
4. Bổ sung hỏi đáp RAG có trích nguồn trên kho cuộc họp với kiểm soát quyền.
5. Đánh giá trên cuộc họp thật của các nhóm sinh viên và doanh nghiệp nhỏ, đo thời gian người dùng cần để rà soát biên bản do AI đề xuất so với tự viết, làm cơ sở định lượng hiệu quả tiết kiệm thời gian.

4. Kiến nghị

Kết quả đề tài có thể áp dụng trước hết cho các nhóm dự án phần mềm, nhóm nghiên cứu và câu lạc bộ sinh viên thường xuyên họp trực tuyến trên Google Meet, nơi việc ghi biên bản và giao việc sau họp đang làm thủ công. Với các tổ chức có yêu cầu bảo mật cao, kiến trúc của hệ thống cho phép chạy toàn bộ phần xử lý âm thanh trên máy chủ nội bộ và chỉ gọi LLM qua nhà cung cấp trong nước, hoặc thay bằng mô hình tự triển khai.

Nhóm nghiên cứu kính đề nghị Viện Khoa học Kỹ thuật Bưu điện xem xét nghiệm thu đề tài, và tạo điều kiện để nhóm tiếp tục phát triển các hướng nêu trên, đặc biệt là thử nghiệm hệ thống trên các cuộc họp thực tế trong Học viện. Nhóm xin chân thành cảm ơn TS. Lê Văn Hoàng đã tận tình hướng dẫn trong suốt quá trình thực hiện đề tài.

TÀI LIỆU THAM KHẢO

[1] Atlassian (2026), *Jira Cloud platform REST API v3*, https://developer.atlassian.com/cloud/jira/platform/rest/v3/, truy cập ngày 04/11/2026.

[2] Atlassian (2026), *Trello REST API*, https://developer.atlassian.com/cloud/trello/rest/, truy cập ngày 04/11/2026.

[3] Baevski A., Zhou H., Mohamed A., Auli M. (2020), "wav2vec 2.0: A framework for self-supervised learning of speech representations", *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 12449–12460.

[4] Bredin H., Yin R., Coria J. M., Gelly G., Korshunov P., Lavechin M., Fustes D., Titeux H., Bouaziz W., Gill M.-P. (2020), "pyannote.audio: neural building blocks for speaker diarization", *Proceedings of IEEE ICASSP 2020*, pp. 7124–7128.

[5] Brown T. B., Mann B., Ryder N., Subbiah M., Kaplan J., et al. (2020), "Language models are few-shot learners", *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 1877–1901.

[6] Carletta J., Ashby S., Bourban S., Flynn M., Guillemot M., Hain T., et al. (2005), "The AMI meeting corpus: A pre-announcement", *Machine Learning for Multimodal Interaction (MLMI 2005)*, LNCS 3869, Springer, pp. 28–39.

[7] Chen Y., Zheng S., Wang H., Cheng L., Chen Q., Qi J. (2023), "An enhanced Res2Net with local and global feature fusion for speaker verification", *Proceedings of Interspeech 2023*.

[8] DeepSeek-AI (2024), *DeepSeek-V3 Technical Report*, arXiv:2412.19437.

[9] Fireflies.ai (2026), *Fireflies.ai – AI notetaker for meetings*, https://fireflies.ai, truy cập ngày 04/11/2026.

[10] FPT Smart Cloud (2026), *FPT AI Factory – Serverless Inference documentation*, https://docs.fptcloud.com/docs/category/fpt-ai-factory/, truy cập ngày 04/11/2026.

[11] Google (2026), *Chrome Extensions – Manifest V3*, https://developer.chrome.com/docs/extensions, truy cập ngày 04/11/2026.

[12] Hardt D. (Ed.) (2012), *The OAuth 2.0 Authorization Framework*, RFC 6749, Internet Engineering Task Force (IETF).

[13] Hu V. C., Ferraiolo D., Kuhn R., Schnitzer A., Sandlin K., Miller R., Scarfone K. (2014), *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162.

[14] Ji Z., Lee N., Frieske R., Yu T., Su D., Xu Y., Ishii E., Bang Y. J., Madotto A., Fung P. (2023), "Survey of hallucination in natural language generation", *ACM Computing Surveys*, 55(12), pp. 1–38.

[15] k2-fsa (2026), *sherpa-onnx: Speech-to-text, text-to-speech, speaker diarization using next-gen Kaldi with onnxruntime*, https://github.com/k2-fsa/sherpa-onnx, truy cập ngày 04/11/2026.

[16] Kleppmann M. (2017), *Designing Data-Intensive Applications*, O'Reilly Media.

[17] Levenshtein V. I. (1966), "Binary codes capable of correcting deletions, insertions, and reversals", *Soviet Physics Doklady*, 10(8), pp. 707–710.

[18] Luong H.-T., Vu H.-Q. (2016), "A non-expert Kaldi recipe for Vietnamese speech recognition system", *Proceedings of the Third International Workshop on Worldwide Language Service Infrastructure and Second Workshop on Open Infrastructures and Analysis Frameworks for Human Language Technologies (WLSI/OIAF4HLT 2016)*, pp. 51–55.

[19] Microsoft (2026), *Playwright documentation*, https://playwright.dev, truy cập ngày 04/11/2026.

[20] Myśliwiec K. (2026), *NestJS documentation*, https://docs.nestjs.com, truy cập ngày 04/11/2026.

[21] Newman S. (2021), *Building Microservices: Designing Fine-Grained Systems*, 2nd ed., O'Reilly Media.

[22] Otter.ai (2026), *Otter.ai – AI meeting assistant*, https://otter.ai, truy cập ngày 04/11/2026.

[23] Park T. J., Kanda N., Dimitriadis D., Han K. J., Watanabe S., Narayanan S. (2022), "A review of speaker diarization: Recent advances with deep learning", *Computer Speech & Language*, 72, 101317.

[24] RabbitMQ (2026), *RabbitMQ documentation: Reliability guide, consumer acknowledgements and publisher confirms*, https://www.rabbitmq.com/docs, truy cập ngày 04/11/2026.

[25] Radford A., Kim J. W., Xu T., Brockman G., McLeavey C., Sutskever I. (2023), "Robust speech recognition via large-scale weak supervision", *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, PMLR 202, pp. 28492–28518.

[26] Richardson C. (2018), *Microservices Patterns: With Examples in Java*, Manning Publications.

[27] Sandhu R. S., Coyne E. J., Feinstein H. L., Youman C. E. (1996), "Role-based access control models", *IEEE Computer*, 29(2), pp. 38–47.

[28] Snyder D., Garcia-Romero D., Sell G., Povey D., Khudanpur S. (2018), "X-vectors: Robust DNN embeddings for speaker recognition", *Proceedings of IEEE ICASSP 2018*, pp. 5329–5333.

[29] SYSTRAN (2026), *faster-whisper: Faster Whisper transcription with CTranslate2*, https://github.com/SYSTRAN/faster-whisper, truy cập ngày 04/11/2026.

[30] Tactiq (2026), *Tactiq – AI meeting transcripts for Google Meet, Zoom and Teams*, https://tactiq.io, truy cập ngày 04/11/2026.

[31] Team GLM (2024), *ChatGLM: A family of large language models from GLM-130B to GLM-4 All Tools*, arXiv:2406.12793.

[32] Telegram (2026), *Telegram Bot API*, https://core.telegram.org/bots/api, truy cập ngày 04/11/2026.

[33] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser Ł., Polosukhin I. (2017), "Attention is all you need", *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pp. 5998–6008.

[34] W3C (2021), *WebRTC 1.0: Real-Time Communication Between Browsers*, W3C Recommendation, https://www.w3.org/TR/webrtc/.

[35] Wei J., Wang X., Schuurmans D., Bosma M., Ichter B., Xia F., Chi E., Le Q., Zhou D. (2022), "Chain-of-thought prompting elicits reasoning in large language models", *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 24824–24837.

[36] Zhong M., Yin D., Yu T., Zaidi A., Mutuma M., Jha R., Awadallah A. H., Celikyilmaz A., Liu Y., Qiu X., Radev D. (2021), "QMSum: A new benchmark for query-based multi-domain meeting summarization", *Proceedings of NAACL-HLT 2021*, pp. 5905–5921.